队伍编号	MCB260XXXX
赛道	B

吉隆泥石流的触发评估、预报与仿真

摘 要

2026 年 8 月 26 日吉隆口岸泥石流据报道疑由高位冰崩引发，当地并无暴雨，常规的降雨预报难以适用。本文以吉隆观测塔逐时资料为主，补充 ERA5 再分析、历史天气预报与 Copernicus DEM，并把同一流域 2025 年 7 月 8 日的冰湖溃决洪水作为第二个同类事件，建立关联排序、触发临界、概率预报与过程仿真模型。

针对问题一，建立了**熵权 TOPSIS 排序模型**。首先，由气象梯度、辐射、土壤与湍流观测构造 7 类 34 项逐日特征，分析事件前的边界层日变化与天气过程；然后，以异常程度、一致性、前后期变化量与标准化距平为指标，用熵权 TOPSIS 计算关联贴近度；最后，按物理类别筛选并做显著性检验。结果表明，关联性最高的 **5** 个因素依次为饱和水汽压差、气温日较差、夜间逆温频率、向下长波辐射与 30 日降水，2025 年事件前 3 日饱和水汽压差为 **1.01 kPa**，同期为 **0.61 kPa**，前两个因素在 **0.05** 水平上显著。

针对问题二，建立了**双因子临界模型**与**易发性评价模型**。首先，用订正后的 ERA5 降水检验雨强历时阈值，两次事件前 24 h 源区雨量仅 6.6 mm 和 2.0 mm，降雨不是主要触发因素；然后，以前期降水与源区正积温为双因子，由两次事件确定临界值，用留一检验和误报日比例筛选变量；最后，用随机森林评价历史泥石流沟的空间易发性。结果表明，临界条件为前 15 日降水不少于 **39.6 mm** 且源区 3 日正积温不少于 **22.2 °C·d**，两次事件全部报中，汛期误报日比例为 **1.36%**，易发性模型空间分块验证的 AUC 为 **0.92**。

针对问题三，建立了**滞后概率预报模型**。首先，把 1 至 5 d 历史预报订正为源区气温，按问题二的临界条件判定警戒日；然后，按源区气温日内正距平滞后 τ 分配触发时刻，由两次事件估计 $\tau = 15$ h；最后，蒙特卡洛抽样 2000 次求触发时间的 95% 置信区间，并结合问题四给出空间范围。结果表明，提前 12 h 起报时，两次事件的 95% 置信区间分别为 **7 月 7 日 18:25 至 9 日 08:41** 和 **8 月 25 日 22:11 至 27 日 15:12**，中位数与实际相差不超过 **1.1 h**；提前 96 h 以内的覆盖率为 **100%**，全部起报为 **85.7%**；预警时间为 **36 至 48 h**；预警范围为冰川末端以下 **24.3 km**、河道两侧 **214 m** 以内。

针对问题四，建立了**粒子仿真模型**。首先，由冰川冰厚积分确定释放体积；然后，在填洼后的 DEM 上用 Voellmy 摩擦计算冰岩崩、泥石流、山洪三个阶段，并计入沿程侵蚀；最后，做 60 次参数蒙特卡洛、收敛检验与建模方式对比。结果表明，释放 7.56×10^{6} m^3 冰体时前锋 **13.9 min** 到达口岸，口岸流深约 **3.8 m**，**14.0 至 23.7 km** 河段局部堆积超过 5 m，可能堵塞河道；参数抽样中 **50%** 的情景到达口岸。

关键词：熵权 TOPSIS　双因子临界条件　概率预报　蒙特卡洛　粒子仿真

目录

1 问题重述 1
1.1 问题背景 1
1.2 问题要求 1
2 问题分析 1
2.1 问题一的分析 1
2.2 问题二的分析 1
2.3 问题三的分析 1
2.4 问题四的分析 1
2.5 总体思路 2
3 模型假设 2
4 符号说明 3
5 数据准备 3
5.1 数据来源 3
5.2 研究区与两次事件 3
6 模型的建立与求解 5
6.1 问题一：熵权 TOPSIS 排序模型 5
6.2 问题二：双因子临界模型与易发性评价模型 10
6.3 问题三：滞后概率预报模型 15
6.4 问题四：粒子仿真模型 19
7 灵敏度分析 24
8 模型的评价、改进与推广 25
8.1 模型的优点 25
8.2 模型的缺点 25
8.3 改进与推广 25
参考文献 26
AI 工具使用声明 27
附录 27

1 问题重述

1.1 问题背景

泥石流是暴雨或冰雪融水激发的、挟带大量泥砂石块的特殊洪流，形成需要陡峻地形、丰富松散物质和短时大量水源三个条件[23]。2026 年 8 月 26 日 10 时 30 分，据报道尼泊尔一侧高位冰崩引发碎屑流，演变为泥石流冲击吉隆口岸，造成重大伤亡，并在下游形成堰塞湖。同一条 Lhende 河在 2025 年 7 月 8 日已因冰湖溃决冲毁口岸大桥，两次事件都不是当地暴雨所致[21,22]。

1.2 问题要求

问题一：基于气象梯度、辐射、土壤与湍流观测，分析大气边界层结构、天气过程及极端事件与吉隆泥石流的关联，给出关联性最高的 5 个因素。

问题二：补充卫星遥感、天气预报等数据，研究触发关键参数的临界条件，建立触发评估模型并讨论其时空可靠性。

问题三：建立预报模型，给出触发时间的 95% 置信区间、预警时间、区间覆盖率与空间距离范围。

问题四：构建吉隆泥石流仿真环境，设定关键参数，给出仿真过程与可视化结果。

2 问题分析

2.1 问题一的分析

问题一要求从观测中找出与泥石流关联最强的因素。首先，观测塔位于吉隆县城，坐标 28.86°N、85.29°E，海拔 4140 m，距口岸约 65 km，数据说明指出它只能反映区域背景[1]，因此关联只能理解为灾前区域天气的异常，不能当作灾点处的直接观测。然后，一次事件无法区分偶然与必然，本文把同一流域、同类机制的 2025 年 7 月 8 日事件作为第二个样本，只保留两次事件方向一致的异常。最后，用多指标综合评价排序，并按物理类别筛选，避免前 5 个因素都是同一类量。

2.2 问题二的分析

问题二要求求出触发参数的临界条件。首先，用补充的再分析降水检验常用的雨强历时阈值，若事件前降雨远低于阈值，说明触发的主因不是降雨。然后，冰崩与冰湖溃决需要充足的水分和足够的热量同时具备：前期降水使冰川与冰碛蓄水，事件前的升温使冰体软化、冰下排水增强。因此建立降水与正积温的双因子临界条件，以两次事件中的较小值定阈值，以误报日比例最小选取变量。最后，时间上用逐年警戒日数、留一检验与窗口长度检验可靠性，空间上用历史灾点建立易发性模型并做空间分块验证。

2.3 问题三的分析

问题三要求给出触发时间的概率区间。首先，在日尺度上沿用问题二的临界条件，未来几天的气温取 Open-Meteo 存档的 1 至 5 d 预报，并订正系统偏差。然后，日内的触发时刻取决于融水下渗所需的时间，用滞后 τ 的气温正距平作为权重，τ 由两次事件的似然估计。最后，对 τ、预报误差、阈值和气温直减率抽样得到区间，在多个起报时刻检验覆盖率与预警时间，空间距离范围取问题四的仿真统计。

2.4 问题四的分析

问题四要求搭建仿真环境。首先，以附件 4 冰川中坡度大于 30°、海拔高于 5000 m 的冰体为释放区，由冰厚积分得到体积。然后，用粒子 Voellmy 模型在 DEM 上计算运动，冰岩崩、泥石流、山洪三个阶段分别取摩擦参数，泥石流阶段计入沿程侵蚀。最后，输出到达时间、最大流深与速度、沿程剖面和堆积分布，并对参数做蒙特卡洛，给出到达

口岸的概率和停积范围。

2.5 总体思路

全文思路如图 1 所示。问题一的结果为问题二选取正积温因子提供依据，问题二的临界条件是问题三判定警戒日的依据，问题四的停积统计用于问题三的空间距离范围。

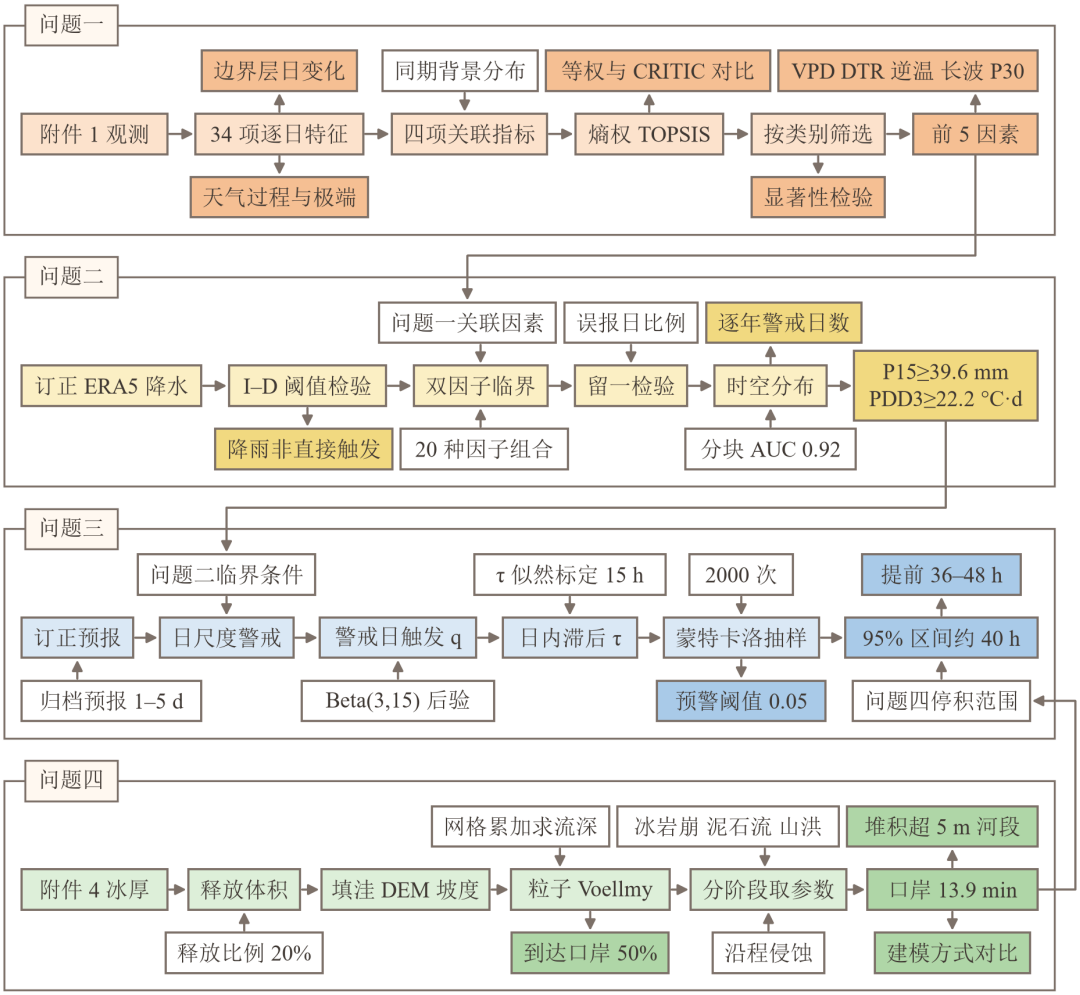


图 1　全文思路

3 模型假设

1. 吉隆观测塔反映区域天气背景，源区气温由站点气温按 0.65 °C/100 m 的直减率外推，直减率误差在预报中作为随机量处理。

2. 2025 年 7 月 8 日冰湖溃决洪水与 2026 年 8 月 26 日冰崩泥石流同属 Lhende 河上游的冰川灾害，触发机制相同。

3. 2021 至 2026 年汛期除这两次外，报道中没有同类重大灾害，其余满足临界条件的日子按误报计。

4. 冰崩体积为释放区冰量的固定比例，仿真中粒子之间只通过流深相互作用。

4 符号说明

表 1　主要符号

符号	说明	单位
P_{15}	吉隆站前 15 日累积降水	mm
PDD_3	源区海拔 5000 m 处前 3 日正积温	°C·d
c_1, c_2	双因子临界值	mm，°C·d
$T_s(t)$	源区逐时气温	°C
$T'(t)$	源区气温相对 24 h 滑动均值的正距平	°C
τ	气温峰值到触发的滞后	h
q	警戒日触发概率	无量纲
μ, ξ	Voellmy 摩擦系数与湍流系数	无量纲，m/s^2
h, $\boldsymbol{v}$	流深与速度	m，m/s
V_0	冰崩释放体积	m^3

其它符号在文中具体说明。

5 数据准备

5.1 数据来源

所用数据见表 2。附件 1 四类观测为逐时数据，采用北京时间，有效数据截至 2026 年 8 月 27 日 08:00；质控标志只保留 0 正确与 4 插补两类，其余数据与缺测值 9999.9 一并置为缺失，FLUX 中的非数值字符统一转为缺失。附件 3 只用覆盖源区的 N28E085 图幅。附件 2 为周边县市逐日气温[2]，与附件 1 的气温信息重复，仅用于核对；附件 8 为吉隆县第七次人口普查[8]，只有县级汇总，全县常住人口 17536 人，用于说明受威胁人口规模。附件 5 的类别编码与青藏高原土地覆盖数据集一致，1 为耕地，10 为冰雪[14]。

表 2　数据来源

数据	内容	用途
附件 1[1]	吉隆塔 MET、RADM、SOIL、FLUX 逐时观测，2021 至 2026 年	问题一、二、三
附件 3[3,9]	冰面高程变化速率，2000 至 2020 年，每 5 年一期	物质条件
附件 4[4]	源区冰川高程、坡度、冰厚、流速，30 m	释放区与体积
附件 5、6、7[5–7]	地表覆盖、84 条历史泥石流沟、灾区 DEM	易发性、仿真
ERA5、ERA5-Land[10,11]	源区、口岸、站点格点逐时降水与气温，补充数据	降雨检验、代表性
Open-Meteo 历史预报[12]	2024 至 2026 年提前 1 至 5 d 的逐时气温与降水，补充数据	问题三
Copernicus GLO-30[13]	30 m 卫星 DEM，覆盖全部灾点，补充数据	问题二空间评估

5.2 研究区与两次事件

研究区如图 2 所示。

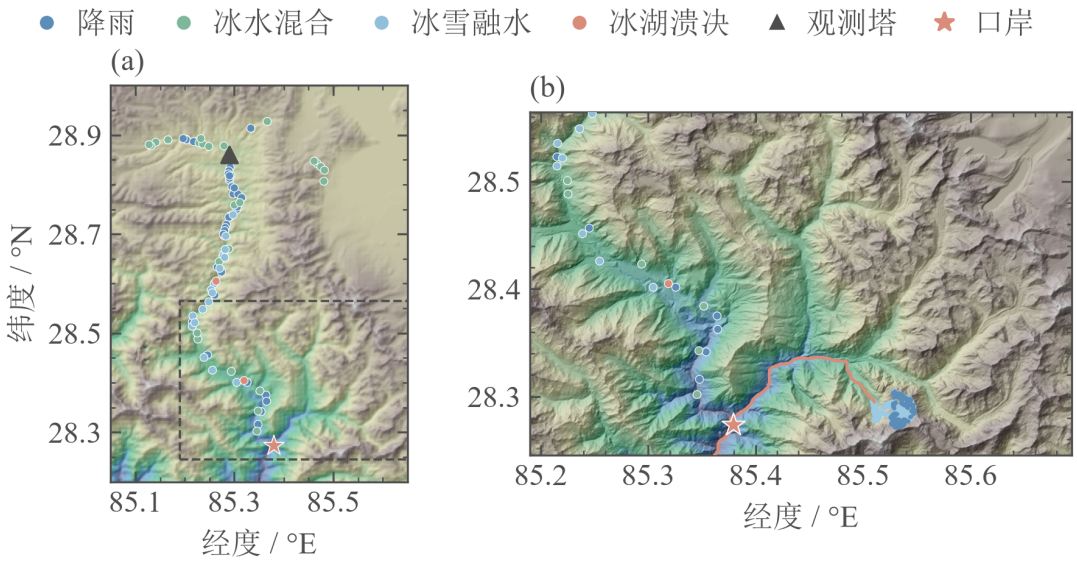


图 2　研究区概况：(a) 吉隆县；(b) 灾区

图 2(a) 为吉隆县范围，点为按类型着色的 84 条历史泥石流沟，三角为观测塔，星为口岸，虚线框为灾区 DEM 范围；图 2(b) 为灾区，浅蓝为附件 4 冰川，深蓝为释放区，红线为冰川末端至口岸的流路。冰川位于 Lhende 河源头，范围 28.27°N 至 28.31°N、85.51°E 至 85.55°E，海拔 3664 至 5994 m，面积 6.4 km^2，冰体积约 0.32 km^3。沿填洼后 DEM 的最陡下降路径追踪，冰川末端经 Lhende 河谷先向西北、再折向西南，21.4 km 后到达口岸，即 Lhende 河与吉隆藏布的汇口，高差约 1900 m。两次事件都发生在这条通道上。2025 年 7 月 8 日约 06 时，冰湖溃决洪水冲毁口岸大桥，时刻由下游 Syafrubesi 水位在当地时间 03:40 至 06:00 之间陡涨推算[22]；2026 年 8 月 26 日 10:30，冰崩堵塞 Lhende 河后溃决[21]。

灾区的三维地形如图 3 所示，红线为流路，蓝点为源区冰川，星为口岸。源区冰川位于海拔 5000 m 以上的陡峭山体上，Lhende 河谷深切，谷底到两侧山脊的高差普遍超过 2000 m，冰崩物质沿狭窄河谷集中下泄，口岸恰好位于河谷出口与吉隆藏布的交汇处。

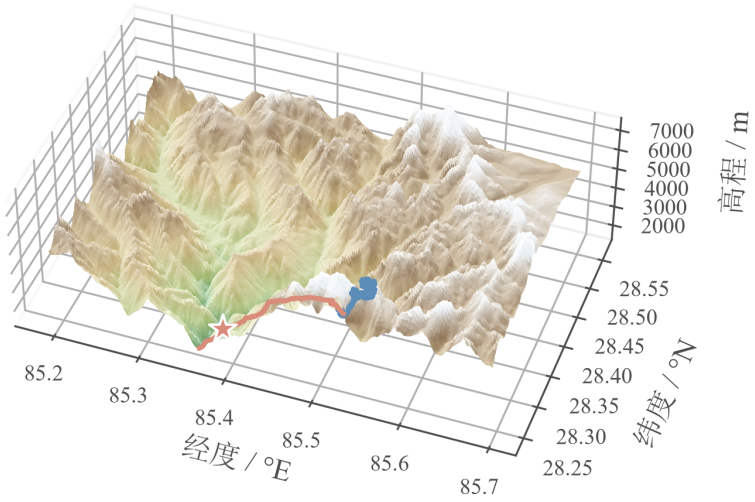


图 3　灾区三维地形

链式过程如图 4 所示。汛期降水使冰川、冰碛与松散堆积体蓄水，用 P_{15} 表示；晴朗天气下源区正积温上升，用 PDD_3 表示。融水下渗使冰体与冰床软化，高位冰体失稳

崩落，碎屑在陡坡上加速并铲刮沟床形成泥石流，进入主河后稀释为山洪，并在窄谷段堆积堵江。图中剖面不按实际比例。

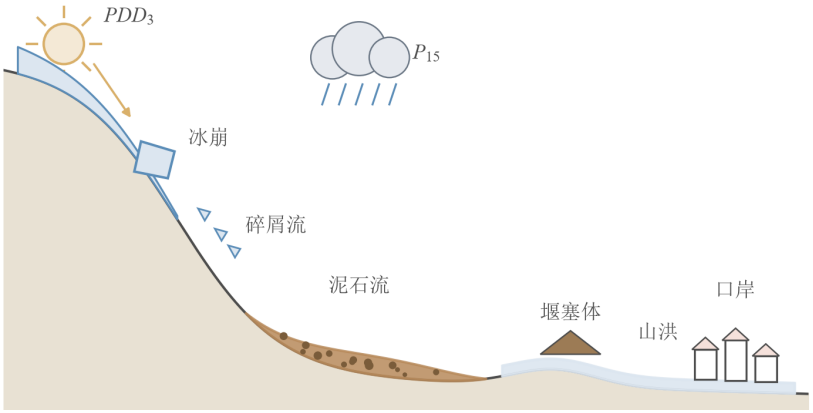


图 4　灾害链示意

6　模型的建立与求解

6.1　问题一：熵权 TOPSIS 排序模型

6.1.1　思路框架

问题一的求解路线如图 5 所示。先对四类逐时观测做质控与时间对齐，构造逐日特征；再为每次事件建立同期背景分布，计算四项关联指标；最后用熵权 TOPSIS 综合排序，并按物理类别筛选出前 5 个因素。

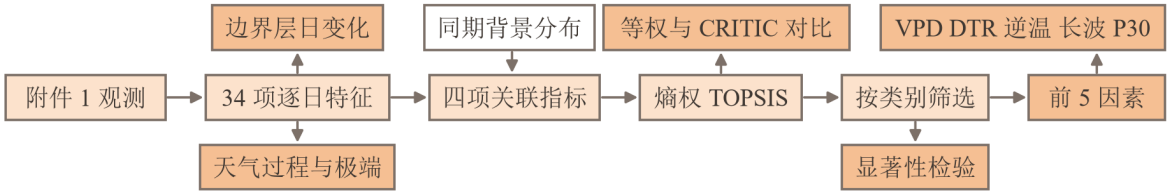


图 5　问题一思路

6.1.2　机理分析

高寒山谷的边界层在昼夜之间有两种不同结构，如图 6 所示。

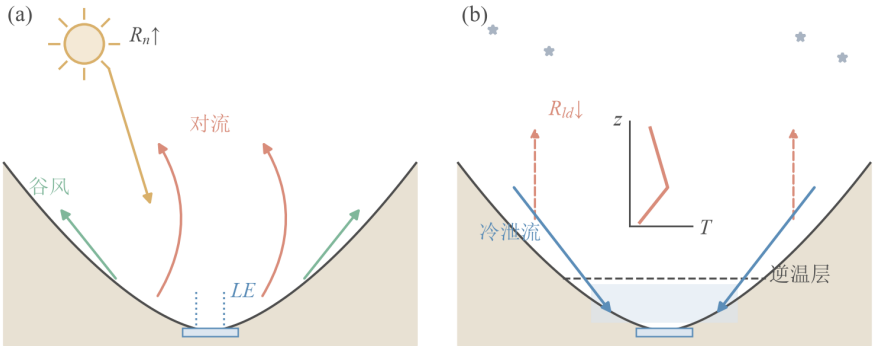


图 6　山谷边界层昼夜结构：(a) 白天；(b) 夜间

图 6(a) 为白天，净辐射 R_n 加热谷底与坡面，近地层不稳定，对流把热量与水汽向上输送，沿坡上升的谷风把暖空气带到冰川所在高度；前期降水较多时地表湿润，更多能量以潜热 LE 释放。图 6(b) 为晴朗的夜间，向下长波辐射 R_{ld} 小，地面冷却强，冷空气沿坡下泄并在谷底形成逆温层，近地层气温随高度先升后降。冰川与冻土在白天融化、夜间冻结的交替中反复胀缩，融水下渗后再冻结，冰体与冰床的连接逐渐减弱。因此，描

述晴朗、干燥、白天加热强、夜间冷却强的量可能与冰崩类灾害有关，本文据此把水汽、辐射、热力、边界层与湍流量都纳入候选特征，由数据检验其关联。

6.1.3 特征构建

由逐时观测构造 7 类 34 项逐日特征。天气过程类包括日降水、7 日与 30 日降水、前期有效降水 $API_{15}=\sum_{i=0}^{14}0.9^{i}P_{t-i}$、气压与 24 h 变压；热力类包括日平均与日最高气温、气温日较差 DTR、3 日升温、源区正积温 $PDD=\overline{\max(T_s,0)}$ 及其 7 日累积；水汽类包括相对湿度与饱和水汽压差 VPD；辐射类包括向下短波、向下长波、净辐射与反照率；土壤类包括 10、40、160 cm 含水量，土温与土壤热通量；湍流类包括感热 H、潜热 LE、波文比、摩擦速度与湍流动能 TKE；边界层类包括 1 至 20 m 温度梯度、夜间逆温频率、风切变、风廓线幂指数与整体理查森数 R_b。

6.1.4 边界层结构与天气过程

两次事件前一周的边界层日变化如图 7 所示，实线为事件前一周，虚线为同期气候平均，图 7(a) 至 (f) 依次为 1 至 20 m 温度梯度、净辐射、感热、潜热、湍流动能与 2 m 气温。

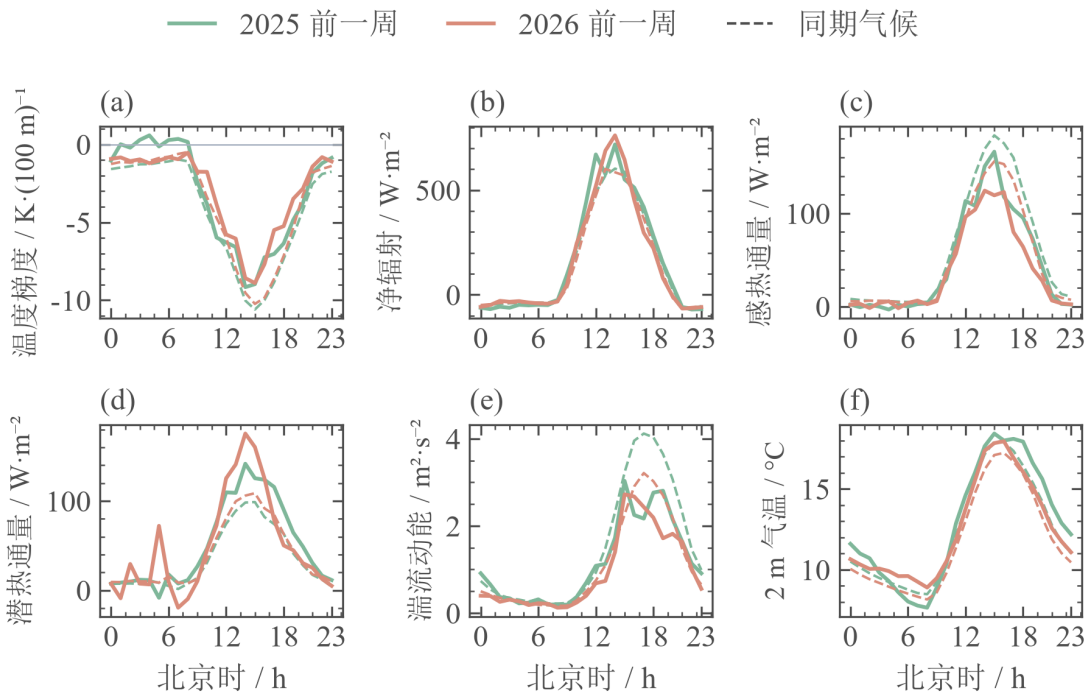


图 7 事件前边界层日变化：(a) 温度梯度；(b) 净辐射；(c) 感热；(d) 潜热；(e) 湍流动能；(f) 气温

白天净辐射峰值分别为 718 和 762 W/m^2，比同期偏高 19% 和 27%，但能量更多分配给潜热，LE 峰值由同期的 99 和 109 W/m^2 增至 142 和 175 W/m^2，感热与湍流动能峰值反而偏低，说明前期降水使地表湿润，午后对流混合偏弱。夜间则更稳定，图 7(a) 中温度梯度大于 0 表示逆温，2025 年事件前夜间平均温度梯度为 -0.26 K/100 m，同期为 -1.44 K/100 m，夜间逆温频率达 50%，同期仅 8%，说明晴空条件下地面长波冷却强烈。2026 年事件前夜间温度梯度为 -0.98 K/100 m，与同期的 -1.15 K/100 m 接近，夜间逆温只在 2025 年明显偏多。白天辐射强、夜间冷却强的天气，有利于高海拔地区的冻融循环。

天气过程如图 8 所示，柱为日降水，虚线为事件日，点线为同期均值。

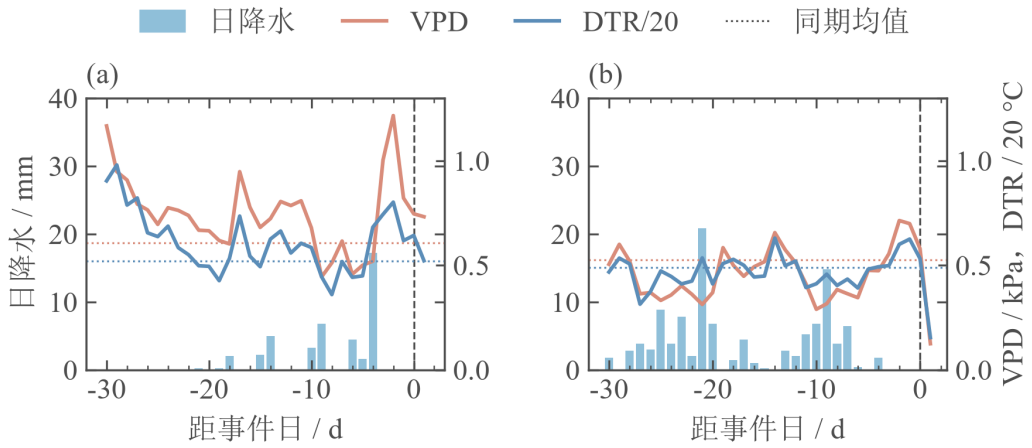


图 8　事件前 30 日天气过程：(a) 2025 年；(b) 2026 年

2026 年事件发生在一段多雨期之后，8 月 1 日至 26 日累计降水 101 mm，是 2021 至 2025 年同期 28 至 51 mm 的 2 至 3.6 倍，事件前 6 天中站点有 5 天无雨；2025 年事件前 30 日降水的同期百分位为 72.8，并不异常，事件前 4 天即 7 月 4 日站点出现 17.2 mm 的强降水，之后站点无雨，但源区的再分析降水显示小雨持续到事件前约 5 h，说明站点与源区的降水并不同步。两次事件前 3 日 VPD 与 DTR 都升到同期均值以上。按 5 至 10 月湿日降水第 95 百分位 14.8 mm、日最高气温第 95 百分位 21.1 °C，以及连续 3 日以上气温距平超过 1 个标准差且无雨，统计事件前 10 日的强降水日、高温日与高温少雨日，并与同期任意 10 日的平均日数比较，结果见表 3。两次事件前 10 日各有 1 个强降水日，约为同期平均的 4 至 6 倍；2025 年还有 2 个高温日，约为同期的 2 倍；两次事件前都没有连续 3 日的高温少雨。

表 3　事件前 10 日极端日数

类型	2025 年事件前	同期平均	2026 年事件前	同期平均
强降水日	1	0.23	1	0.16
高温日	2	0.98	0	0.43
高温少雨日	0	0.16	0	0.23

6.1.5　模型的建立

对特征 f 和事件 e，取事件前 3 个整日的统计量 x_{fe}，累积量取事件前一日的值；背景分布 $\mathcal{C}_{fe}$ 由 2021 至 2026 年中年积日相差不超过 30 天、且距两次事件 15 天以外的同长度窗口组成。四项关联指标为

$$
\begin{cases}
p_{fe} = \mathrm{Pr}(\mathcal{C}_{fe} < x_{fe}), \quad s_{fe} = 2p_{fe} - 1, & \text{同期百分位} \\
I_1 = \tfrac{1}{2}\sum_e |s_{fe}|, \quad I_2 = \dfrac{|\sum_e s_{fe}|}{\sum_e |s_{fe}|}, & \text{异常程度与一致性} \\
I_3 = \tfrac{1}{2}\sum_e \mathrm{sgn}(s_{fe})\dfrac{\bar{x}_{fe}^{(-3,-1)} - \bar{x}_{fe}^{(-10,-4)}}{\sigma_{fe}}, & \text{前后期变化量} \\
I_4 = \tfrac{1}{2}\sum_e \left|\dfrac{x_{fe} - \bar{\mathcal{C}}_{fe}}{\mathrm{sd}\,\mathcal{C}_{fe}}\right|, & \text{标准化距平}
\end{cases}
\tag{1}
$$

其中前后期变化量为事件前 3 日均值与前 4 至 10 日均值之差除以同期逐日标准差。四项指标经极差标准化为 y_{fk} 后，按信息熵定权，计算各特征到正、负理想解的距离与贴

近度，再在每个物理类别中取贴近度最高的一项。综上，熵权 TOPSIS 排序模型为

$$
\begin{cases}
y_{fk} = \dfrac{I_k(f) - \min_f I_k}{\max_f I_k - \min_f I_k}, \quad p_{fk} = \dfrac{y_{fk}}{\sum_f y_{fk}}, \\
e_k = -\dfrac{1}{\ln n}\sum_f p_{fk}\ln p_{fk}, \quad w_k = \dfrac{1-e_k}{\sum_k (1-e_k)}, \\
D_f^{\pm} = \left[\sum_k w_k^2 \left(y_{fk} - y_k^{\pm}\right)^2\right]^{1/2}, \quad C_f = \dfrac{D_f^-}{D_f^+ + D_f^-}, \\
f_g^* = \arg\max_{f\in g} C_f, \quad \text{前 5 个因素为} C_{f_g^*} \text{最大的 5 个类别} g,
\end{cases}
\tag{2}
$$

其中 y_k^+、y_k^- 为第 k 项指标的最大值与最小值，$n = 34$。

6.1.6 求解结果

熵权依次为：异常程度 0.247、一致性 0.142、前后期变化量 0.246、标准化距平 0.365。前 12 项特征及排在第 13 位的 30 日降水见表 4，带星号者为按类别筛选后的前 5 个因素。前 13 项的一致性都为 1，说明它们在两次事件中的偏离方向相同，一致性指标的作用是剔除两次事件方向相反的特征，因而熵权较小。按类别筛选后，30 日降水虽然贴近度排在第 13 位，但它是天气过程类中最高的一项，因此入选，所以前 5 个因素是不同类别中关联最高的因素。土壤类最高的 10 cm 土温与湍流类最高的潜热排在前 5 类之后，未入选，说明站点土壤与湍流量对远处源区的指示作用较弱。排序与两次事件的百分位见图 9，图 9(b) 中灰带为 10 至 90 百分位。前 5 个因素的具体数值见表 5。

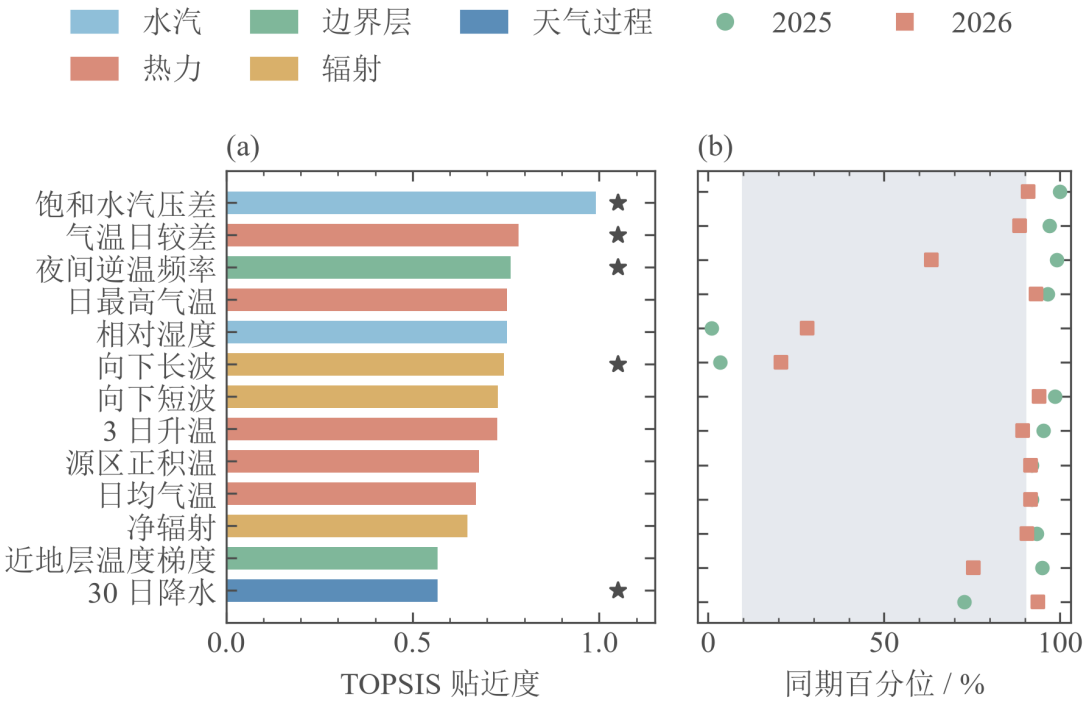


图 9 关联排序结果：(a) 贴近度；(b) 同期百分位

表 4　关联排序

排名	特征	类别	方向	异常程度	一致性	变化量	$\lvert z\rvert$	贴近度
1	饱和水汽压差 *	水汽	偏高	0.909	1.00	2.64	2.21	0.991
2	气温日较差 *	热力	偏高	0.855	1.00	1.67	1.66	0.784
3	夜间逆温频率 *	边界层	偏高	0.625	1.00	1.33	1.96	0.763
4	日最高气温	热力	偏高	0.895	1.00	1.53	1.55	0.754
5	相对湿度	水汽	偏低	0.710	1.00	2.23	1.53	0.753
6	向下长波 *	辐射	偏低	0.760	1.00	2.07	1.48	0.745
7	向下短波	辐射	偏高	0.925	1.00	1.54	1.44	0.729
8	3 日升温	热力	偏高	0.846	1.00	1.61	1.45	0.727
9	源区正积温	热力	偏高	0.835	1.00	1.44	1.29	0.678
10	日均气温	热力	偏高	0.835	1.00	1.41	1.27	0.671
11	净辐射	辐射	偏高	0.839	1.00	0.87	1.30	0.647
12	近地层温度梯度	边界层	偏高	0.702	1.00	0.12	1.27	0.567
13	30 日降水 *	天气过程	偏高	0.666	1.00	0.12	1.30	0.567

表 5　前 5 个因素

因素	单位	方向	2025-07-08			2026-08-26		
			事件前	同期	百分位	事件前	同期	百分位
饱和水汽压差	kPa	偏高	1.01	0.61	100.0	0.66	0.53	90.9
气温日较差	℃	偏高	14.42	10.42	97.0	11.49	9.79	88.4
夜间逆温频率	无量纲	偏高	0.50	0.08	99.1	0.08	0.10	63.4
向下长波	W/m^2	偏低	282.55	323.53	3.3	313.33	325.55	20.6
30 日降水	mm	偏高	42.80	29.82	72.8	112.40	56.07	93.8

5 个因素共同描述了灾前的天气背景。30 日降水偏多表示前期蓄水，2026 年为 112 mm，是同期均值的 2 倍；VPD 偏高、向下长波偏低表示事件前空气干燥、云量少；日较差大、夜间逆温多表示白天加热与夜间冷却都强。这与媒体报道中此次灾害并非降雨触发、疑由冰崩引起的初步判断一致。

6.1.7 方法对比

用同一组指标比较五种综合评价方法：熵权 TOPSIS、等权 TOPSIS、CRITIC 法定权的 TOPSIS、分辨系数为 0.5 的灰色关联分析，以及只用异常程度的单指标排序。以按类别筛选后的前 5 与熵权法结果的重合数、34 项特征排序与熵权法的 Kendall τ 衡量各方法结论的一致程度，耗时取 200 次运行的平均，结果见表 6。

表 6　综合评价方法对比

方法	前 5 个因素	与本文重合	Kendall τ	耗时/ms
熵权 TOPSIS	饱和水汽压差、气温日较差、夜间逆温频率、向下长波、30 日降水	5/5	1.000	0.206
等权 TOPSIS	饱和水汽压差、气温日较差、向下长波、夜间逆温频率、日降水	4/5	0.886	0.094
CRITIC 赋权 TOPSIS	饱和水汽压差、气温日较差、向下长波、夜间逆温频率、日降水	4/5	0.822	0.663
灰色关联分析	饱和水汽压差、向下短波、日最高气温、夜间逆温频率、日降水	2/5	0.829	0.113
只用异常程度	向下短波、饱和水汽压差、日最高气温、整体理查森数、160 cm 土壤含水量	1/5	0.707	0.000

三种 TOPSIS 的前 5 有 4 项相同，Kendall τ 都在 0.82 以上，区别只在第 5 个因素是 30 日降水还是日降水；灰色关联与单指标排序把向下短波、日最高气温等同类热力量排在前面，与本文分别只重合 2 项和 1 项。各方法耗时都在 1 ms 以内，相差不大。本文采用熵权 TOPSIS，因为它的权重完全由数据决定，不依赖主观判断，且结论与另外两种 TOPSIS 基本一致。由于这一比较以熵权法为参照，只能说明结论对评价方法不敏感，不能证明哪种方法更准确，准确性由下面的显著性检验说明。

6.1.8 模型检验

单事件分别排序。只用一次事件的百分位与标准化距平做熵权 TOPSIS，两次排序的 Spearman 相关为 0.54，前 5 中只有饱和水汽压差相同：2025 年事件偏重夜间逆温与长波，2026 年事件偏重 30 日降水与最高气温。单个事件的结论受当时天气的偶然性影响较大，合并两次事件并要求方向一致，可以减小这种影响。

显著性检验。若某特征与事件无关，两次事件的同期百分位 p_1、p_2 应独立服从 $[0,1]$ 上的均匀分布，此时 $s_e = 2p_e - 1$ 独立服从 $[-1,1]$ 上的均匀分布，$s_1 + s_2$ 服从三角分布。记 $r = |s_1 + s_2|/2$，则 $\Pr(R \geq r) = (1-r)^2$。前 5 个因素的 p 值依次为：饱和水汽压差 0.008、气温日较差 0.021、夜间逆温频率 0.140、向下长波 0.057、30 日降水 0.112。前两个因素在 0.05 水平上显著，向下长波接近显著，夜间逆温与 30 日降水只在单次事件中明显，结论应谨慎对待。

物理一致性。VPD 偏高与相对湿度偏低、向下长波偏低与净辐射偏高、日较差偏大与夜间逆温偏多两两对应，分别从水汽、辐射、热力与边界层四个方面描述了晴朗干燥的天气。其中夜间逆温在 2026 年事件前略低于同期，方向与 2025 年不一致，是前 5 个因素中最弱的一项。

6.2 问题二：双因子临界模型与易发性评价模型

6.2.1 思路框架

问题二分时间与空间两条线求解，如图 10 所示。第一行用订正后的再分析降水检验降雨触发；第二行在排除降雨直接触发之后，以两次事件定阈值，求双因子临界条件，并用留一检验与误报日比例筛选变量；第三行用卫星 DEM 与历史灾点建立易发性模型，检验空间上的可靠性。

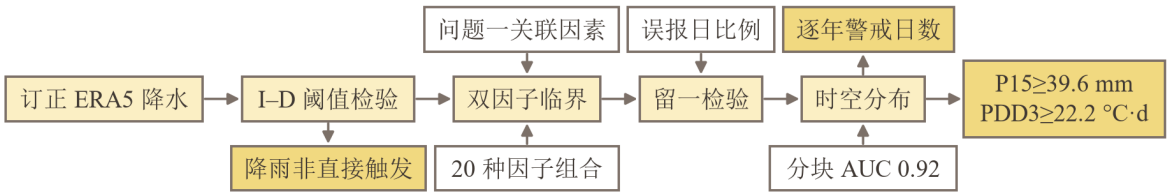


图 10　问题二思路

6.2.2 机理分析

冰崩与冰湖溃决需要水分与热量同时满足条件，只满足其一不足以触发。如图 11 所示，以前 15 日降水为横轴、源区 3 日正积温为纵轴，把逐日状态画成点。汛期前段降水使冰川、冰碛与松散堆积体蓄水，状态点右移；此时若天气阴凉，热量不足，状态点停在临界线下方。进入晴热天气后正积温上升，状态点上移，越过两条临界线进入右上角的触发区域，冰体软化、冰下排水增强，失稳随之发生。因此临界条件应当是两个条件同时成立，而不是单个指标或两者相加。

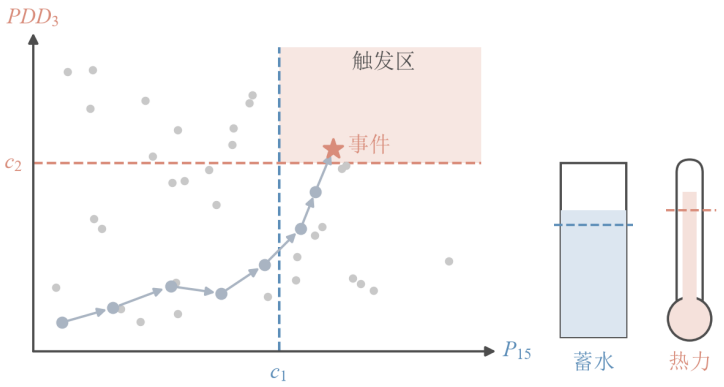


图 11　双因子触发机理

6.2.3　补充数据与站点代表性

ERA5 在站点格点的降水是站点雨量的 3.2 倍，订正系数取 0.314，二者日相关仅 0.46；ERA5 源区与口岸格点降水与站点的日相关为 0.38 和 0.28。ERA5-Land 源区格点海拔 4982 m，其气温与站点气温的日相关为 0.94，扣除季节变化后的距平相关仍有 0.77，按直减率外推的源区气温比 ERA5-Land 高 1.65 °C。可见气温在 65 km 范围内变化基本一致，降水的空间代表性较弱，站点降水只能表示区域的湿润程度，而用站点气温推算源区热量是可靠的。

6.2.4　降雨触发的检验

以 12 h 无雨为间隔，把 2021 至 2026 年订正后的源区逐时降水划分为降雨事件，与 Caine[15]、Guzzetti[16] 和 Dahal[17] 的雨强历时阈值比较，三条阈值依次为 $I = 14.82D^{-0.39}$、$I = 2.20D^{-0.44}$、$I = 73.90D^{-0.79}$，结果见图 12(a)，星号为两次事件前的最后一场雨。

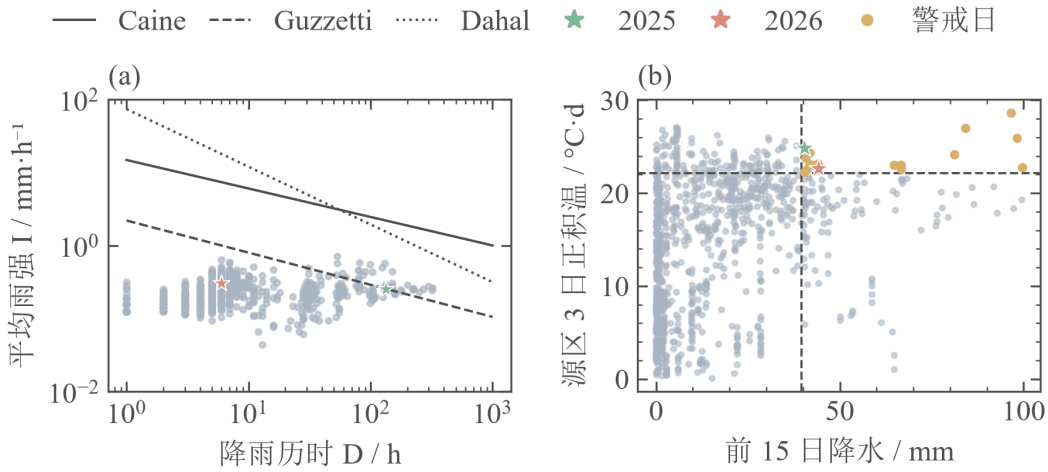


图 12　临界条件：(a) 雨强与历时；(b) 双因子平面

2025 年事件前最后一场雨历时 133 h，平均雨强 0.25 mm/h，前 24 h 雨量 6.6 mm；2026 年事件前 15.5 h 才结束一场历时 6 h、雨强 0.30 mm/h 的小雨，前 24 h 雨量 2.0 mm。2026 年事件远低于三条阈值；2025 年事件低于 Caine 与 Dahal 阈值，但与 Guzzetti 阈值几乎相等，在历时 133 h 处阈值为 0.256 mm/h，实际为 0.254 mm/h，而且这一结果依赖 0.314 的订正系数。因此降雨可以排除为 2026 年事件的直接触发因素，对 2025 年事件只能说长历时弱降雨处于临界附近，主要作用是增加前期蓄水。

6.2.5 模型的建立

设第 t 日的降水因子 $X_1(t)$ 与正积温因子 $X_2(t)$ 只用 $t-1$ 日及以前的数据，临界值取两次事件中较小值的 0.98 倍，两个因子同时超过临界值的日子判为警戒日。综上，双因子临界模型为

$$\begin{cases} A(t) = \mathbb{1}\big[X_1(t) \ge c_1\big] \cdot \mathbb{1}\big[X_2(t) \ge c_2\big], \\ c_1 = 0.98 \min\limits_{e} X_1(t_e), \quad c_2 = 0.98 \min\limits_{e} X_2(t_e), \\ (X_1, X_2)^{*} = \arg\max \big\{ \text{命中数}, \ \text{留一命中数}, \ -\text{误报日比例} \big\}, \end{cases} \tag{3}$$

其中 $X_1 \in \{P_{15}, P_{30}, P_{45}, API_{15}\}$，$X_2 \in \{PDD_1, PDD_3, PDD_5, \mathrm{VPD}_3, \mathrm{DTR}_3\}$，共 20 种组合。留一检验只用一次事件定阈值，看能否报中另一次事件。两次事件的 P_{15} 为 40.4 与 44.2 mm，PDD_3 为 24.9 与 22.6 °C·d，相差约 10%，因此留一检验的系数取 0.9；若与主模型一样取 0.98，留一检验为 0/2。

6.2.6 求解结果

前 8 种组合见表 7。最优组合为 $P_{15} \ge 39.6$ mm 且 $PDD_3 \ge 22.2$ °C·d，两次事件全部报中，留一检验 2/2，5 至 10 月约 1030 天中误报 14 天，误报日比例 1.36%。按警戒日计，至事件日为止的 16 个警戒日中有 2 个对应事件，其余 14 个为误报。只用降水因子误报 113 天，只用正积温因子误报 146 天，双因子使误报减少到原来的八分之一至十分之一，说明水分与热量缺一不可。需要指出，临界值由两次事件确定，又用这两次事件检验，命中数属于样本内结果，留一检验才是对推广能力的检验。图 12(b) 为 5 至 10 月逐日的两个因子与临界线，橙点为警戒日。前期有效降水与 3 日 VPD 的组合误报更少，只有 6 天，但留一检验仅 1/2，推广能力不足，未采用。

表 7　双因子组合比较

降水因子	正积温因子	c_1	c_2	命中	留一	误报日	误报比例/%
15 日降水	源区 3 日正积温	39.59	22.18	2/2	2/2	14	1.36
15 日降水	源区 1 日正积温	39.59	7.60	2/2	2/2	14	1.36
15 日降水	源区 5 日正积温	39.59	35.21	2/2	2/2	22	2.13
前期有效降水	3 日 VPD	18.67	0.64	2/2	1/2	6	0.58
15 日降水	3 日 VPD	39.59	0.64	2/2	1/2	7	0.68
前期有效降水	源区 3 日正积温	18.67	22.18	2/2	1/2	13	1.26
前期有效降水	3 日 DTR	18.67	11.26	2/2	1/2	14	1.36
15 日降水	3 日 DTR	39.59	11.26	2/2	1/2	16	1.55

6.2.7 方法对比

在同一套逐日数据上比较五种判别方法：双因子阈值、两种单因子阈值、乘积指数 $(X_1/c_1)(X_2/c_2)$ 与逻辑回归。乘积指数与逻辑回归的判别值都取两次事件中的较小值，使两次事件都能报中，再比较误报，耗时取 50 次运行的平均，结果见表 8。

表 8　时间判别方法对比

方法	命中	误报日	误报比例/%	耗时/ms
双因子阈值	2/2	14	1.36	0.96
单因子 P15	2/2	113	10.96	0.53
单因子 PDD3	2/2	146	14.16	0.60
乘积指数	2/2	40	3.88	2.15
逻辑回归	2/2	41	3.98	65.89

五种方法都能报中两次事件，差别在误报：双因子阈值误报 14 天，乘积指数与逻

辑回归约 40 天，单因子 113 至 146 天。乘积指数与逻辑回归允许两个因子相互补偿，一个因子很高时另一个不足也会报警，与图 11 的机理不符，误报因此约为双因子的 3 倍。各方法耗时都在 0.1 s 以内，双因子阈值误报最少，予以采用。

6.2.8 时间可靠性

2021 至 2026 年各年警戒日数依次为 2、0、0、5、5、5 天，分布在 7 月上旬至 8 月下旬，依次为 2021 年 7 月 26 日与 8 月 3 日，2024 年 7 月 30 日至 8 月 3 日，2025 年 7 月 7 日至 8 日与 8 月 3 日至 5 日，2026 年 8 月 13 日至 15 日与 8 月 26 日至 27 日，其中 2025 年 7 月 8 日与 2026 年 8 月 26 日对应事件。其余连续警戒没有对应的灾害报道，按误报计，是本模型的主要误差来源。窗口长度方面，降水因子取 15 日降水时，1、3、5 日正积温三种正积温因子都能 2/2 命中且留一 2/2；改取 30 或 45 日降水后命中不变，但留一降为 1/2 或 0/2，误报比例升至 3% 至 10%，说明 2 周左右的前期降水最为关键。

6.2.9 空间评估与可靠性

易发性因子的空间分布如图 13 所示。图 13(a) 为吉隆县 2023 年地表覆盖，灰点为历史灾点，三角为观测塔，星为口岸。灾点沿吉隆藏布主谷呈带状分布，主要位于林地、灌丛与高寒草甸之间的过渡带，谷坡植被稀疏、松散物质丰富。图 13(b) 为灾区坡度，蓝线为汇水面积大于 20 km^2 的河网，红线为流路。源区与 Lhende 河谷两侧坡度普遍超过 40°，为冰崩与泥石流提供了充足的势能。

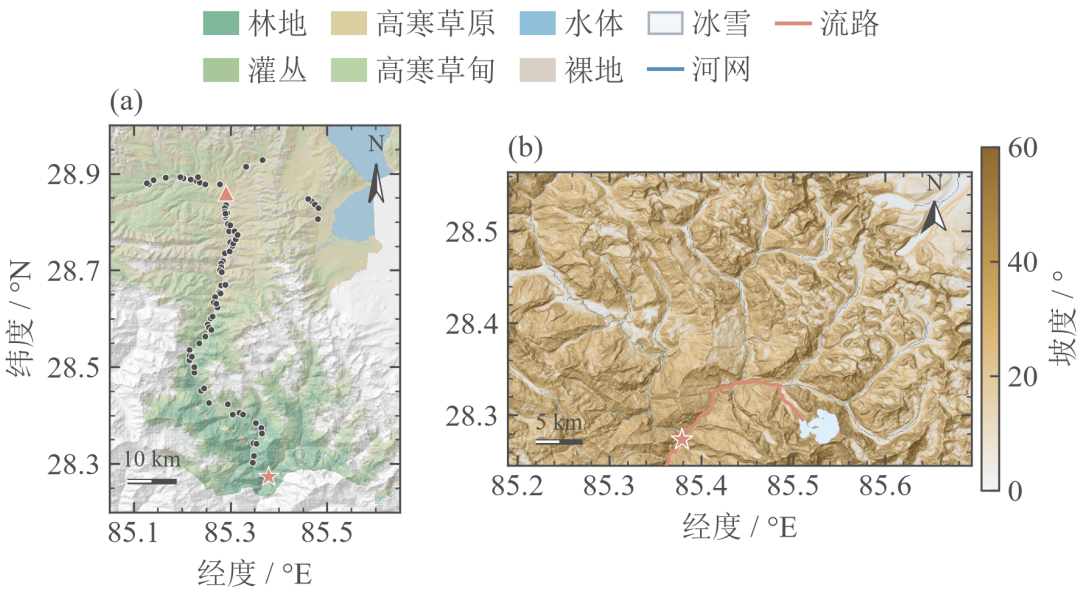


图 13 地表覆盖与地形：(a) 吉隆县地表覆盖；(b) 灾区坡度

以 84 条历史泥石流沟的沟口为正样本，在研究区内距沟口 1 km 以外、海拔低于 5500 m 处随机取 840 个点为负样本，由重采样至约 90 m 的 Copernicus DEM 与地表覆盖数据提取高程、坡度、1 km 起伏度、坡向、汇水面积、距河道距离、地形湿度指数、距冰雪距离、植被比例与裸地比例 10 个因子，记为 $\boldsymbol{z}$。综上，易发性评价模型为

$$
\begin{cases}
\mathcal{S}=\{(\boldsymbol{z}_i,y_i)\},\quad y_i=1\ \text{为灾点},\ y_i=0\ \text{为随机点},\quad |\{y_i=0\}|=10\,|\{y_i=1\}|,\\
F(\boldsymbol{z})=\dfrac{1}{B}\sum_{b=1}^{B}\mathrm{Pr}_b(y=1\mid\boldsymbol{z}),\quad B=500,\\
\mathrm{AUC}=\Pr\big(F(\boldsymbol{z}_+)>F(\boldsymbol{z}_-)\big),\quad \text{按纬度分 5 块逐块留出验证},
\end{cases}
\tag{4}
$$

其中 Pr_b 为第 b 棵决策树的叶节点灾点比例，F 为易发性。结果如图 14 所示，图 14(a) 中点为历史灾点，星为口岸。

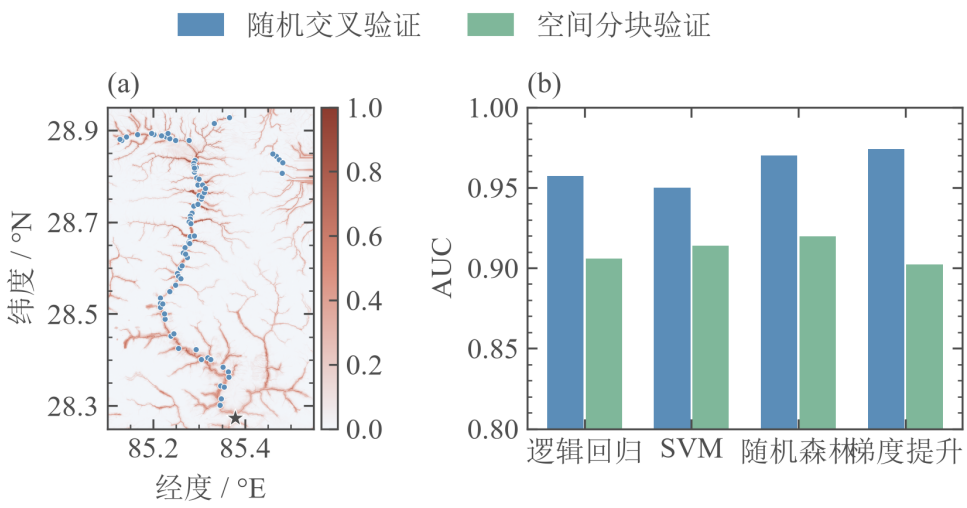


图 14　易发性评价：(a) 易发性分布；(b) 模型 AUC

随机森林的 5 折交叉验证 AUC 为 0.970，按纬度分为 5 块逐块留出验证时为 0.920。在训练样本上，易发性大于 0.5 的区域只占研究区的 2.0%，却包含 83.3% 的历史灾点；口岸 1 km 范围内的最大易发性为 0.56。因子重要度前五依次为距河道距离 0.219、高程 0.186、裸地比例 0.108、汇水面积 0.098 与距冰雪距离 0.094。

四种空间模型的对比见表 9，耗时为 5 折交叉验证中单折的平均训练时间。梯度提升树的交叉验证 AUC 最高，为 0.974，但分块验证降到 0.902，降幅最大，说明它过多拟合了训练区域的局部特征；随机森林的分块验证 AUC 最高，为 0.920，易发性前 10% 的区域包含 77.4% 的灾点，与梯度提升树相同。随机森林训练耗时约 3.7 s，对本题的样本量可以接受，因此采用随机森林。

表 9　空间模型对比

模型	交叉验证 AUC	分块验证 AUC	前 10% 区域命中/%	耗时/ms
逻辑回归	0.957	0.906	66.7	54
支持向量机	0.950	0.914	69.0	95
随机森林	0.970	0.920	77.4	3728
梯度提升树	0.974	0.902	77.4	852

分块验证的 AUC 比交叉验证低约 0.05，说明模型可以推广到研究区内相邻的纬度带，但历史灾点集中在吉隆藏布主谷，对远离主谷的高海拔支沟代表性不足，Lhende 河源区就属于这种情况。物质条件方面，附件 3 显示源区冰川平均减薄速率四期依次为 0.41、0.45、0.51、0.62 m/a，20 年加快约 50%。冰体变薄、冰床暴露，可能使高位冰体在同样的热量条件下更容易失稳，临界值在不同年代可能有所变化。源区冰川的冰厚、流速与 2015 至 2020 年高程变化如图 15 所示，图 15(a) 中红线围成的区域为坡度大于 30°、海拔高于 5000 m 的释放区。冰体最厚处约 150 m，位于冰川东部；流速最大的区域与释放区大体重合，年流速可达 40 m 以上；冰川大部分区域高程下降，说明冰体整体在减薄。

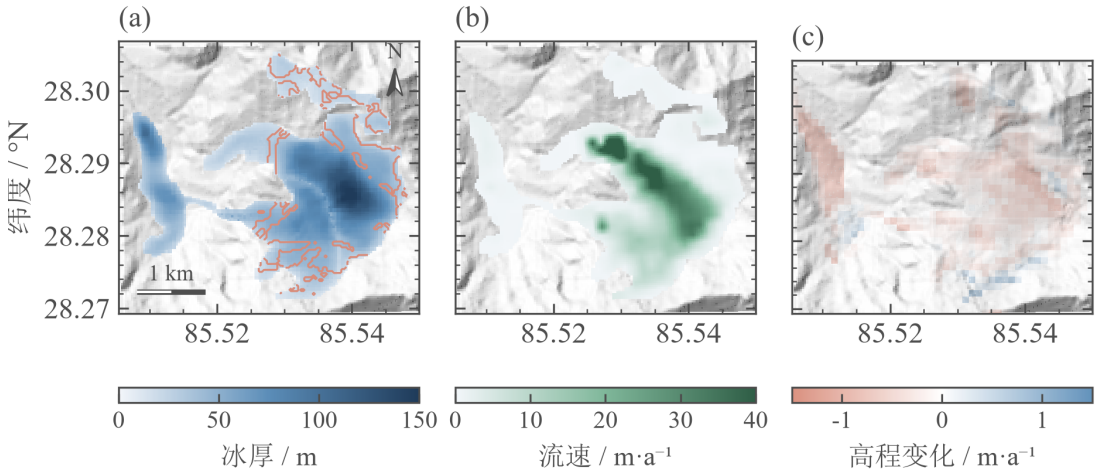


图 15　源区冰川：(a) 冰厚；(b) 流速；(c) 高程变化

6.3 问题三：滞后概率预报模型

6.3.1 思路框架

问题三在问题二临界条件的基础上加入时间维度，路线如图 16 所示。先把存档的多个预见期的预报订正为源区气温，用临界条件判定未来几天的警戒日；再给警戒日赋予触发概率，并在日内按滞后的气温正距平分配触发时刻；最后用蒙特卡洛得到 95% 置信区间，用滚动起报确定预警阈值与预警时间。

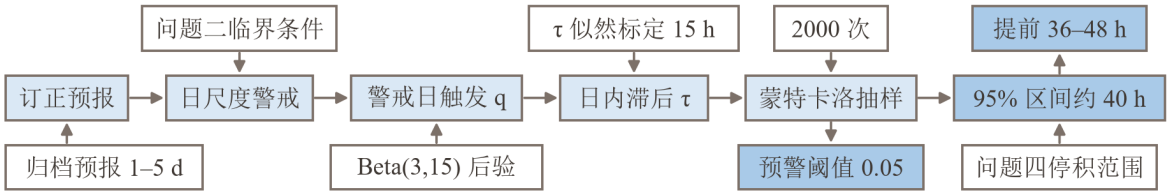


图 16　问题三思路

6.3.2 预报订正

未来各日的源区气温取 Open-Meteo 存档的提前 L 天预报 $\hat{T}_L$。不同时期预报模式的格点高度不同，2025 年 7 月提前 1 天的预报比当日分析值偏高约 7 °C，因此先减去起报前 30 天的平均偏差，再用 2024 年 1 月至 2025 年 5 月的分析值与站点外推源区气温建立线性关系 $T_s = 4.10 + 0.871\hat{T}$。$L = 1$ 至 5 时，日平均源区气温误差的标准差依次为 1.66、1.77、1.75、1.79、1.84 °C；预报降水乘以 0.204 订正到站点量级。

6.3.3 模型的建立

预报时序如图 17 所示，底纹为警戒日。在日尺度上，起报时刻 t_0 之前用观测、之后用订正预报计算 P_{15} 与 PDD_3，按问题二的临界条件判定警戒日。至最后一次事件当日为止共有 16 个警戒日，其中 2 个发生事件，警戒日触发概率取 Beta 后验 $q \sim \mathrm{Beta}(3, 15)$，均值 0.17。在日内，午后气温峰值产生的融水需要下渗一段时间才使冰体失稳，因此触发时刻按滞后 τ 的气温正距平分配。综上，滞后概率预报模型为

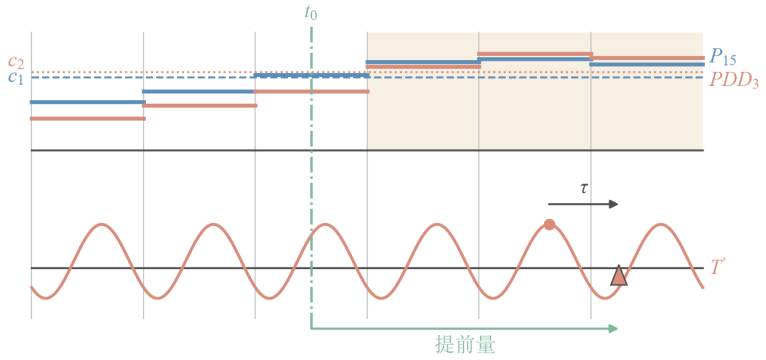


图 17　预报时序示意

$$
\begin{cases}
A(d) = \mathbb{1}\big[P_{15}(d) \ge c_1\big] \cdot \mathbb{1}\big[PDD_3(d) \ge c_2\big], & \text{警戒日} \\
D^{*} = \min\{d \ge d_0 :\ A(d) = 1,\ U_d < q\}, \quad U_d \sim U(0,1), & \text{触发日} \\
\Pr(T \in [t, t+1) \mid D^{*}) \propto T'(t-\tau) + \epsilon, \quad t \in D^{*}, & \text{触发时刻} \\
T'(t) = \max\left\{T_s(t) - \overline{T_s}^{\,24\,\mathrm{h}}(t), 0\right\}, \quad \epsilon = 0.05,
\end{cases}
\tag{5}
$$

其中随机量为 $\tau \sim U(\tau_1, \tau_2)$、预报误差 $\varepsilon_L \sim N(0, \sigma_L^2)$、直减率误差 $N(0, 0.05^2)$ °C/100 m，临界值 c_1、c_2 在 ±10% 内均匀扰动。每个起报时刻抽样 2000 次，以触发时刻样本的 2.5% 与 97.5% 分位数为 95% 置信区间。

6.3.4　求解步骤

Step1 读入起报时刻 t_0 前的观测与 t_0 时存档的 1 至 5 d 预报；Step2 按平均偏差与线性关系把预报订正为源区气温；Step3 抽取 τ、预报误差、直减率误差与临界值扰动；Step4 逐日计算 P_{15} 与 PDD_3，判定警戒日；Step5 对每个警戒日以概率 q 抽签，取第一个触发日；Step6 在该日内按 $T'(t-\tau)$ 加权抽取触发时刻；Step7 重复 Step3 至 Step6 共 2000 次，取 2.5% 与 97.5% 分位数。全部样本以矩阵形式一次计算，单次起报约需 0.26 s。

6.3.5　滞后的估计

用观测气温计算事件当日各小时的权重，事件时刻的似然随 τ 的变化如图 18(a) 所示，点线为峰值的 15%。

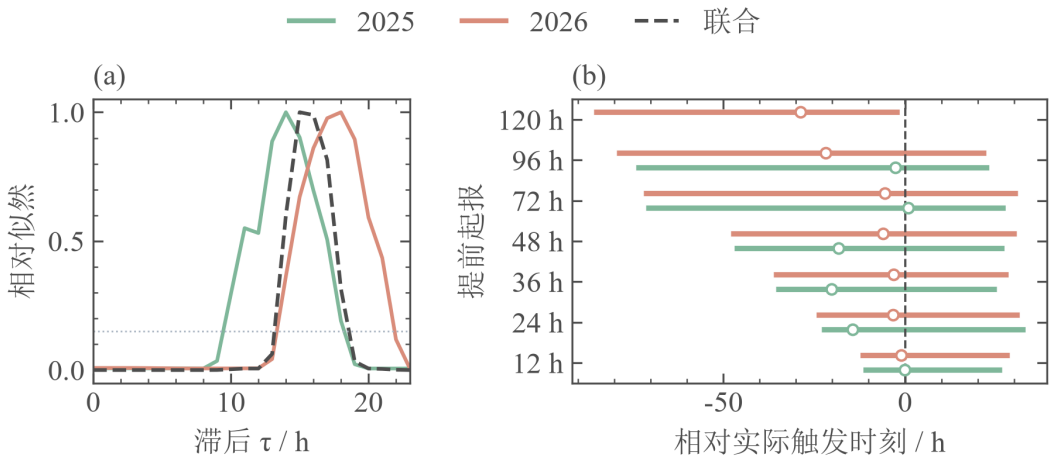


图 18　触发时间预报：(a) 滞后的似然；(b) 95% 置信区间

2025 年事件单独估计的峰值在 14 h，2026 年在 18 h，联合似然的峰值 $\hat{\tau} = 15$ h，似然不低于峰值 15% 的区间为 14 至 18 h。只用一次事件估计时，区间分别为 14 至 21 h 与

10 至 18 h，二者重叠，说明滞后可以在两次事件之间推广。午后 15 时左右的气温峰值经过约半天，在次日清晨至上午引发失稳，与两次事件的 06:00 与 10:30 相符。

6.3.6 求解结果与检验

在事件前 12 至 120 h 的 7 个时刻起报，结果见表 10 与图 18(b)，表中时间为月-日时:分，图中横线为 95% 置信区间，圆圈为中位数，0 为实际触发时刻。提前 12 h 起报时，2025 年事件的 95% 置信区间为 7 月 7 日 18:25 至 9 日 08:41，中位数 8 日 05:53，实际约 06:00；2026 年事件的区间为 8 月 25 日 22:11 至 27 日 15:12，中位数 26 日 09:25，实际 10:30。提前 12、24、48、72 h 时区间平均宽度依次约 40、56、77、101 h，提前量越大，区间越宽。

表 10　触发时间预报

事件	提前/h	触发概率	48 h 概率	2.5%	中位数	97.5%	宽度/h	覆盖
2025-07-08	12	0.159	0.159	07-07 18:25	07-08 05:53	07-09 08:41	38.3	是
2025-07-08	24	0.107	0.103	07-07 07:00	07-07 15:37	07-09 15:06	56.1	是
2025-07-08	36	0.089	0.082	07-06 18:30	07-07 09:47	07-09 07:10	60.7	是
2025-07-08	48	0.074	0.049	07-06 06:58	07-07 11:44	07-09 09:16	74.3	是
2025-07-08	72	0.050	0.011	07-05 06:35	07-08 06:50	07-09 09:41	99.1	是
2025-07-08	96	0.062	0.011	07-05 03:59	07-08 03:23	07-09 05:09	97.2	是
2025-07-08	120	0.000	0.000	无	无	无	无	否
2026-08-26	12	0.221	0.220	08-25 22:11	08-26 09:25	08-27 15:12	41.0	是
2026-08-26	24	0.174	0.162	08-25 10:03	08-26 07:15	08-27 17:56	55.9	是
2026-08-26	36	0.166	0.113	08-24 22:19	08-26 07:25	08-27 14:56	64.6	是
2026-08-26	48	0.115	0.075	08-24 10:32	08-26 04:23	08-27 17:09	78.6	是
2026-08-26	72	0.091	0.032	08-23 10:28	08-26 04:56	08-27 17:32	103.1	是
2026-08-26	96	0.169	0.038	08-23 03:09	08-25 12:41	08-27 08:52	101.7	是
2026-08-26	120	0.173	0.023	08-22 20:45	08-25 05:48	08-26 08:57	84.2	否

区间覆盖率。提前 96 h 以内的 12 个起报，用两次事件联合估计或只用一次事件估计 τ，95% 置信区间都覆盖了实际触发时刻，覆盖率为 100%。提前 120 h 时，2025 年事件尚未出现警戒日，触发概率为 0；2026 年事件的区间整体偏早，两者都未覆盖，全部起报的覆盖率为 85.7%。因此本模型的有效预见期约为 4 天。需要指出，覆盖率 100% 是在区间宽 40 至 101 h 的条件下取得的，宽度越大越容易覆盖，两者应结合起来看；而且 τ、q 与临界值都由这两次事件确定，覆盖率属于样本内检验，只用一次事件估计 τ 的结果也为 100%，说明 τ 的估计可以推广。

预警时间。警戒日的触发概率只有 0.17 左右，预警不能以 0.5 为判据。本文在 2024 至 2026 年汛期每日 08 时滚动起报，共 486 次，其中事件发生前 48 h 以内的起报有 4 次，每次事件 2 次。比较不同阈值 p^{*} 下 48 h 触发概率的命中率与误报率，结果见表 11 与图 19，命中率为这 4 次起报中发出预警的比例，误报率为其余起报中发出预警的比例；图 19(a) 中星号为选定阈值，图 19(b) 中虚线为 p^{*}，点线为事件日。

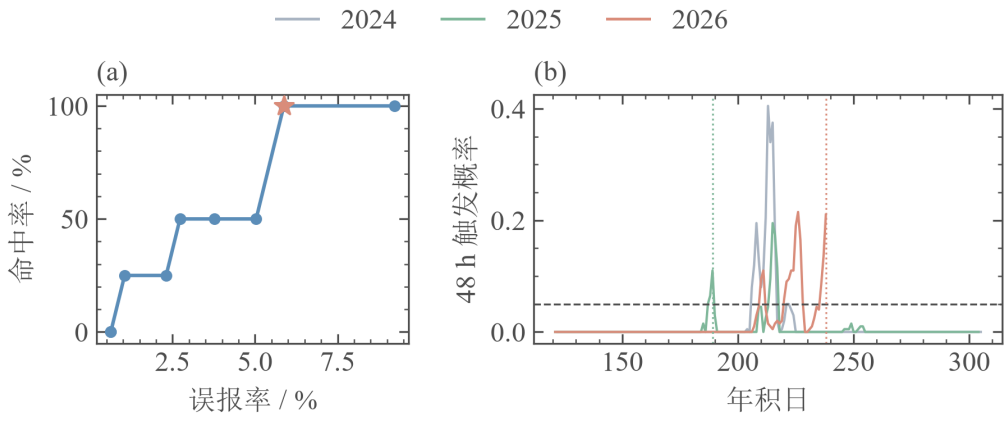


图 19　预警检验：(a) 命中率与误报率；(b) 滚动起报概率

p^* 由 0.02 增至 0.30 时，误报率由 9.2% 降至 0.4%，但 p^* 超过 0.05 后开始漏报。以命中率与误报率之差最大为准，取 $p^* = 0.05$，4 次起报全部命中，误报率 5.9%，共 28 天，约每个汛期 9 天。

表 11　预警阈值选择

p^*	命中率/%	误报率/%	预警日	误报日	命中率与误报率之差
0.02	100	9.22	51	44	0.908
0.05	100	5.87	34	28	0.941
0.08	50	5.03	28	24	0.450
0.10	50	3.77	21	18	0.462
0.12	50	2.73	15	13	0.473
0.15	25	2.31	12	11	0.227
0.20	25	1.05	6	5	0.240
0.25	0	0.63	3	3	-0.006
0.30	0	0.63	3	3	-0.006

按此判据，2025 年与 2026 年事件分别可提前 36 h 与 48 h 预警，且相应的 95% 置信区间覆盖实际时刻。

方法对比。在两次事件、提前 12 至 96 h 的 12 个起报上比较四种日内时刻模型：本文模型、不计滞后、日内均匀分配，以及不考虑警戒日触发概率即第一个警戒日必然触发。结果见表 12，耗时为单次起报 1000 次抽样的平均。

表 12　日内时刻模型对比

模型	覆盖率/%	平均宽度/h	中位数误差/h	24 h 内误差/h	耗时/ms
本文模型	100	72.9	4.6	1.5	145
不计滞后	100	73.6	6.4	5.5	125
日内均匀分配	100	76.3	6.1	3.3	121
首个警戒日必然触发	100	65.2	21.5	14.9	128

四种模型的覆盖率都是 100%，区间宽度相近，因为宽度主要取决于在哪一天触发。区别在中位数。对全部 12 次起报，本文模型的平均中位数误差为 4.6 h，不计滞后为 6.4 h，日内均匀分配为 6.1 h，首个警戒日必然触发为 21.5 h；对提前 24 h 以内的起报，四者依次为 1.5、5.5、3.3、14.9 h。首个警戒日必然触发的模型把触发时刻固定在第一个警戒日内，而 2025 年事件发生在第二个警戒日，因此误差最大。四者耗时相当，本文模型在不增加计算量的条件下误差最小。

空间距离范围。由问题四的 60 次蒙特卡洛仿真，以冰川末端为起点、95% 物质所到达的位置为停积距离，其中位数为 14.2 km，95% 区间为 0.5 至 24.3 km。其中 11 次达到灾区 DEM 下边界，因此上限 24.3 km 是计算区截断造成的，实际影响还会延伸到吉隆藏布下游。口岸位于 21.4 km 处，仿真中到达口岸的比例为 50%，到达时间的 95% 区间为 11.2 至 21.1 min。横向范围取基准情景中冰川末端下游 3 km 至口岸河段流深大于 0.5 m 的范围，离主流路距离的中位数为 109 m，95% 分位为 214 m；蒙特卡洛统计的横向距离含有离主流路 2 至 4 km 的释放区，不宜采用。

预警分区。综上，预警的空间范围为 Lhende 河自冰川末端起 24.3 km 以内、河道两侧 214 m 以内，包括口岸及其下游，并向吉隆藏布下游延伸。冰崩后最快 11.2 min 即到达口岸，现场撤离时间只有十余分钟，必须依靠提前 36 至 48 h 的气象预警组织疏散。按基准情景的到达时间分为四级，如图 20 所示，白点为沿程 0、5、10、15、20 km 处，黑色为堆积厚度超过 5 m 的位置：冰川末端以下 5 km 内 5 min 内到达，几乎没有避险时间；5 至 15 km 的河谷为 5 至 10 min；口岸附近为 10 至 15 min；口岸下游在 15 min 之后。

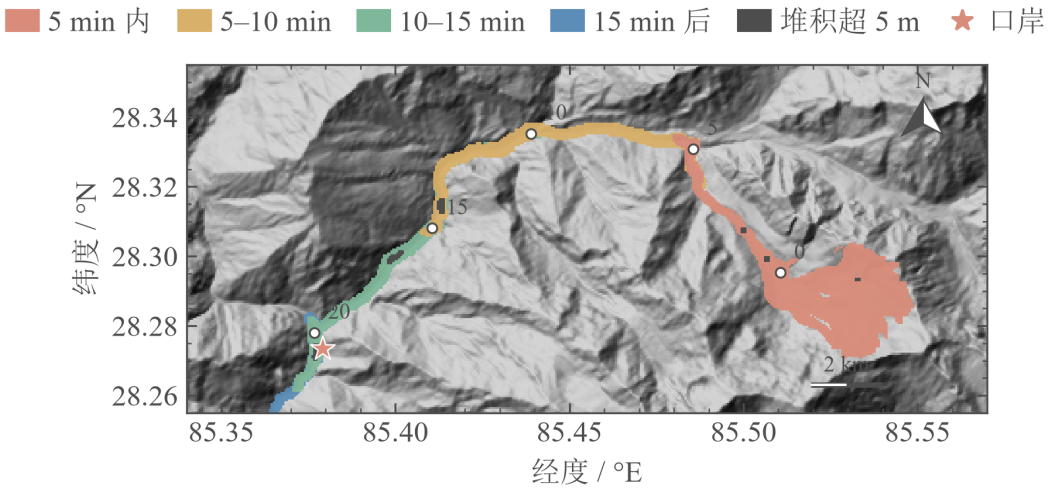


图 20　预警分区

6.4 问题四：粒子仿真模型

6.4.1 思路框架

仿真的路线如图 21 所示。由附件 4 冰厚积分确定释放体积，在填洼后的 DEM 上计算坡度，用粒子 Voellmy 模型逐步计算运动，各阶段分别取参数，泥石流阶段计入沿程侵蚀，输出到达时间与停积范围；再用 60 次蒙特卡洛给出参数不确定条件下的统计结果。

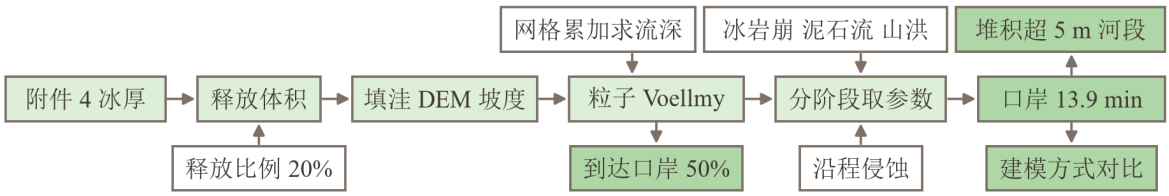


图 21　问题四思路

6.4.2 模型的建立

粒子受力如图 22 所示。把释放体积 V_0 平均分给 N 个粒子，每个粒子在 DEM 表面受沿坡重力分量驱动，受 Voellmy 阻力[18,19] 减速。综上，粒子仿真模型为

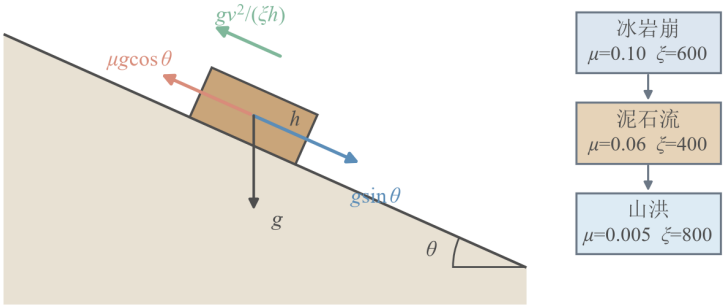


图 22　粒子受力示意

$$
\begin{cases}
\dfrac{\mathrm{d}\boldsymbol{v}}{\mathrm{d}t} = g\sin\theta\,\boldsymbol{e} - \left(\mu_k g\cos\theta + \dfrac{g|\boldsymbol{v}|^2}{\xi_k h}\right)\dfrac{\boldsymbol{v}}{|\boldsymbol{v}|}, & \boldsymbol{e} = -\nabla z/|\nabla z| \\
\dfrac{\mathrm{d}\boldsymbol{x}}{\mathrm{d}t} = \boldsymbol{v}, \qquad h = \dfrac{1}{\Delta A}\,\max_{3\times 3}\sum_{i\in \text{格}} V_i, & 1 \le h \le 30\ \mathrm{m} \\
\dfrac{\mathrm{d}V_i}{\mathrm{d}s} = \dfrac{b-1}{8000}V_i, \quad V_i \le bV_0/N, & \text{泥石流阶段} \\
k = \begin{cases} \text{冰岩崩}, & z > 3700\ \mathrm{m} \\ \text{泥石流}, & \text{越过冰川末端} \\ \text{山洪}, & A > 150\ \mathrm{km}^2,\ \tan\theta < 0.06 \end{cases}
\end{cases} \tag{6}
$$

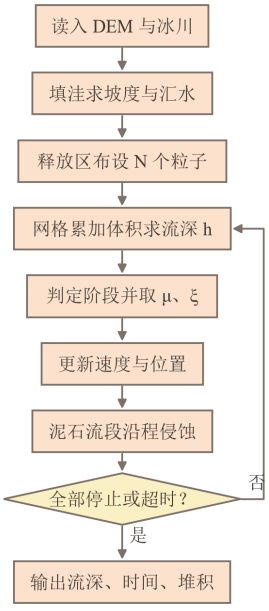
其中 b 为侵蚀使体积增大的倍数，A 为汇水面积。停止条件为 $|\boldsymbol{v}| < 0.05$ m/s 且 $\tan\theta < \mu_k$。灾区 DEM 为数字表面模型，窄深峡谷中存在深达 42 m 的虚假洼地，会把粒子困住，因此坡度由优先洪泛法填洼后的 DEM 计算。山洪阶段的 ξ 由曼宁公式估算：流深 5 m、坡降 0.05、糙率 0.05 时流速约 13 m/s，对应 $\xi \approx v^2/(hS) \approx 700$ m/s²，考虑到流深常大于 5 m，取 800 m/s²。释放体积取释放区冰量的 20%，这一比例缺少实测依据，因此在蒙特卡洛中让体积在 10^6 至 2×10^7 m³ 之间变化，相当于释放区冰量的 3% 至 53%。各参数见表 13。

表 13　仿真参数

参数	取值	说明
释放体积 V_0	7.56×10^6 m³	坡度大于 30°、海拔高于 5000 m 的冰体共 37.8×10^6 m³，取其 20%
冰岩崩 μ, ξ	0.1，600 m/s²	海拔 3700 m 以上
泥石流 μ, ξ	0.06，400 m/s²	沿程侵蚀，体积最多增至 2.0 倍
山洪 μ, ξ	0.005，800 m/s²	汇水面积 >150 km² 且纵坡 <0.06
网格、粒子数、步长	55 m×61 m，800，1.0 s	坡度由填洼后的 DEM 计算

6.4.3　求解算法

仿真算法如图 23 所示。



每个时间步先把粒子体积累加到网格上求流深，再按粒子所在位置的海拔、汇水面积与坡度确定阶段并取相应的 μ、ξ；速度采用半隐式格式更新，先加重力项，再按阻力减小速度的大小，避免阻力过大使速度反向；然后更新位置，泥石流阶段增大体积，直到全部粒子停止或计算时长达到 40 min。

图 23　仿真算法流程

6.4.4 仿真过程与结果

基准情景的过程如图 24 所示，蓝点为冰岩崩，黄点为泥石流，红点为山洪，星为口岸。

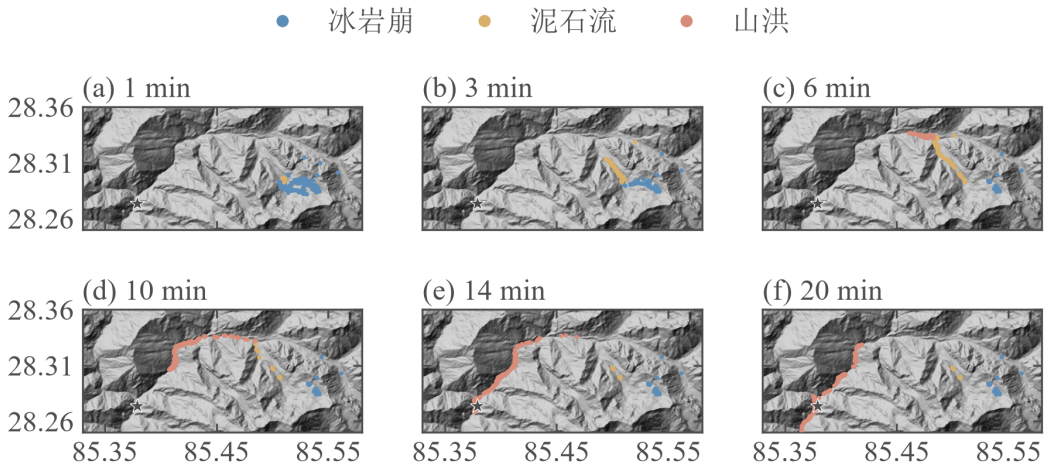


图 24　基准情景过程

冰岩崩在陡峭冰面上迅速加速，冰面最大速度约 85 m/s，前 2 min 粒子平均速度为 30 至 40 m/s；越过冰川末端后在较缓的河段减速，并铲刮沟床形成泥石流；进入 Lhende 河谷后沿程最大速度为 30 至 56 m/s，接近口岸时降至约 27 m/s。沿流路，前锋 3.2 min 到达冰川末端下游 3 km，4.3 min 到达 5 km，7.0 min 到达 10 km，9.6 min 到达 15 km，13.1 min 到达 20 km，13.9 min 到达 21.4 km 处的口岸，口岸处流深约 3.8 m。大部分泥石流在 14 km 附近减速堆积，稀释后的山洪继续下泄，3.4% 的物质越过口岸进入吉隆藏布下游。泥石流沿程侵蚀后总体积为 13.4×10^6 m^3。

最大流深与到达时间的分布见图 25，沿程剖面见图 26，图 26(a) 中堆积厚度放大 5 倍，虚线为口岸。

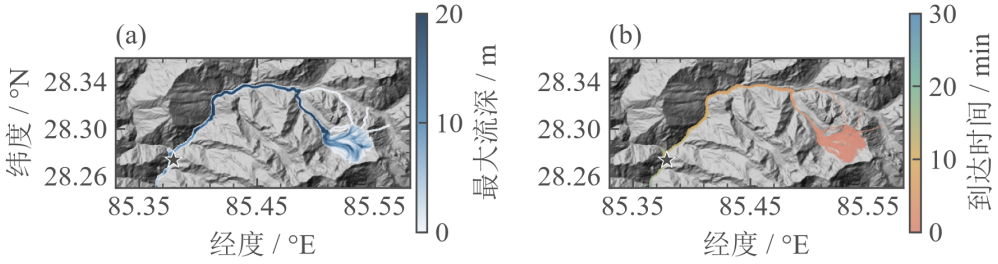


图 25　基准情景分布：(a) 最大流深；(b) 到达时间

堆积体积中位数所在位置为 14.1 km，14.0 至 23.7 km 河段有堆积厚度超过 5 m 的位置，超过一般河道的水深，可能堵塞河道形成堰塞湖。题目所述堰塞湖位于普热普强藏布与错坚河交汇处，本文的计算区与资料不足以确定二者是否对应，只能说明 Lhende 河段同样存在堵江风险。堰塞体溃决会形成第二次洪水，是灾后救援中需要重点防范的次生灾害。

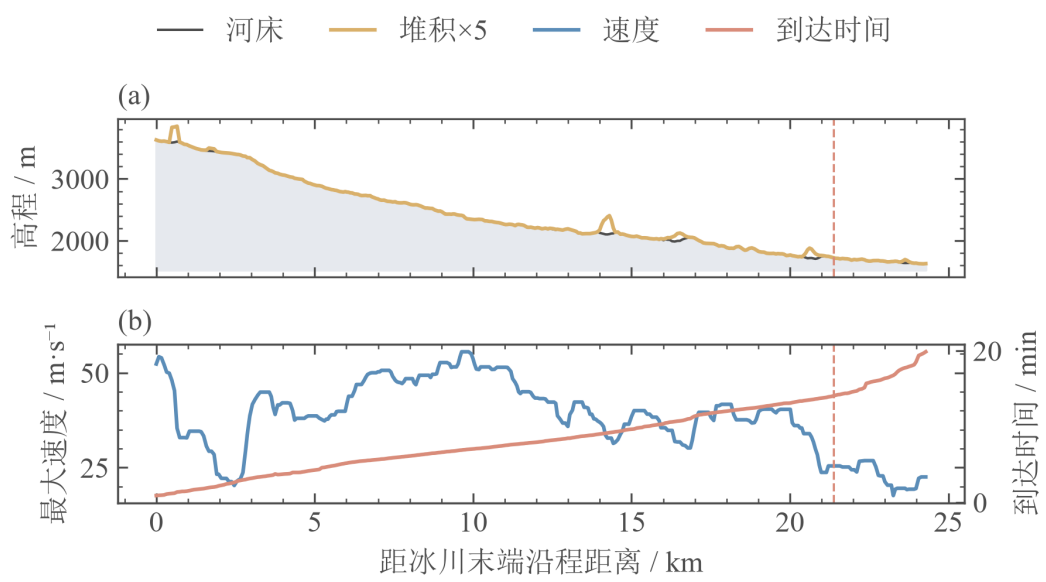


图 26　沿程剖面：(a) 高程与堆积；(b) 速度与到达时间

6.4.5　参数蒙特卡洛

释放体积在 10^{6} 至 2×10^{7} m³ 内按对数均匀抽样，泥石流阶段 μ 取 0.04 至 0.10、ξ 取 200 至 600，山洪阶段 μ 取 0.002 至 0.01、ξ 取 500 至 1200，侵蚀增大倍数 b 取 1.5 至 3，共抽样 60 次，每次 300 个粒子。到达口岸的比例为 50%，逻辑回归得到到达概率为 50% 时的体积约为 5.3×10^{6} m³，如图 27(a) 所示，曲线为逻辑回归。

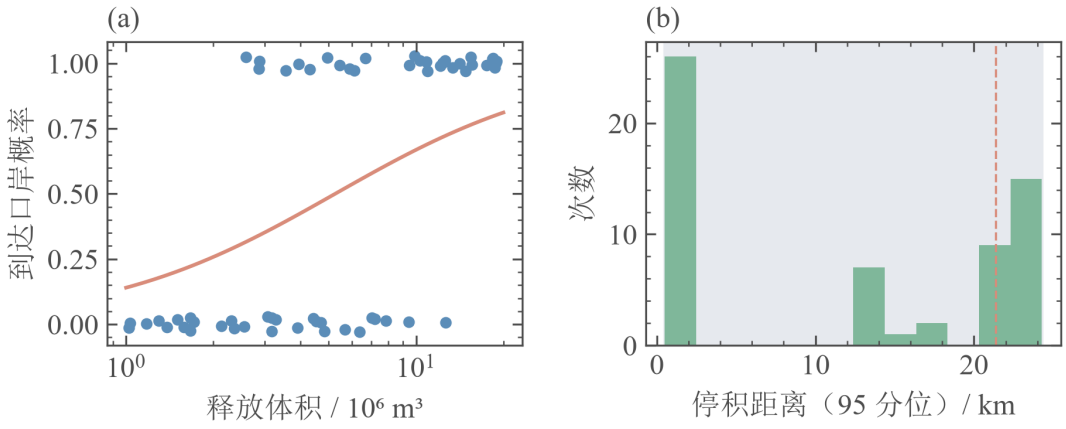


图 27　蒙特卡洛结果：(a) 到达概率；(b) 停积距离

停积距离的 95% 区间为 0.5 至 24.3 km，分布呈双峰，如图 27(b) 所示，灰带为 95% 区间，虚线为口岸。26 次仿真在冰川末端下游 2.5 km 以内停止，这些情景体积小，中位数 2.8×10^{6} m³，泥石流摩擦系数大，均值 0.085；其余 34 次都进入 12.6 至 24.3 km 的峡谷段或越过口岸，体积中位数 9.7×10^{6} m³，摩擦系数均值 0.066。可见冰崩一旦冲出冰川末端并完成侵蚀，就很可能一直到达口岸，很少在中途停止，这对预警不利。各参数与停积距离的 Spearman 相关见表 14。

表 14　参数相关性

参数	相关系数
泥石流 μ	-0.695
释放体积	0.652
泥石流 ξ	0.187
山洪 μ	-0.184
侵蚀增大倍数	-0.130
山洪 ξ	-0.076

6.4.6　方法对比

为检验分阶段取参数与沿程侵蚀是否必要，在基准情景下比较四种建模方式：本文的分阶段参数与侵蚀、三个阶段都用泥石流参数、不计侵蚀、以及两者都不考虑，结果见表 15，耗时为单次运行时间。

表 15　建模方式对比

方式	到达口岸/min	堆积中位/km	越界/%	体积/10^6 m^3	耗时/s
分阶段参数与侵蚀	13.8	14.2	3.9	13.4	61
单一泥石流参数	未到达	11.3	0.0	14.1	58
不计侵蚀	未到达	13.8	0.0	7.6	57
单一参数且不计侵蚀	未到达	11.2	0.0	7.6	56

只有本文的分阶段参数与侵蚀能使前锋到达口岸，用时 13.8 min；三个阶段都用泥石流参数时，物质在主河道中按泥石流的摩擦减速，堆积中位位置后退到 11.3 km，前锋到不了口岸；不计侵蚀时体积保持 7.6×10^6 m^3，流深与速度不足，同样到不了口岸。两次实际事件中洪水都冲到了口岸，只有本文的建模方式与这一事实相符，说明进入主河后的稀释与沿程侵蚀是链式过程不可缺少的两个环节。四种方式耗时都在 1 min 左右，计算量相当。

6.4.7　模型检验

数值收敛。基准情景下取粒子数 200、400、800、1600 与时间步长 0.5、1、2 s，共 12 种组合重新计算，结果见表 16 与图 28。

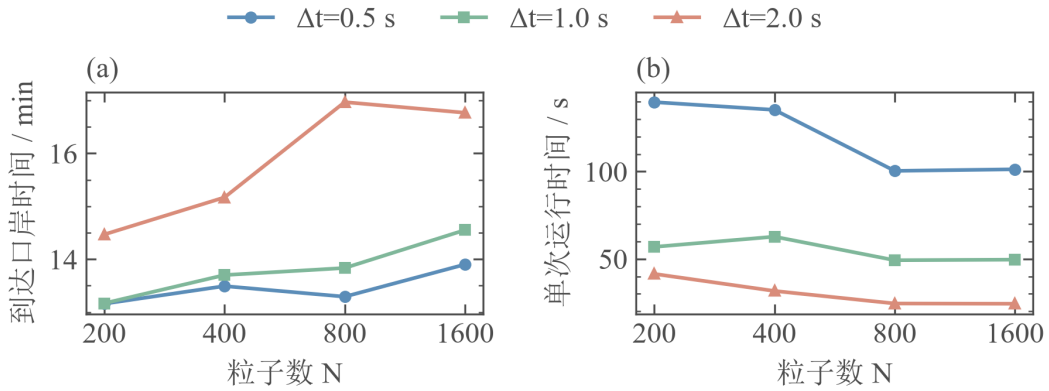


图 28　数值收敛：(a) 到达时间；(b) 运行时间

收敛试验的计算时长为 30 min。步长不超过 1 s 时，各粒子数下到达口岸的时间在 13.2 至 14.6 min 之间，相差不到 1.5 min；步长取 2 s 时到达时间推迟到 14.5 至 17.0 min，说明步长过大会低估运动速度。堆积体积中位数所在位置在所有组合中都稳定在 14.0 至 14.2 km。但停积距离与越过口岸的比例仍随粒子数变化，1 s 步长下粒子数由 800 增至

1600，停积距离由 23.6 km 降至 20.9 km，越过口岸的比例由 3.9% 降为 0，说明少量前锋物质的去向对粒子数敏感。运行时间主要取决于步长，步长减半耗时约增加一倍，而与粒子数关系不大，因为每步的主要计算量是网格累加与滤波。因此取 800 个粒子、1 s 步长，到达时间与堆积位置已收敛，耗时约 50 s，前锋越界比例应视为有较大不确定性的结果。

表 16　数值收敛检验

粒子数	步长/s	到达口岸/min	停积距离/km	堆积中位/km	耗时/s
200	0.5	13.2	24.3	14.2	139.7
400	0.5	13.5	24.3	14.1	135.3
800	0.5	13.3	24.3	14.2	100.4
1600	0.5	13.9	21.9	14.2	101.2
200	1.0	13.2	24.3	14.1	56.9
400	1.0	13.7	24.3	14.0	62.7
800	1.0	13.8	23.6	14.2	49.2
1600	1.0	14.6	20.9	14.1	49.6
200	2.0	14.5	24.3	14.2	41.4
400	2.0	15.2	21.9	14.2	31.6
800	2.0	17.0	20.8	14.0	24.5
1600	2.0	16.8	20.8	14.1	24.3

经验公式复核。大型岩崩的落差与水平运动距离之比 H/L 与体积 V 有经验关系 $\lg(H/L) = -0.15666\lg V + 0.62419$[25]。基准情景释放区中位海拔 5439 m，堆积中位处海拔 2102 m，水平运动距离 16.8 km，仿真的 H/L 为 0.199；同体积干岩崩的经验值为 0.352，对应运动距离 9.5 km。仿真的运动距离大于干岩崩，原因是冰崩体含冰，沿程铲刮饱水沟床并转化为泥石流，等效摩擦明显降低。Chamoli 等冰岩崩泥石流的运动距离也远大于同体积干岩崩[20]。偏差方向与这一机理相符。另外，仿真中前锋到达并越过口岸，与两次事件中洪水冲毁口岸大桥的实际情况一致，但由于缺少实测流速与到达时间，仿真的到达时间还无法直接检验。

7　灵敏度分析

三个关键参数的灵敏度如图 29 所示。

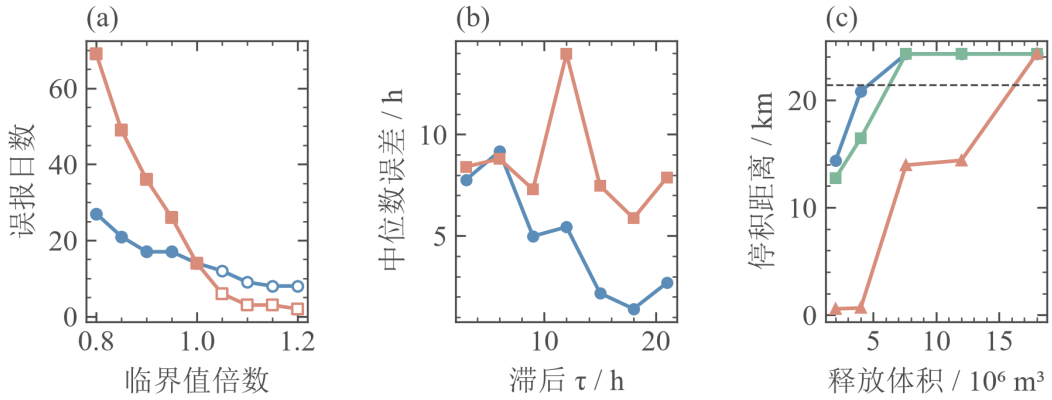


图 29　灵敏度分析：(a) 临界值；(b) 滞后；(c) 体积与摩擦系数

图 29(a) 为两个临界值分别乘以 0.80 至 1.20 时的误报日数，圆点为 c_1，方块为 c_2，实心表示两次事件都报中，空心表示有漏报：临界值下调时误报迅速增加，上调 5% 即出现漏报，双因子模型对临界值敏感。图 29(b) 为固定 τ 时的中位数误差，圆点为提前

24 h 以内的起报，方块为全部起报。τ 在 3 至 21 h 内覆盖率始终为 100%；提前 24 h 以内的误差在 τ 为 15 至 18 h 时最小，为 1.4 至 2.2 h，τ 取其他值时为 2.7 至 9.2 h。全部起报的误差波动较大，固定 $\tau = 15$ h 时为 7.5 h，而 τ 在 14 至 18 h 内随机取值时为 4.6 h，原因是提前 36 h 以上起报时触发时刻的分布跨越多个警戒日，中位数落在哪一天对抽样很敏感，这类起报应以区间为准。图 29(c) 为不同释放体积与泥石流摩擦系数下的停积距离，圆点、方块、三角依次为 $\mu = 0.05$、0.065、0.08，虚线为口岸。$\mu = 0.05$ 时释放 4×10^6 m^3 前锋即可到达口岸，μ 为 0.065 与 0.08 时需要 7.6×10^6 m^3；体积为 2×10^6 m^3 时三种摩擦系数下都到不了口岸。$\mu = 0.08$ 时，体积达到 1.8×10^7 m^3 停积距离才越过口岸，停积距离对体积与摩擦系数都敏感。

临界条件的窗口与变量。表 7 中前 8 种组合都能 2/2 报中，误报比例在 0.58% 至 2.13% 之间，说明约 2 周的前期降水与 1 至 5 日的正积温这一组合对正积温的窗口不敏感；把正积温换成 VPD 或 DTR，误报最少只有 6 天，但留一检验降为 1/2。

临界值扰动。把 c_1、c_2 分别乘以 0.9、1.0、1.1，共 9 种组合：c_2 降低 10% 时两次事件仍全部报中，但误报由 14 天增至 36 天；c_2 提高 10% 时误报降到 3 天，但漏报 2026 年事件；c_1 提高 10% 则漏报 2025 年事件。临界值只由两次事件确定，余量很小，因此问题三把 c_1、c_2 作为随机量处理。

滞后的估计区间。改用只用一次事件估计的区间 10 至 21 h 时，提前 96 h 以内的覆盖率仍为 100%。提前 12 h 时两种估计的中位数相差不超过 2 h；提前 24 至 36 h 时，2025 年事件相差可达 19 h，因为此时触发时刻的分布跨越两个警戒日并呈双峰，中位数落在哪一天对 τ 敏感。因此 12 h 以内的时刻估计可靠，更早的起报应以区间为准，不宜只看中位数。

仿真参数。由表 14，泥石流阶段摩擦系数与释放体积对停积距离影响最大，相关系数为 -0.70 和 0.65。按体积分组，V_0 不超过 3×10^6 m^3 时到达口岸的比例为 18%，超过 10^7 m^3 时为 94%。山洪阶段参数与侵蚀增大倍数的相关系数绝对值不超过 0.19，主要影响口岸以下的传播。

8 模型的评价、改进与推广

8.1 模型的优点

1. 用同一流域的两次同类事件相互验证，结论都要求两次事件方向一致，减小了单个事件的偶然性。
2. 补充数据与附件数据相互印证：再分析降水排除了降雨直接触发，卫星 DEM 用于空间评估，存档预报使预警检验接近实际业务条件。
3. 预报给出概率与置信区间，并用滚动起报统计命中与误报，评价方式客观。
4. 仿真的释放区与冰量由附件 4 冰厚算出，摩擦参数参考文献取值，并用蒙特卡洛与收敛检验量化了不确定性。

8.2 模型的缺点

1. 观测塔距灾点 65 km，源区气温靠直减率外推，降水的代表性弱；只有两次事件，临界值、τ 与预警阈值都由这两次事件确定，命中率与覆盖率主要是样本内结果。
2. 粒子模型没有求解完整的浅水方程，释放比例缺少实测依据，计算区下边界截断了下游的影响范围，堵江与溃坝过程只能定性判断。

8.3 改进与推广

在源区布设自动气象站与冰面位移监测，可以用实测气温代替直减率外推；引入 Sentinel-1 干涉形变与 Sentinel-2 冰面湖监测，可以在临界条件中加入冰体形变因子。本

文的双因子临界、滞后概率预报与粒子仿真方法，可推广到喜马拉雅其他存在冰湖溃决与冰崩风险的流域。

参考文献

[1] 马耀明，马伟强，谢志鹏等，西藏吉隆高寒山区地气相互作用逐小时综合观测数据集（2021–2026），https://www.tpdc.ac.cn/zh-hans/data/5b09afd1-d11b-4150-a291-50f0484dddf1，2026 年 10 月 28 日。

[2] 国家青藏高原科学数据中心，2005–2025 年吉隆县及周边县市逐日平均气温数据，https://www.tpdc.ac.cn/zh-hans/data/94d1bc8e-b912-413e-bd2b-851f2ff8f8f7，2026 年 10 月 28 日。

[3] 国家青藏高原科学数据中心，日喀则市吉隆县特大洪水——200 km 周围内的冰川及冰川表面高程变化数据集（2000–2020），https://www.tpdc.ac.cn/zh-hans/data/f94c33c4-f521-4247-80ec-55dfa2dd453a，2026 年 10 月 28 日。

[4] 赵洪宇，张通，吉隆口岸山洪灾害——蓝塘里壤峰冰崩前冰川地形、冰厚及表面速度数据集，https://www.tpdc.ac.cn/zh-hans/data/c2875b3e-7fd4-4b38-9814-fef64193d1a4，2026 年 10 月 28 日。

[5] 国家青藏高原科学数据中心,西藏日喀则市吉隆县泥石流灾害区土地覆盖数据（2020–2023），https://www.tpdc.ac.cn/zh-hans/data/4e1e74fb-dcee-4ce4-8b07-ec3f3109eabe，2026 年 10 月 28 日。

[6] 国家青藏高原科学数据中心，西藏日喀则市吉隆县泥石流灾害区历史灾点数据，https://www.tpdc.ac.cn/zh-hans/data/db142594-2819-403b-8da0-5ebb2a0ca951，2026 年 10 月 28 日。

[7] 国家青藏高原科学数据中心，2026 年 8 月 26 日日喀则市吉隆县泥石流灾害区 DEM 数据，https://www.tpdc.ac.cn/zh-hans/data/88b5ab22-19a1-480f-bcdc-cec41776ab46，2026 年 10 月 28 日。

[8] 国家青藏高原科学数据中心，西藏日喀则市吉隆县泥石流灾害区吉隆县第七次人口普查数据，https://www.tpdc.ac.cn/zh-hans/data/8662c472-43d6-4e92-bff8-e26923970642，2026 年 10 月 28 日。

[9] Hugonnet R, McNabb R, Berthier E, et al. Accelerated global glacier mass loss in the early twenty-first century. Nature, 592(7856): 726–731, 2021.

[10] Hersbach H, Bell B, Berrisford P, et al. The ERA5 global reanalysis. Quarterly Journal of the Royal Meteorological Society, 146(730): 1999–2049, 2020.

[11] Muñoz-Sabater J, Dutra E, Agustí-Panareda A, et al. ERA5-Land: a state-of-the-art global reanalysis dataset for land applications. Earth System Science Data, 13(9): 4349–4383, 2021.

[12] Open-Meteo，Historical Weather API 与 Previous Runs API，https://open-meteo.com，2026 年 10 月 28 日。

[13] European Space Agency，Copernicus DEM GLO-30，https://copernicus-dem-30m.s3.amazonaws.com，2026 年 10 月 28 日。

[14] Li S, Ji Q, Ge Q, et al. Annual 30 m land cover dataset on the Tibetan Plateau from 1990 to 2023，https://doi.org/10.5281/zenodo.13989241，2026 年 10 月 28 日。

[15] Caine N. The rainfall intensity–duration control of shallow landslides and debris flows. Geografiska Annaler A, 62(1–2): 23–27, 1980.

[16] Guzzetti F, Peruccacci S, Rossi M, et al. The rainfall intensity–duration control of shallow landslides and debris flows: an update. Landslides, 5(1): 3–17, 2008.

[17] Dahal R K, Hasegawa S. Representative rainfall thresholds for landslides in the Nepal Himalaya. Geomorphology, 100(3–4): 429–443, 2008.

[18] Voellmy A. Über die Zerstörungskraft von Lawinen. Schweizerische Bauzeitung, 73: 159–165, 1955.

[19] Christen M, Kowalski J, Bartelt P. RAMMS: numerical simulation of dense snow avalanches in three-dimensional terrain. Cold Regions Science and Technology, 63(1–2): 1–14, 2010.

[20] Shugar D H, Jacquemart M, Shean D, et al. A massive rock and ice avalanche caused the 2021 disaster at Chamoli, Indian Himalaya. Science, 373(6552): 300–306, 2021.

[21] The Kathmandu Post，Scientists suspect ice avalanche triggered Bhotekoshi flood，https://kathmandupost.com/national/2026/08/26/scientists-suspect-ice-avalanche-triggered-bhotekoshi-flood，2026 年 10 月 28 日。

[22] Himal Press，What caused Rasuwagadhi floods，https://en.himalpress.com/what-caused-rasuwagadhi-floods/，2026 年 10 月 28 日。

[23] 中华人民共和国自然资源部，地球科普：泥石流形成的基本条件，https://www.mnr.gov.cn/zt/hd/dqr/43earthday/dqkp/201203/t20120316_2057013.html，2026 年 10 月 28 日。

[24] deepseek，2026 年 10 月 23 日至 27 日。

[25] Scheidegger A E. On the prediction of the reach and velocity of catastrophic landslides. Rock Mechanics, 5(4): 231–236, 1973.

AI 工具使用声明

本队将 deepseek 作为资料与文档管理的辅助工具[24]，于 2026 年 10 月 23 日至 27 日期间使用。使用内容包括查找气象、预报及地形数据的获取渠道，检索吉隆灾害背景和数据产品说明，归整既有程序的文件关系与运行指引，以及规范图表、引文和附件的格式。这些工作分别对应数据获取、资料查询、代码整理与格式调整四项用途，具体对象及文件安排见支撑材料中的《AI 工具使用详情说明》。

附录

A 支撑材料文件清单

```
支撑材料/
    code/              全部源程序，见附录 B，shumo_tools/ 为绘图样式工具
    results/           中间结果与最终结果
        q1_daily.csv               34 项逐日特征
        q1_association.csv         关联指标与排序
        q2_threshold_combos.csv    20 种双因子组合
        q2_critical_daily.csv      逐日临界判别
        q2_hazard_points.csv       84 条灾点的易发性
        q3_forecast_eval.csv       各起报时刻的区间与覆盖
        q3_rolling.csv             2024 至 2026 年汛期滚动起报
        q4_profile.csv             基准情景沿程剖面
        q4_mc.csv                  60 次蒙特卡洛
        cmp_*.csv, sens_*.csv      方法对比与灵敏度
```

```
    *_summary.json          各问关键数字
figures/        全部图，png，300 dpi
data_ext/       补充数据：ERA5、ERA5-Land、历史预报、Copernicus DEM
AI工具使用详情说明.pdf
```

B 软件、运行命令与源程序

软件为 Python 3.12，所用库有 pandas、numpy、scipy、scikit-learn 1.9、rasterio 1.5、netCDF4、joblib、matplotlib 3.11 与 libarchive-c，论文用 XeLaTeX 排版。在 code 目录依次运行：

```
python q1.py ; python q2.py ; python q4.py ; python q3.py
python compare.py ; python compare_q4.py ; python cmp_q4b.py
python figs_res.py ; python figs_schema.py
python figs_gis.py ; python tables.py
```

补充数据的获取方法见说明文档:再分析取自 Open-Meteo Archive API 的 era5 与 era5_land 模型，历史预报取自 Previous Runs API，时段为 2024-01-01 至 2026-08-27，卫星 DEM 为 Copernicus GLO-30 的 N28E085 图幅。

B.1 common.py 公共模块：观测读取、质控与逐日特征

```python
from pathlib import Path
import glob
import numpy as np
import pandas as pd

ROOT = Path(__file__).resolve().parent.parent
DATA, EXT, RES, FIG = ROOT / "data", ROOT / "data" / "ext", ROOT / "results", ROOT / "figures"
RES.mkdir(exist_ok=True)
FIG.mkdir(exist_ok=True)
SEED = 2026

STATION = (28.86, 85.29, 4140.0)
PORT = (28.2735, 85.3790, 1850.0)
SRC_Z = 5000.0
LAPSE = 0.0065

EVENTS = {"2025-07-08": pd.Timestamp("2025-07-08 06:00"), "2026-08-26": pd.Timestamp("
    2026-08-26 10:30")}
DATA_END = pd.Timestamp("2026-08-27 08:00")

def load_station(kind):
    d = pd.concat([pd.read_csv(f) for f in sorted(glob.glob(str(DATA / "d2" / f"{kind}_*.csv")
    ))])
    d["t"] = pd.to_datetime(d.Timestamp, format="mixed")
    d = d.set_index("t").drop(columns="Timestamp").sort_index()
    d = d[~d.index.duplicated()].apply(pd.to_numeric, errors="coerce")
    qc = [c for c in d.columns if c.startswith("QC")]
    for c in qc:
        v = c.replace("QC_", "").lower()
        for x in [x for x in d.columns if x.lower().startswith(v) and not x.startswith("QC")]:
            d.loc[~d[c].isin([0, 4]), x] = np.nan
    d = d.drop(columns=qc).replace([9999.9, -9999.9], np.nan)
    return d.loc[:DATA_END]

def hourly():
    m, r, s, f = (load_station(k) for k in ["MET", "RADM", "SOIL", "FLUX"])
    h = pd.DataFrame(index=pd.date_range("2021-01-01", DATA_END, freq="h"))
    h["P"] = m["Precip(mm)"]
    for z in [1, 2, 4, 10, 20]:
        h[f"T{z}"] = m[f"Ta_{z}m(degree C)"]
        h[f"U{z}"] = m[f"WS_{z}m(m/s)"]
    h["RH"] = m["RH_2m(%)"]
    h["Pa"] = m["Pressure(hPa)"]
    h["WD10"] = m["WD_10m(degree)"]
    h["Rsd"], h["Rsu"], h["Rld"], h["Rlu"] = r["Rsd(W/m2)"], r["Rsu(W/m2)"], r["Rld(W/m2)"], r
    ["Rlu(W/m2)"]
```

```
    h["Rn"] = h.Rsd - h.Rsu + h.Rld - h.Rlu
    for z in ["0.1", "0.4", "1.6"]:
        h[f"VWC{z}"] = s[f"VWC_{z}m(m3/m3)"]
        h[f"Ts{z}"] = s[f"soilT_{z}m(degree C)"]
    h["G"] = s["G_0.1m(W/m2)"]
    h["H"], h["LE"], h["USTAR"], h["TKE"] = f["H(W/m2)"], f["LE(W/m2)"], f["USTAR(m/s)"], f["
    TKE(m2/s2)"]

    h["dTdz"] = (h.T20 - h.T1) / 19 * 100
    h["shear"] = (h.U20 - h.U1) / 19
    with np.errstate(divide="ignore", invalid="ignore"):
        lu = np.log(h[["U1", "U2", "U4", "U10", "U20"]].clip(lower=0.1))
        lz = np.log([1, 2, 4, 10, 20])
        lzc = lz - lz.mean()
        h["alpha"] = (lu.sub(lu.mean(1), axis=0) * lzc).sum(1) / (lzc ** 2).sum()
        th = (h.T20 + 273.15) - (h.T1 + 273.15) + 0.0098 * 19
        h["Rb"] = 9.81 / (h.T2 + 273.15) * th * 19 / ((h.U20 - h.U1) ** 2 + 0.1)
    es = 0.6108 * np.exp(17.27 * h.T2 / (h.T2 + 237.3))
    h["VPD"] = es * (1 - h.RH / 100)

    h["Tsrc"] = h.T2 - LAPSE * (SRC_Z - STATION[2])
    return h

def daily(h=None):
    h = hourly() if h is None else h
    g = h.resample("D")
    d = pd.DataFrame({
        "P": g.P.sum(min_count=18), "Ta": g.T2.mean(), "Tmax": g.T2.max(), "Tmin": g.T2.min(),
        "RH": g.RH.mean(), "VPD": g.VPD.mean(), "Pa": g.Pa.mean(), "U10": g.U10.mean(),
        "Rsd": g.Rsd.mean(), "Rn": g.Rn.mean(), "Rld": g.Rld.mean(),
        "VWC0.1": g["VWC0.1"].mean(), "VWC0.4": g["VWC0.4"].mean(), "VWC1.6": g["VWC1.6"].mean
    (),
        "Ts0.1": g["Ts0.1"].mean(), "G": g.G.mean(),
        "H": g.H.mean(), "LE": g.LE.mean(), "USTAR": g.USTAR.mean(), "TKE": g.TKE.mean(),
        "dTdz": g.dTdz.mean(), "shear": g.shear.mean(), "alpha": g.alpha.median(), "Rb": g.Rb.
    median(),
        "PDDsrc": g.Tsrc.apply(lambda x: np.clip(x, 0, None).mean() if x.notna().sum() >= 18
    else np.nan),
    })
    night = h[(h.index.hour >= 20) | (h.index.hour < 8)]
    d["inv_frac"] = (night.dTdz > 0).groupby(night.index.floor("D")).mean()
    d["DTR"] = d.Tmax - d.Tmin
    d["Bowen"] = d.H / d.LE.where(d.LE.abs() > 5)
    d["albedo"] = (g.Rsu.sum() / g.Rsd.sum()).where(g.Rsd.sum() > 500)

    w = 0.9 ** np.arange(15)
    d["API15"] = d.P.fillna(0).rolling(15).apply(lambda x: (x[::-1] * w).sum(), raw=True)
    d["P7"], d["P30"] = d.P.rolling(7, min_periods=5).sum(), d.P.rolling(30, min_periods=24).
    sum()
    d["PDD7"] = d.PDDsrc.rolling(7, min_periods=5).sum()
    d["dPa"] = d.Pa.diff()
    d["dT3"] = d.Ta - d.Ta.shift(3)
    return d

def load_era5(point, model="era5"):
    cols = ["T", "P", "rain", "snow", "sd", "sm1", "sm2", "rh", "sw"]
    f = EXT / f"{model}_{point}.csv"
    head = open(f).readlines()[1].split(",")
    d = pd.read_csv(f, skiprows=3, parse_dates=["time"]).set_index("time")
    d.columns = cols
    d.attrs["lat"], d.attrs["lon"], d.attrs["z"] = map(float, head[:3])
    return d

FEATS = {
    "P": ("日降水", "天气过程"), "API15": ("前期有效降水", "天气过程"), "P7": ("7 日降水", "天气过程")
    ,
    "P30": ("30 日降水", "天气过程"), "Ta": ("日均气温", "热力"), "Tmax": ("日最高气温", "热力"),
    "DTR": ("气温日较差", "热力"), "dT3": ("3 日升温", "热力"), "PDDsrc": ("源区正积温", "热力"),
    "PDD7": ("源区 7 日正积温", "热力"), "RH": ("相对湿度", "水汽"), "VPD": ("饱和水汽压差", "水汽"),
    "Pa": ("气压", "天气过程"), "dPa": ("24 h 变压", "天气过程"), "U10": ("10 m 风速", "边界层"),
    "Rsd": ("向下短波", "辐射"), "Rn": ("净辐射", "辐射"), "Rld": ("向下长波", "辐射"), "albedo": ("
    反照率", "辐射"),
    "VWC0.1": ("10 cm 土壤含水量", "土壤"), "VWC0.4": ("40 cm 土壤含水量", "土壤"), "VWC1.6": ("160
     cm 土壤含水量", "土壤"),
    "Ts0.1": ("10 cm 土温", "土壤"), "G": ("土壤热通量", "土壤"), "H": ("感热通量", "湍流"), "LE": (
    "潜热通量", "湍流"),
```

```
    "Bowen": ("波文比", "湍流"), "USTAR": ("摩擦速度", "湍流"), "TKE": ("湍流动能", "湍流"),
    "dTdz": ("近地层温度梯度", "边界层"), "inv_frac": ("夜间逆温频率", "边界层"), "shear": ("风切变",
    "边界层"),
    "alpha": ("风廓线指数", "边界层"), "Rb": ("整体理查森数", "边界层"),
}
```

B.2 q1.py 问题一：关联排序

```
import json
import numpy as np
import pandas as pd
from common import RES, EVENTS, FEATS, hourly, daily

CUM = {"API15", "P7", "P30", "PDD7"}
SUM = {"P"}

def window_stat(d, f, day):
    if f in CUM:
        return d[f].get(day - pd.Timedelta(days=1), np.nan)
    w = d[f].loc[day - pd.Timedelta(days=3): day - pd.Timedelta(days=1)]
    return w.sum(min_count=2) if f in SUM else w.mean()

def clim(d, f, day, excl):
    doy = d.index.dayofyear
    ok = (np.abs(doy - day.dayofyear) <= 30)
    for e in excl:
        ok &= ~((d.index >= e - pd.Timedelta(days=15)) & (d.index <= e + pd.Timedelta(days=15)
    ))
    vals = [window_stat(d, f, x) for x in d.index[ok]]
    return np.array([v for v in vals if np.isfinite(v)])

def entropy_topsis(M):
    X = (M - M.min(0)) / (M.max(0) - M.min(0) + 1e-12)
    P = (X + 1e-12) / (X + 1e-12).sum(0)
    e = -(P * np.log(P)).sum(0) / np.log(len(M))
    w = (1 - e) / (1 - e).sum()
    V = X * w
    dp = np.sqrt(((V - V.max(0)) ** 2).sum(1))
    dn = np.sqrt(((V - V.min(0)) ** 2).sum(1))
    return w, dn / (dp + dn)

def main():
    h = hourly()
    d = daily(h)
    d.to_csv(RES / "q1_daily.csv")
    days = {k: v.floor("D") for k, v in EVENTS.items()}
    rows = []
    for f in FEATS:
        r = {"feature": f, "name": FEATS[f][0], "group": FEATS[f][1]}
        s_all, z_all, per_all = [], [], []
        for k, day in days.items():
            x = window_stat(d, f, day)
            c = clim(d, f, day, list(days.values()))
            if not np.isfinite(x) or len(c) < 30:
                s_all.append(np.nan); z_all.append(np.nan); per_all.append(0)
                continue
            p = (np.sum(c < x) + 0.5 * np.sum(c == x)) / len(c)
            z = (x - c.mean()) / (c.std() + 1e-12)

            early = d[f].loc[day - pd.Timedelta(days=10): day - pd.Timedelta(days=4)].mean()
            late = d[f].loc[day - pd.Timedelta(days=3): day - pd.Timedelta(days=1)].mean()
            sd = d[f][np.abs(d.index.dayofyear - day.dayofyear) <= 30].std()
            per = np.sign(2 * p - 1) * (late - early) / (sd + 1e-12) if np.isfinite(early) and
     np.isfinite(late) else 0
            r[f"x_{k}"], r[f"clim_mean_{k}"], r[f"p_{k}"], r[f"z_{k}"] = x, c.mean(), p, z
            s_all.append(2 * p - 1); z_all.append(z); per_all.append(per)
        s = np.array(s_all)
        r["rarity"] = np.nanmean(np.abs(s))
        r["consistency"] = abs(np.nansum(s)) / (np.nansum(np.abs(s)) + 1e-12)
        r["trend"] = np.mean(per_all)
        r["absz"] = np.nanmean(np.abs(z_all))
        r["direction"] = "偏高" if np.nansum(s) > 0 else "偏低"
        rows.append(r)
    A = pd.DataFrame(rows).dropna(subset=["rarity"])
    M = A[["rarity", "consistency", "trend", "absz"]].values
```

```
    w, cc = entropy_topsis(M)
    A["closeness"] = cc
    A = A.sort_values("closeness", ascending=False).reset_index(drop=True)
    A["rank"] = np.arange(1, len(A) + 1)

    sel = list(A.drop_duplicates("group").feature[:5])
    A["top5"] = A.feature.isin(sel)
    A.to_csv(RES / "q1_association.csv", index=False)

    Xn = (M - M.min(0)) / (M.max(0) - M.min(0) + 1e-12)
    V = Xn / 4
    cc_eq = np.sqrt(((V - V.min(0)) ** 2).sum(1)) / (np.sqrt(((V - V.max(0)) ** 2).sum(1)) +
    np.sqrt(((V - V.min(0)) ** 2).sum(1)))
    top_e = set(sel)
    top_eq = set(pd.Series(cc_eq, index=pd.DataFrame(rows).dropna(subset=["rarity"]).feature).
    nlargest(5).index)
    top_r = set(A.nlargest(5, "rarity").feature)

    hh = h.copy()
    hh["hour"] = hh.index.hour
    comp = {}
    for k, ev in EVENTS.items():
        pre = hh.loc[ev - pd.Timedelta(days=7): ev - pd.Timedelta(hours=1)]
        doy = hh.index.dayofyear
        base = hh[(np.abs(doy - ev.dayofyear) <= 30) & (hh.index.year != ev.year)]
        for v in ["dTdz", "alpha", "H", "LE", "TKE", "T2", "Rn"]:
            comp[(k, v, "pre")] = pre.groupby("hour")[v].mean()
            comp[(k, v, "clim")] = base.groupby("hour")[v].mean()
    pd.DataFrame(comp).to_csv(RES / "q1_diurnal.csv")

    js = d[d.index.month.isin(range(5, 11))]
    thr = {"P": js.P[js.P >= 0.1].quantile(0.95), "Tmax": js.Tmax.quantile(0.95), "Ta_anom":
    None}
    clim_ta = js.Ta.groupby(js.index.dayofyear).mean().rolling(15, center=True, min_periods=1)
    .mean()
    anom = js.Ta - clim_ta.reindex(js.index.dayofyear).values
    sd_ta = anom.std()
    warm_dry = ((anom > sd_ta) & (js.P < 1)).astype(int)

    run = warm_dry.groupby((warm_dry != warm_dry.shift()).cumsum()).transform("sum") *
    warm_dry
    ext = pd.DataFrame({"heavy_rain": (js.P >= thr["P"]), "heat": (js.Tmax >= thr["Tmax"]),
                        "warm_dry_spell": run >= 3})
    ext_year = ext.groupby(ext.index.year).sum()
    ext_year.to_csv(RES / "q1_extremes_by_year.csv")
    pre_ext = {k: ext.loc[v - pd.Timedelta(days=10): v - pd.Timedelta(days=1)].sum().to_dict()
     for k, v in days.items()}

    aug = d.loc["2026-07-20":"2026-08-26", ["P", "Ta", "RH", "VWC0.1", "PDDsrc", "Rsd"]]
    aug.to_csv(RES / "q1_aug2026.csv")
    out = {"entropy_weights": dict(zip(["稀有度", "一致性", "增强度", "|z|"], np.round(w, 4).
    tolist())),
           "top5": A[A.top5][["feature", "name", "direction", "closeness", "p_2025-07-08", "
    p_2026-08-26",
                              "z_2025-07-08", "z_2026-08-26", "trend"]].round(3).to_dict("
    records"),
           "top5_overlap_equal_weight": len(top_e & top_eq), "top5_overlap_rarity_only": len(
    top_e & top_r),
           "thresholds": {"P95_wet": float(thr["P"]), "Tmax95": float(thr["Tmax"]), "
    Ta_anom_sd": float(sd_ta)},
           "extremes_10d_before": pre_ext,
           "aug2026": {"P_aug1_26": float(d.loc["2026-08-01":"2026-08-26"].P.sum()),
                       "P_aug1_26_other_years": {int(y): float(d.loc[f"{y}-08-01":f"{y}-08-26"
    ].P.sum()) for y in range(2021, 2026)},
                       "dry_days_before": int((d.loc["2026-08-20":"2026-08-25"].P < 0.5).sum()
    )}}
    json.dump(out, open(RES / "q1_summary.json", "w", encoding="utf-8"), ensure_ascii=False,
    indent=1, default=float)
    print(json.dumps(out, ensure_ascii=False, indent=1, default=float))
    print(A[["rank", "feature", "name", "direction", "rarity", "consistency", "trend", "absz",
     "closeness",
             "p_2025-07-08", "p_2026-08-26"]].round(3).head(15).to_string())

if __name__ == "__main__":
    main()
```

B.3 q2.py 问题二：降雨检验、双因子临界条件与易发性评估

```
import json
import glob
import struct
import heapq
import numpy as np
import pandas as pd
import rasterio
from rasterio.windows import from_bounds
from scipy import ndimage
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score
from sklearn.model_selection import StratifiedKFold
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from common import DATA, EXT, RES, SEED, EVENTS, STATION, LAPSE, SRC_Z, load_era5

BOX = (85.10, 28.25, 85.55, 28.95)

def precip_scale(d):
    e = load_era5("station").P.resample("D").sum()
    j = pd.concat([d.P, e], axis=1, keys=["obs", "era"]).dropna()
    j = j[j.index.month.isin(range(5, 11))]
    return j.obs.sum() / j.era.sum(), np.corrcoef(j.obs, j.era)[0, 1]

def rain_events(p, gap=12, thr=0.1):
    wet = p >= thr
    ev, start, last = [], None, None
    for t, w in wet.items():
        if w:
            if start is None:
                start = t
            elif (t - last) > pd.Timedelta(hours=gap):
                ev.append((start, last))
                start = t
            last = t
    if start is not None:
        ev.append((start, last))
    rows = []
    for s, e in ev:
        seg = p.loc[s:e]
        D = (e - s) / pd.Timedelta(hours=1) + 1
        rows.append({"start": s, "end": e, "D": D, "R": seg.sum(), "I": seg.sum() / D})
    return pd.DataFrame(rows)

def alarm_stats(X1, X2, c1, c2, days, months=range(5, 11)):
    on = (X1 >= c1) & (X2 >= c2)
    sea = on[on.index.month.isin(months)]
    near = np.zeros(len(sea), bool)
    for dd in days:
        near |= (sea.index >= dd - pd.Timedelta(days=3)) & (sea.index <= dd)
    hits = [bool(on.get(dd, False)) for dd in days]
    fa = int((sea & ~near).sum())
    n = int((~near).sum())
    return hits, fa, fa / n

def read_dbf(p):
    f = open(p, "rb").read()
    n, hl, rl = struct.unpack("<IHH", f[4:12])
    fl = [(f[i:i + 11].split(b"\0")[0].decode("gbk", "replace"), f[i + 16]) for i in range(32,
     hl - 1, 32)]
    out = []
    for r in range(n):
        p0 = hl + r * rl + 1
        row = {}
        for nm, ln in fl:
            row[nm] = f[p0:p0 + ln].decode("gbk", "replace").strip()
            p0 += ln
        out.append(row)
    return pd.DataFrame(out)

def flow_acc(z):
    ny, nx = z.shape
    fill = z.copy()
```

```
    done = np.zeros(z.shape, bool)
    pq = []
    for i in range(ny):
        for j in range(nx):
            if i in (0, ny - 1) or j in (0, nx - 1):
                heapq.heappush(pq, (fill[i, j], i, j))
                done[i, j] = True
    nb = [(-1, -1), (-1, 0), (-1, 1), (0, -1), (0, 1), (1, -1), (1, 0), (1, 1)]
    while pq:
        h, i, j = heapq.heappop(pq)
        for di, dj in nb:
            a, b = i + di, j + dj
            if 0 <= a < ny and 0 <= b < nx and not done[a, b]:
                done[a, b] = True
                if fill[a, b] <= h:
                    fill[a, b] = h + 1e-3
                heapq.heappush(pq, (fill[a, b], a, b))

    best = np.full(z.shape, -1)
    smax = np.zeros(z.shape)
    pad = np.pad(fill, 1, constant_values=np.inf)
    for k, (di, dj) in enumerate(nb):
        s = (fill - pad[1 + di:1 + di + ny, 1 + dj:1 + dj + nx]) / np.hypot(di, dj)
        upd = s > smax
        smax[upd], best[upd] = s[upd], k
    acc = np.ones(z.shape)
    order = np.argsort(-fill, axis=None)
    flat = acc.ravel()
    bi = best.ravel()
    for idx in order:
        k = bi[idx]
        if k < 0:
            continue
        i, j = divmod(idx, nx)
        a, b = i + nb[k][0], j + nb[k][1]
        if 0 <= a < ny and 0 <= b < nx:
            flat[a * nx + b] += flat[idx]
    return fill, acc

def terrain_stack():
    with rasterio.open(EXT / "cop30_N28E085.tif") as r:
        win = from_bounds(*BOX, r.transform)
        z = r.read(1, window=win).astype(float)
        tr = r.window_transform(win)
    z = z[: z.shape[0] // 3 * 3, : z.shape[1] // 3 * 3].reshape(z.shape[0] // 3, 3, z.shape[1]
     // 3, 3).mean((1, 3))
    tr = tr * tr.scale(3, 3)
    res_y = abs(tr.e) * 111320
    res_x = tr.a * 111320 * np.cos(np.deg2rad(28.6))
    gy, gx = np.gradient(z, res_y, res_x)
    slope = np.degrees(np.arctan(np.hypot(gx, gy)))
    relief = ndimage.maximum_filter(z, 11) - ndimage.minimum_filter(z, 11)
    north = np.cos(np.arctan2(-gx, gy))
    fill, acc = flow_acc(z)
    area_km2 = acc * res_x * res_y / 1e6
    river = area_km2 >= 5
    dist_river = ndimage.distance_transform_edt(~river) * (res_x + res_y) / 2
    twi = np.log(area_km2 * 1e6 / (res_x) / np.tan(np.deg2rad(np.clip(slope, 0.5, None))))

    with rasterio.open(DATA / "d5" / "jilong_LUCD_2023.tif") as r:
        rows, cols = np.meshgrid(np.arange(z.shape[0]), np.arange(z.shape[1]), indexing="ij")
        xs, ys = rasterio.transform.xy(tr, rows.ravel(), cols.ravel())
        lc_all = r.read(1)
        ri, ci = rasterio.transform.rowcol(r.transform, xs, ys)
        ri, ci = np.clip(ri, 0, lc_all.shape[0] - 1), np.clip(ci, 0, lc_all.shape[1] - 1)
        lc = lc_all[ri, ci].reshape(z.shape)
    ice = lc == 10
    dist_ice = ndimage.distance_transform_edt(~ice) * (res_x + res_y) / 2
    veg = ndimage.uniform_filter(np.isin(lc, [2, 3, 5]).astype(float), 11)
    bare = ndimage.uniform_filter((lc == 7).astype(float), 11)
    F = {"elev": z, "slope": slope, "relief": relief, "north": north, "log_area": np.log10(
    area_km2),
         "dist_river": dist_river, "twi": twi, "dist_ice": dist_ice, "veg": veg, "bare": bare}
    return F, tr, lc

def main():
```

```
d = pd.read_csv(RES / "q1_daily.csv", index_col=0, parse_dates=True)
days = [v.floor("D") for v in EVENTS.values()]
out = {}

k, r_st = precip_scale(d)
out["precip_scale"], out["precip_corr_station_era5"] = k, r_st
eg, ep = load_era5("glacier"), load_era5("port")
el = load_era5("glacier", "era5land")
rep = {}
for nm, e, col in [("ERA5 源区降水", eg, "P"), ("ERA5 口岸降水", ep, "P"), ("ERA5-Land 源区气
温", el, "T")]:
    x = e[col].resample("D").sum() if col == "P" else e[col].resample("D").mean()
    y = d.P if col == "P" else d.Ta
    j = pd.concat([x, y], axis=1).dropna()
    j = j[j.index.month.isin(range(5, 11))]
    rep[nm] = float(np.corrcoef(j.iloc[:, 0], j.iloc[:, 1])[0, 1])

tsrc = d.Ta - LAPSE * (SRC_Z - STATION[2])
tl = el["T"].resample("D").mean() - LAPSE * (SRC_Z - el.attrs["z"])
j = pd.concat([tsrc, tl], axis=1).dropna()
rep["源区气温（外推 vs ERA5-Land）偏差"] = float((j.iloc[:, 0] - j.iloc[:, 1]).mean())
out["representativeness"] = rep

pg = eg.P * k
ev = rain_events(pg.loc["2021-01-01":"2026-08-26 10:00"])
ev.to_csv(RES / "q2_rain_events.csv", index=False)
lit = {"Caine 1980": (14.82, 0.39), "Guzzetti 2008": (2.20, 0.44), "Dahal 2008 喜马拉雅":
(73.90, 0.79)}
ev["exceed_guzzetti"] = ev.I >= 2.20 * ev.D ** -0.44
ev["exceed_dahal"] = ev.I >= 73.90 * ev.D ** -0.79
pre = {}
for nm, t in EVENTS.items():
    last = ev[ev.end <= t].iloc[-1]
    pre[nm] = {"last_rain_end": str(last.end), "hours_since": float((t - last.end) / pd.
Timedelta(hours=1)),
               "D": float(last.D), "I": float(last.I), "R": float(last.R),
               "exceed_guzzetti": bool(last.exceed_guzzetti), "exceed_dahal": bool(last.
exceed_dahal),
               "P72h_src": float(pg.loc[t - pd.Timedelta(hours=72): t].sum()),
               "P24h_src": float(pg.loc[t - pd.Timedelta(hours=24): t].sum())}
out["rain_before_events"] = pre
out["rain_events_exceed_dahal_per_year"] = float(ev.exceed_dahal.sum() / 5.65)

X1c = {"P30": d.P30, "P15": d.P.rolling(15, min_periods=12).sum(), "P45": d.P.rolling(45,
min_periods=36).sum(),
       "API15": d.API15}
tsrc_d = d.PDDsrc
X2c = {"PDD3": tsrc_d.rolling(3).sum(), "PDD1": tsrc_d, "PDD5": tsrc_d.rolling(5).sum(),
       "VPD3": d.VPD.rolling(3).mean(), "DTR3": d.DTR.rolling(3).mean()}
combos = []
for n1, x1 in X1c.items():
    for n2, x2 in X2c.items():
        x1s, x2s = x1.shift(1), x2.shift(1)
        v1 = [x1s.get(dd) for dd in days]
        v2 = [x2s.get(dd) for dd in days]
        if any(pd.isna(v) for v in v1 + v2):
            continue
        c1, c2 = min(v1) * 0.98, min(v2) * 0.98
        hits, fa, far = alarm_stats(x1s, x2s, c1, c2, days)

        loo = []
        for a in range(2):
            b = 1 - a
            hh, _, _ = alarm_stats(x1s, x2s, v1[a] * 0.9, v2[a] * 0.9, [days[b]])
            loo.append(hh[0])
        combos.append({"X1": n1, "X2": n2, "c1": c1, "c2": c2, "hits": sum(hits), "
false_days": fa,
                       "FAR_daily": far, "loo_hits": sum(loo)})
C = pd.DataFrame(combos).sort_values(["hits", "loo_hits", "FAR_daily"], ascending=[False,
False, True])
C.to_csv(RES / "q2_threshold_combos.csv", index=False)
best = C.iloc[0]
out["critical"] = best.to_dict()
x1s, x2s = X1c[best.X1].shift(1), X2c[best.X2].shift(1)
on = (x1s >= best.c1) & (x2s >= best.c2)
sea = on[on.index.month.isin(range(5, 11))]
```

```
out["alarm_days_by_year"] = {int(y): int(v) for y, v in sea.groupby(sea.index.year).sum().
items()}

for nm, (a, b) in {"only_X1": (best.c1, -np.inf), "only_X2": (-np.inf, best.c2)}.items():
    _, fa, far = alarm_stats(x1s, x2s, a, b, days)
    out[nm] = {"false_days": fa, "FAR_daily": far}
pd.DataFrame({"X1": x1s, "X2": x2s, "alarm": on}).to_csv(RES / "q2_critical_daily.csv")

F, tr, lc = terrain_stack()
hz = read_dbf(glob.glob(str(DATA / "d6" / "*" / "*.dbf"))[0])
lon, lat = hz.x_coord.astype(float).values, hz.y_coord.astype(float).values
rr, cc = rasterio.transform.rowcol(tr, lon, lat)
rr, cc = np.array(rr), np.array(cc)
rng = np.random.default_rng(SEED)
ny, nx = F["elev"].shape
pos = set(zip(rr, cc))
neg = []
while len(neg) < 10 * len(rr):
    i, j = rng.integers(0, ny), rng.integers(0, nx)
    if F["elev"][i, j] < 5500 and min(np.hypot(i - rr, j - cc)) > 11 and (i, j) not in pos
:
        neg.append((i, j))
neg = np.array(neg)
keys = list(F)
Xp = np.column_stack([F[k][rr, cc] for k in keys])
Xn = np.column_stack([F[k][neg[:, 0], neg[:, 1]] for k in keys])
X = np.vstack([Xp, Xn])
y = np.r_[np.ones(len(Xp)), np.zeros(len(Xn))]
rowcol = np.vstack([np.c_[rr, cc], neg])
lat_all = tr.f + tr.e * (rowcol[:, 0] + 0.5)
blocks = np.floor((lat_all - BOX[1]) / 0.14).astype(int)
models = {"逻辑回归": make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000, C
=1.0)),
          "随机森林": RandomForestClassifier(n_estimators=500, min_samples_leaf=3,
random_state=SEED, n_jobs=-1)}
sp = {}
for nm, m in models.items():
    pr = np.zeros(len(y))
    for trn, tst in StratifiedKFold(5, shuffle=True, random_state=SEED).split(X, y):
        pr[tst] = m.fit(X[trn], y[trn]).predict_proba(X[tst])[:, 1]
    pb = np.zeros(len(y))
    for b in np.unique(blocks):
        tst = blocks == b
        if y[tst].sum() == 0 or y[~tst].sum() == 0:
            pb[tst] = np.nan
            continue
        pb[tst] = m.fit(X[~tst], y[~tst]).predict_proba(X[tst])[:, 1]
    okb = ~np.isnan(pb)
    sp[nm] = {"AUC_random_cv": float(roc_auc_score(y, pr)), "AUC_spatial_block": float(
roc_auc_score(y[okb], pb[okb]))}
rf = models["随机森林"].fit(X, y)
sp["rf_importance"] = dict(sorted(zip(keys, rf.feature_importances_.round(4).tolist()),
key=lambda t: -t[1]))
lr = models["逻辑回归"].fit(X, y)
sp["lr_coef_std"] = dict(zip(keys, lr[-1].coef_[0].round(3).tolist()))

Zall = np.column_stack([F[k].ravel() for k in keys])
prob = rf.predict_proba(np.nan_to_num(Zall))[:, 1].reshape(ny, nx)
np.save(RES / "q2_suscept.npy", prob.astype(np.float32))
json.dump({"transform": list(tr)[:6], "shape": [ny, nx]}, open(RES / "q2_suscept_meta.json
", "w"))
pi, pj = rasterio.transform.rowcol(tr, 85.379, 28.2735)
sp["prob_port_area_max_1km"] = float(prob[max(pi - 11, 0):pi + 12, max(pj - 11, 0):pj +
12].max())
sp["share_high"] = float((prob >= 0.5).mean())
sp["hazard_in_high"] = float((prob[rr, cc] >= 0.5).mean())
out["spatial"] = sp
hz2 = hz.assign(lon=lon, lat=lat, prob=prob[rr, cc])
hz2[["Code", "lon", "lat", "类型", "规模", "频率", "prob"]].to_csv(RES / "q2_hazard_points.
csv", index=False)

import netCDF4
from rasterio.warp import transform as wtrans
ds = netCDF4.Dataset(glob.glob(str(DATA / "d4" / "*.nc"))[0])
Xg, Yg = np.meshgrid(ds["x"][:], ds["y"][:])
gm = ds["glacier_mask"][:] > 0
```

```
    glon, glat = wtrans("EPSG:32645", "EPSG:4326", Xg[gm].tolist(), Yg[gm].tolist())
    dh = {}
    for f in sorted(glob.glob(str(DATA / "d3g" / "dhdt_GCS__*.tif"))):
        with rasterio.open(f) as r:
            v = np.array(list(r.sample(zip(glon, glat))))[:, 0]
        v = v[np.isfinite(v) & (np.abs(v) < 100)]
        dh[f.split("__")[1][8:12] + "-" + f.split("__")[1][19:23]] = float(v.mean())
    out["dhdt_src_m_per_yr"] = dh
    json.dump(out, open(RES / "q2_summary.json", "w", encoding="utf-8"), ensure_ascii=False,
    indent=1, default=str)
    print(json.dumps(out, ensure_ascii=False, indent=1, default=str))
    print(C.head(8).round(3).to_string())

if __name__ == "__main__":
    main()
```

B.4 q3.py 问题三：滞后概率预报

```
import json
import numpy as np
import pandas as pd
from common import RES, EXT, EVENTS, STATION, SRC_Z, SEED, hourly

NMC = 2000
LEADS_H = [12, 24, 36, 48, 72, 96, 120]
HORIZON_D = 5

def load_fc(point="glacier"):
    d = pd.read_csv(EXT / f"prevruns_{point}.csv", skiprows=3, parse_dates=["time"]).set_index
    ("time")
    cols = {}
    for c in d.columns:
        lead = int(c.split("previous_day")[1].split(" ")[0]) if "previous_day" in c else 0
        cols[c] = ("T" if c.startswith("temperature") else "P", lead)
    d.columns = pd.MultiIndex.from_tuples([cols[c] for c in d.columns])
    return d

class Forecaster:
    def __init__(self):
        self.h = hourly()
        self.tsrc_obs = (self.h.T2 - 0.0065 * (SRC_Z - STATION[2])).interpolate(limit=6)
        self.p_obs = self.h.P.fillna(0)
        self.fc = load_fc()
        cr = json.load(open(RES / "q2_summary.json", encoding="utf-8"))["critical"]
        self.c1, self.c2 = float(cr["c1"]), float(cr["c2"])
        al = pd.read_csv(RES / "q2_critical_daily.csv", index_col=0, parse_dates=True).alarm
        self.n_alarm = int(al.loc[:max(EVENTS.values()).floor("D")].sum())
        self.qa, self.qb = 2 + 1, self.n_alarm - 2 + 1

        tr = slice("2024-01-01", "2025-05-31")
        j = pd.concat([self.fc[("T", 0)], self.tsrc_obs], axis=1).loc[tr].dropna()
        self.b, self.a = np.polyfit(j.iloc[:, 0], j.iloc[:, 1], 1)
        pj = pd.concat([self.fc[("P", 0)], self.p_obs], axis=1).loc[tr].dropna()
        self.pscale = pj.iloc[:, 1].sum() / pj.iloc[:, 0].sum()

        self.sig = {}
        for L in range(1, 6):
            bias = (self.fc[("T", L)] - self.fc[("T", 0)]).rolling(24 * 30, min_periods=24 *
    10).mean().shift(24 * L)
            tc = self.a + self.b * (self.fc[("T", L)] - bias)
            e = (tc - self.tsrc_obs).loc[tr].resample("D").mean().dropna()
            self.sig[L] = float(e.std())

    def tsrc_series(self, t0, lapse=0.0065):
        obs = (self.h.T2 - lapse * (SRC_Z - STATION[2])).interpolate(limit=6).loc[: t0 - pd.
    Timedelta(hours=1)]
        parts = [obs.loc[t0 - pd.Timedelta(days=20):]]
        pobs = self.p_obs.loc[t0 - pd.Timedelta(days=20): t0 - pd.Timedelta(hours=1)]
        pp = [pobs]
        for k in range(HORIZON_D + 1):
            L = min(k + 1, 5)
            s, e = t0 + pd.Timedelta(days=k), t0 + pd.Timedelta(days=k + 1) - pd.Timedelta(
    hours=1)
            bias = (self.fc[("T", L)] - self.fc[("T", 0)]).loc[t0 - pd.Timedelta(days=30 + L):
     t0 - pd.Timedelta(days=L)].mean()
```

```
            tf = self.a + self.b * (self.fc[("T", L)].loc[s:e] - bias) + (0.0065 - lapse) * (
    SRC_Z - STATION[2])
            parts.append(tf)
            pp.append(self.fc[("P", L)].loc[s:e] * self.pscale)
        return pd.concat(parts), pd.concat(pp)

    def simulate(self, t0, tau_rng, rng, n=NMC, timing="lag"):
        Ts, Ps = self.tsrc_series(t0)
        valid_end = Ts.last_valid_index()
        d0 = t0.floor("D")
        idx = pd.date_range(d0 - pd.Timedelta(days=20), d0 + pd.Timedelta(days=HORIZON_D + 1)
    - pd.Timedelta(hours=1), freq="h")
        T = Ts.reindex(idx).interpolate(limit_direction="both").values
        P = Ps.reindex(idx).fillna(0).values
        nd = len(idx) // 24
        lead = np.clip(((idx - t0) / pd.Timedelta(days=1)).values.astype(float), -1, None)
        lead = np.where(idx >= t0, np.floor(lead).astype(int) + 1, 0)
        sig = np.array([0] + [self.sig[min(L, 5)] for L in range(1, HORIZON_D + 3)])
        eps = rng.normal(0, 1, (n, len(sig))) * sig
        lapse_d = rng.normal(0, 0.0005, n) * (SRC_Z - STATION[2])
        Tm = T[None, :] - lapse_d[:, None] + eps[:, lead]
        pdd = np.clip(Tm, 0, None).reshape(n, nd, 24).mean(2)
        pdd3 = np.full((n, nd), np.nan)
        pdd3[:, 3:] = pdd[:, :-3] + pdd[:, 1:-2] + pdd[:, 2:-1]
        Pd = P.reshape(nd, 24).sum(1)
        p15 = np.full(nd, np.nan)
        for k in range(15, nd):
            p15[k] = Pd[k - 15:k].sum()
        c1 = self.c1 * rng.uniform(0.9, 1.1, n)
        c2 = self.c2 * rng.uniform(0.9, 1.1, n)
        alarm = (p15[None, :] >= c1[:, None]) & (pdd3 >= c2[:, None])
        hz = np.where((idx >= t0) & (idx <= valid_end))[0][: 24 * HORIZON_D]
        tau = np.round(rng.uniform(*tau_rng, n)).astype(int)
        anom = Tm - pd.DataFrame(Tm.T).rolling(24, center=True, min_periods=12).mean().values.
    T
        melt = np.clip(anom[np.arange(n)[:, None], hz[None, :] - tau[:, None]], 0, None) +
    0.05
        if timing == "uniform":
            melt = np.ones_like(melt)

        q = rng.beta(self.qa, self.qb, n) if timing != "noq" else np.ones(n)
        days = hz // 24
        ud = np.unique(days)
        fire = (alarm[:, ud] & (rng.uniform(0, 1, (n, len(ud))) < q[:, None]))
        first = np.where(fire.any(1), ud[np.argmax(fire, 1)], -1)
        w = (days[None, :] == first[:, None]) * melt
        tot = w.sum(1)
        cw = np.cumsum(w, 1)
        u = rng.uniform(0, 1, n) * tot
        k = np.minimum((cw < u[:, None]).sum(1), len(hz) - 1)
        t = idx[hz[k]] + pd.to_timedelta(rng.uniform(0, 60, n), unit="min")
        t = t.to_numpy().astype("datetime64[ns]")
        return pd.Series(pd.to_datetime(np.where(tot > 0, t, np.datetime64("NaT"))))

def tau_likelihood(F, ev, taus):
    T = F.tsrc_obs
    anom = (T - T.rolling(24, center=True, min_periods=12).mean()).clip(lower=0) + 0.05
    hours = pd.date_range(ev.floor("D"), ev.floor("D") + pd.Timedelta(hours=23), freq="h")
    L = []
    for tau in taus:
        w = anom.reindex(hours - pd.Timedelta(hours=float(tau)), method="nearest").values
        w = np.nan_to_num(w) / np.nansum(w)
        L.append(w[ev.hour])
    return np.array(L)

def main():
    rng = np.random.default_rng(SEED)
    F = Forecaster()
    taus = np.arange(0, 24, 1.0)
    lik = {k: tau_likelihood(F, v, taus) for k, v in EVENTS.items()}
    Lj = lik["2025-07-08"] * lik["2026-08-26"]
    tau_hat = float(taus[np.argmax(Lj)])
    ok = taus[Lj >= 0.15 * Lj.max()]
    tau_rng = (float(ok.min()), float(ok.max()))
    loo = {}
    for k in EVENTS:
```

```
    other = [x for x in EVENTS if x != k][0]
    okk = taus[lik[other] >= 0.15 * lik[other].max()]
    loo[k] = (float(okk.min()), float(okk.max()))
pd.DataFrame({"tau": taus, **{f"L_{k}": v for k, v in lik.items()}, "L_joint": Lj}).to_csv
(RES / "q3_tau_lik.csv", index=False)

rows, samples = [], {}
for k, ev in EVENTS.items():
    for lead in LEADS_H:
        t0 = (ev - pd.Timedelta(hours=lead)).floor("h")
        for mode, tr in [("联合标定", tau_rng), ("留一标定", loo[k])]:
            S = F.simulate(t0, tr, rng, n=NMC)
            trig = S.dropna()
            p = len(trig) / len(S)
            p48 = ((trig >= t0) & (trig < t0 + pd.Timedelta(hours=48))).sum() / len(S)
            if len(trig) > 20:
                lo, md, hi = trig.quantile([0.025, 0.5, 0.975])
                cover = lo <= ev <= hi
                width = (hi - lo) / pd.Timedelta(hours=1)
                err = (md - ev) / pd.Timedelta(hours=1)
            else:
                lo = md = hi = pd.NaT
                cover, width, err = False, np.nan, np.nan
            rows.append({"event": k, "lead_h": lead, "mode": mode, "issue": t0, "p_trigger
": p, "p48": p48, "lo": lo, "median": md,
                         "hi": hi, "width_h": width, "median_err_h": err, "covered": cover
})
            if mode == "联合标定":
                samples[(k, lead)] = trig
R = pd.DataFrame(rows)
R.to_csv(RES / "q3_forecast_eval.csv", index=False)
pd.DataFrame({f"{k}|{l}": v.reset_index(drop=True) for (k, l), v in samples.items()}).
to_csv(RES / "q3_samples.csv", index=False)

issues = [t for t in pd.date_range("2024-05-01 08:00", "2026-08-26 08:00", freq="D") if t.
month in range(5, 11)]
roll = []
for t0 in issues:
    S = F.simulate(t0, tau_rng, rng, n=200)
    trig = S.dropna()
    p48 = ((trig >= t0) & (trig < t0 + pd.Timedelta(hours=48))).sum() / len(S)
    near = any(abs((ev - t0) / pd.Timedelta(hours=1)) <= 72 for ev in EVENTS.values())
    hit_ev = any(0 <= (ev - t0) / pd.Timedelta(hours=1) <= 48 for ev in EVENTS.values())
    roll.append({"issue": t0, "p48": p48, "event_in_48h": hit_ev, "near_event": near})
Rl = pd.DataFrame(roll)
Rl.to_csv(RES / "q3_rolling.csv", index=False)

cand = []
for ps in [0.02, 0.05, 0.08, 0.10, 0.12, 0.15, 0.20, 0.25, 0.30]:
    warn = Rl.p48 >= ps
    hit = (warn & Rl.event_in_48h).sum() / max(Rl.event_in_48h.sum(), 1)
    far = (warn & ~Rl.near_event).sum() / (~Rl.near_event).sum()
    cand.append({"p_star": ps, "hit_rate": hit, "false_rate": far, "warn_days": int(warn.
sum()),
                 "false_days": int((warn & ~Rl.near_event).sum()), "youden": hit - far})
Cd = pd.DataFrame(cand)
Cd.to_csv(RES / "q3_pstar.csv", index=False)
pstar = float(Cd.sort_values(["youden", "p_star"], ascending=[False, False]).iloc[0].
p_star)
lead_time = {}
for k in EVENTS:
    r = R[(R.event == k) & (R["mode"] == "联合标定") & (R.p48 >= pstar) & R.covered]
    lead_time[k] = float(r.lead_h.max()) if len(r) else 0.0
sel = Cd[Cd.p_star == pstar].iloc[0]
out = {"n_alarm_days": F.n_alarm, "q_beta": [F.qa, F.qb], "tau_hat_h": tau_hat, "
tau_range_h": tau_rng,
       "tau_loo": loo, "sigma_fc": F.sig, "calib_a_b": [F.a, F.b], "pscale": F.pscale,
       "p_star": pstar, "lead_time_h": lead_time,
       "coverage_joint": float(R[R["mode"] == "联合标定"].covered.mean()),
       "coverage_loo": float(R[R["mode"] == "留一标定"].covered.mean()),
       "coverage_joint_le96h": float(R[(R["mode"] == "联合标定") & (R.lead_h <= 96)].covered
.mean()),
       "width_h_by_lead": R[R["mode"] == "联合标定"].groupby("lead_h").width_h.mean().round
(1).to_dict(),
       "rolling": {"n_issues": len(Rl), **{k: (float(v) if k != "p_star" else v) for k, v
in sel.to_dict().items()}}}
```

```
    q4 = json.load(open(RES / "q4_summary.json", encoding="utf-8")) if (RES / "q4_summary.json
    ").exists() else {}
    if "mc" in q4:
        out["spatial"] = {"port_chainage_km": q4["port_chainage_km"], "p_reach_port": q4["mc"
    ]["p_reach_port"],
                          "runout_q_km": q4["mc"]["runout_q"], "lateral_q_m": q4["mc"]["
    lateral_q"],
                          "t_port_q_min": q4["mc"]["t_port_q"]}
    json.dump(out, open(RES / "q3_summary.json", "w", encoding="utf-8"), ensure_ascii=False,
    indent=1, default=str)
    print(json.dumps(out, ensure_ascii=False, indent=1, default=str))
    print(R.round(3).to_string())

if __name__ == "__main__":
    main()
```

B.5 sim.py 问题三、四共用：粒子 Voellmy 仿真内核

```
import glob
import numpy as np
import netCDF4
import rasterio
from rasterio.warp import transform
from scipy import ndimage
from scipy.spatial import cKDTree
from common import DATA, RES, PORT

G = 9.81
DEFAULT = dict(V0=3.0e6, N=800, dt=1.0, tmax=3600,
               mu_ice=0.10, xi_ice=600.0,
               mu_df=0.06, xi_df=400.0,
               mu_fl=0.005, xi_fl=800.0,
               bulk=2.0,
               z_term=3700.0,
               slope_fl=0.06,
               area_fl=150.0)

class Terrain:
    def __init__(self, step=2):
        with rasterio.open(DATA / "d7" / "DEM.tif") as r:
            z = r.read(1).astype(float)
            tr = r.transform
        z[z < -1000] = np.nan
        z = np.where(np.isnan(z), np.nanmax(z), z)
        z = z[: z.shape[0] // step * step, : z.shape[1] // step * step]
        z = z.reshape(z.shape[0] // step, step, z.shape[1] // step, step).mean((1, 3))
        self.tr = tr * tr.scale(step, step)
        self.lat0 = self.tr.f
        self.lon0 = self.tr.c
        self.dy = abs(self.tr.e) * 110574.0
        self.dx = self.tr.a * 111320.0 * np.cos(np.deg2rad(28.4))
        self.z = ndimage.gaussian_filter(z, 1.0)
        self.zraw = z
        self.ny, self.nx = z.shape
        gy, gx = np.gradient(self.z, self.dy, self.dx)
        self.gx, self.gy = gx, -gy
        self._flowacc()

        gy, gx = np.gradient(self.fill, self.dy, self.dx)
        self.gx, self.gy = gx, -gy
        self.slope = np.hypot(self.gx, self.gy)

    def _flowacc(self):
        import heapq
        z = self.z.copy()
        ny, nx = z.shape
        done = np.zeros(z.shape, bool)
        pq = [(z[i, j], i, j) for i in range(ny) for j in range(nx) if i in (0, ny - 1) or j
    in (0, nx - 1)]
        heapq.heapify(pq)
        for _, i, j in pq:
            done[i, j] = True
        nb = [(-1, -1), (-1, 0), (-1, 1), (0, -1), (0, 1), (1, -1), (1, 0), (1, 1)]
        while pq:
            h, i, j = heapq.heappop(pq)
```

```
            for di, dj in nb:
                a, b = i + di, j + dj
                if 0 <= a < ny and 0 <= b < nx and not done[a, b]:
                    done[a, b] = True
                    if z[a, b] <= h:
                        z[a, b] = h + 1e-3
                    heapq.heappush(pq, (z[a, b], a, b))
        self.fill = z
        pad = np.pad(z, 1, constant_values=np.inf)
        best = np.full(z.shape, -1)
        smax = np.zeros(z.shape)
        for k, (di, dj) in enumerate(nb):
            s = (z - pad[1 + di:1 + di + ny, 1 + dj:1 + dj + nx]) / np.hypot(di * self.dy, dj
    * self.dx)
            u = s > smax
            smax[u], best[u] = s[u], k
        acc = np.ones(z.shape)
        flat, bi = acc.ravel(), best.ravel()
        for idx in np.argsort(-z, axis=None):
            k = bi[idx]
            if k >= 0:
                i, j = divmod(idx, nx)
                a, b = i + nb[k][0], j + nb[k][1]
                if 0 <= a < ny and 0 <= b < nx:
                    flat[a * nx + b] += flat[idx]
        self.area = acc * self.dx * self.dy / 1e6
        self.best, self.nb = best, nb

    def ll2xy(self, lon, lat):
        x = (np.asarray(lon) - self.lon0) / self.tr.a * self.dx
        y = (np.asarray(lat) - self.lat0) / abs(self.tr.e) * self.dy
        return x, y

    def xy2ij(self, x, y):
        j = np.floor(x / self.dx).astype(int)
        i = np.floor(-y / self.dy).astype(int)
        return i, j

    def path(self, i, j, n=5000):
        P = [(i, j)]
        for _ in range(n):
            k = self.best[i, j]
            if k < 0:
                break
            a, b = i + self.nb[k][0], j + self.nb[k][1]
            if not (0 <= a < self.ny and 0 <= b < self.nx):
                break
            i, j = a, b
            P.append((i, j))
        P = np.array(P)
        seg = np.hypot(np.diff(P[:, 0]) * self.dy, np.diff(P[:, 1]) * self.dx)
        return P, np.r_[0, np.cumsum(seg)]

def port_xy(T, r=600.0):
    x0, y0 = T.ll2xy(PORT[1], PORT[0])
    i0, j0 = T.xy2ij(x0, y0)
    k = int(r / T.dx) + 1
    sub = T.area[i0 - k:i0 + k + 1, j0 - k:j0 + k + 1]
    a, b = np.unravel_index(np.argmax(sub), sub.shape)
    i, j = i0 - k + a, j0 - k + b
    return (j + 0.5) * T.dx, -(i + 0.5) * T.dy

def release_cells(T):
    ds = netCDF4.Dataset(glob.glob(str(DATA / "d4" / "*.nc"))[0])
    X, Y = np.meshgrid(ds["x"][:], ds["y"][:])
    m = (ds["glacier_mask"][:] > 0) & (np.ma.filled(ds["slope"][:], 0) > 30) & (np.ma.filled(
    ds["dem"][:], 0) > 5000)
    lon, lat = transform("EPSG:32645", "EPSG:4326", X[m].tolist(), Y[m].tolist())
    thick = np.ma.filled(ds["thickness"][:], 0)[m]
    gm = ds["glacier_mask"][:] > 0
    glon, glat = transform("EPSG:32645", "EPSG:4326", X[gm].tolist(), Y[gm].tolist())
    return np.array(lon), np.array(lat), thick, (np.array(glon), np.array(glat))

def run(T, p=None, seed=0, frames=False, src=None):
    p = {**DEFAULT, **(p or {})}
    rng = np.random.default_rng(seed)
```

```
lon, lat, thick, _ = src if src is not None else release_cells(T)
w = thick / thick.sum()
k = rng.choice(len(lon), p["N"], p=w)
x, y = T.ll2xy(lon[k] + rng.normal(0, 0.0001, p["N"]), lat[k] + rng.normal(0, 0.0001, p["N
"]))
vx, vy = np.zeros(p["N"]), np.zeros(p["N"])
vol = np.full(p["N"], p["V0"] / p["N"])
phase = np.zeros(p["N"], int)
alive = np.ones(p["N"], bool)
moving = np.ones(p["N"], bool)
dist = np.zeros(p["N"])
grow = (p["bulk"] - 1.0) / 8000.0
hmax = np.zeros((T.ny, T.nx))
vmax = np.zeros((T.ny, T.nx))
tarr = np.full((T.ny, T.nx), np.nan)
px, py = port_xy(T)
t_port, snaps, front = np.nan, [], []
cell_area = T.dx * T.dy
mu = np.array([p["mu_ice"], p["mu_df"], p["mu_fl"]])
xi = np.array([p["xi_ice"], p["xi_df"], p["xi_fl"]])
nsteps = int(p["tmax"] / p["dt"])
for n in range(nsteps):
    t = (n + 1) * p["dt"]
    i, j = T.xy2ij(x, y)
    inside = (i >= 0) & (i < T.ny) & (j >= 0) & (j < T.nx)
    alive &= inside
    if not alive.any():
        break
    ii, jj = np.clip(i, 0, T.ny - 1), np.clip(j, 0, T.nx - 1)

    vgrid = np.bincount((ii * T.nx + jj)[alive], weights=vol[alive], minlength=T.ny * T.nx
).reshape(T.ny, T.nx)
    h = ndimage.uniform_filter(vgrid, 3) / cell_area
    hp = np.clip(ndimage.maximum_filter(vgrid, 3)[ii, jj] / cell_area, 1.0, 30.0)
    gx, gy = T.gx[ii, jj], T.gy[ii, jj]
    s = np.hypot(gx, gy)
    cos = 1 / np.sqrt(1 + s ** 2)
    sin = s * cos
    ex, ey = -gx / (s + 1e-9), -gy / (s + 1e-9)

    zc = T.z[ii, jj]
    phase = np.where((phase == 0) & (zc < p["z_term"]), 1, phase)
    phase = np.where((phase == 1) & (T.area[ii, jj] > p["area_fl"]) & (s < p["slope_fl"]),
 2, phase)
    m = alive & moving
    vx[m] += G * sin[m] * ex[m] * p["dt"]
    vy[m] += G * sin[m] * ey[m] * p["dt"]
    sp = np.hypot(vx, vy)
    fr = (mu[phase] * G * cos + G * sp ** 2 / (xi[phase] * hp)) * p["dt"]
    new = np.maximum(sp - fr, 0)
    scale = np.where(sp > 0, new / (sp + 1e-12), 0)
    vx *= scale
    vy *= scale
    sp = new
    stop = m & (sp < 0.05) & (np.tan(np.arcsin(np.clip(sin, 0, 1))) < mu[phase])
    moving &= ~stop
    step = sp * p["dt"]
    x[m] += vx[m] * p["dt"]
    y[m] += vy[m] * p["dt"]
    dist[m] += step[m]
    vol[m & (phase == 1)] *= (1 + grow * step[m & (phase == 1)])
    vol = np.minimum(vol, p["V0"] / p["N"] * p["bulk"])
    hmax = np.maximum(hmax, h)
    idx = (ii * T.nx + jj)[m]
    vflat = vmax.ravel()
    np.maximum.at(vflat, idx, sp[m])
    tflat = tarr.ravel()
    new_hit = np.isnan(tflat[idx])
    tflat[idx[new_hit]] = t
    if np.isnan(t_port) and (np.hypot(x[alive] - px, y[alive] - py) < 300).any():
        t_port = t
    if frames and n % int(60 / p["dt"]) == 0:
        snaps.append((t, x[alive].copy(), y[alive].copy(), phase[alive].copy(), sp[alive].
copy()))
    front.append((t, dist[alive].max() if alive.any() else np.nan, sp[alive & moving].mean
() if (alive & moving).any() else 0))
```

```
        if not (alive & moving).any():
            break
    i, j = T.xy2ij(x, y)
    ok = alive & (i >= 0) & (i < T.ny) & (j >= 0) & (j < T.nx)
    dep = np.bincount((i * T.nx + j)[ok & ~moving], weights=vol[ok & ~moving], minlength=T.ny
    * T.nx).reshape(T.ny, T.nx)
    dep = ndimage.uniform_filter(dep, 3) / cell_area
    return {"hmax": hmax, "vmax": vmax, "tarr": tarr, "dep": dep, "t_port": t_port, "front":
    np.array(front),
            "snaps": snaps, "exited": float((~alive).mean()), "runout_m": float(dist.max()),
            "vol_final": float(vol.sum()), "x": x, "y": y, "phase": phase, "params": p}
```

B.6 q4.py 问题四：基准情景与蒙特卡洛

```
import json
import pickle
import sys
import numpy as np
import pandas as pd
from joblib import Parallel, delayed
from scipy.spatial import cKDTree
from common import RES, SEED
from sim import Terrain, run, release_cells, port_xy

NMC = 60

def chain_tools(T):
    x, y = T.ll2xy(85.5104, 28.2955)
    i, j = T.xy2ij(x, y)
    P, s = T.path(i, j)
    xy = np.c_[(P[:, 1] + 0.5) * T.dx, -(P[:, 0] + 0.5) * T.dy]
    return P, s, cKDTree(xy)

def mc_one(k, T, src, tree, s, px, py):
    rng = np.random.default_rng(SEED + k)
    p = {"V0": float(10 ** rng.uniform(6, np.log10(2e7))), "mu_df": rng.uniform(0.04, 0.10),
         "xi_df": rng.uniform(200, 600), "mu_fl": rng.uniform(0.002, 0.01), "xi_fl": rng.
    uniform(500, 1200),
         "bulk": rng.uniform(1.5, 3.0), "N": 300, "tmax": 2400}
    R = run(T, p, seed=k, src=src)
    dd, kk = tree.query(np.c_[R["x"], R["y"]])
    ch = s[kk] / 1000
    pport = s[tree.query([px, py])[1]] / 1000

    ii, jj = np.where(R["hmax"] > 0.5)
    lat_d, _ = tree.query(np.c_[(jj + 0.5) * T.dx, -(ii + 0.5) * T.dy])
    return {**{kk2: v for kk2, v in p.items() if kk2 not in ("N", "tmax")},
            "reach_port": bool(np.isfinite(R["t_port"])), "t_port_min": R["t_port"] / 60,
            "runout_km": float(np.percentile(ch, 95)) if R["exited"] < 0.05 else float(s[-1] /
     1000),
            "dep_median_km": float(np.median(ch)), "exited": R["exited"], "port_chainage_km":
    pport,
            "lateral_p95_m": float(np.percentile(lat_d, 95)), "vmax": float(R["vmax"].max()),
            "dep_max_m": float(R["dep"].max())}

def main():
    T = Terrain(2)
    src = release_cells(T)
    P, s, tree = chain_tools(T)
    px, py = port_xy(T)
    cell = 900.0
    V_steep = float(src[2].sum() * cell)
    base = {"V0": 0.2 * V_steep, "N": 800, "tmax": 2400}
    R = run(T, base, seed=1, frames=True, src=src)
    dd, kk = tree.query(np.c_[R["x"], R["y"]])
    prof = pd.DataFrame({"chain_km": s / 1000, "z": T.zraw[P[:, 0], P[:, 1]], "z_fill": T.fill
    [P[:, 0], P[:, 1]],
                         "hmax": R["hmax"][P[:, 0], P[:, 1]], "vmax": R["vmax"][P[:, 0], P[:,
    1]],
                         "tarr_min": R["tarr"][P[:, 0], P[:, 1]] / 60, "dep": R["dep"][P[:,
    0], P[:, 1]],
                         "area_km2": T.area[P[:, 0], P[:, 1]]})
    prof.to_csv(RES / "q4_profile.csv", index=False)
    pport = s[tree.query([px, py])[1]] / 1000
    dams = prof[(prof.dep > 5) & (prof.chain_km > 3)]
```

```
    out = {"V_steep_m3": V_steep, "V0_base_m3": base["V0"], "params": {k: v for k, v in R["
    params"].items()},
           "t_port_min": R["t_port"] / 60, "port_chainage_km": pport, "exited": R["exited"],
           "vol_final_m3": R["vol_final"], "vmax": float(R["vmax"].max()), "hmax_port": float(
    prof.loc[(prof.chain_km - pport).abs().idxmin(), "hmax"]),
           "dep_median_km": float(np.median(s[kk] / 1000)),
           "dam_reaches_km": [round(float(dams.chain_km.min()), 2), round(float(dams.chain_km.
    max()), 2)] if len(dams) else None,
           "dep_max_m": float(R["dep"].max())}
    with open(RES / "q4_base.pkl", "wb") as f:
        pickle.dump({k: R[k] for k in ["hmax", "vmax", "tarr", "dep", "front", "snaps"]} |
                    {"P": P, "s": s, "port_xy": (px, py), "tr": list(T.tr)[:6], "dx": T.dx, "
    dy": T.dy,
                     "z": T.zraw, "src_ll": (src[0], src[1]), "glacier_ll": src[3]}, f)
    print(json.dumps(out, ensure_ascii=False, indent=1, default=float))
    if "--nomc" in sys.argv:
        json.dump(out, open(RES / "q4_summary.json", "w", encoding="utf-8"), ensure_ascii=
    False, indent=1, default=float)
        return
    rows = Parallel(n_jobs=2)(delayed(mc_one)(k, T, src, tree, s, px, py) for k in range(NMC))
    M = pd.DataFrame(rows)
    M.to_csv(RES / "q4_mc.csv", index=False)
    from sklearn.linear_model import LogisticRegression
    lr = LogisticRegression().fit(np.log10(M[["V0"]].values), M.reach_port.values) if M.
    reach_port.nunique() > 1 else None
    out["mc"] = {"n": NMC, "p_reach_port": float(M.reach_port.mean()),
                 "t_port_q": M.t_port_min.dropna().quantile([0.025, 0.5, 0.975]).round(2).
    tolist(),
                 "runout_q": M.runout_km.quantile([0.025, 0.5, 0.975]).round(2).tolist(),
                 "lateral_q": M.lateral_p95_m.quantile([0.025, 0.5, 0.975]).round(0).tolist(),
                 "V50_reach": float(10 ** (-lr.intercept_[0] / lr.coef_[0, 0])) if lr is not
    None else None,
                 "corr_runout": M[["V0", "mu_df", "xi_df", "mu_fl", "xi_fl", "bulk"]].corrwith
    (M.runout_km, method="spearman").round(3).to_dict()}
    json.dump(out, open(RES / "q4_summary.json", "w", encoding="utf-8"), ensure_ascii=False,
    indent=1, default=float)
    print(json.dumps(out["mc"], ensure_ascii=False, indent=1, default=float))

if __name__ == "__main__":
    main()
```

B.7 compare.py 问题一至三：多方法对比与灵敏度

```
import json
import time
import numpy as np
import pandas as pd
from scipy.stats import kendalltau, spearmanr
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score
from sklearn.model_selection import StratifiedKFold
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from common import RES, SEED, EVENTS
from q1 import entropy_topsis

IND = ["rarity", "consistency", "trend", "absz"]

def timeit(f, n=50):
    t = time.perf_counter()
    for _ in range(n):
        r = f()
    return r, (time.perf_counter() - t) / n * 1000

def topsis(M, w):
    X = (M - M.min(0)) / (M.max(0) - M.min(0) + 1e-12)
    V = X * w
    dp = np.sqrt(((V - V.max(0)) ** 2).sum(1))
    dn = np.sqrt(((V - V.min(0)) ** 2).sum(1))
    return dn / (dp + dn)

def critic_w(M):
    X = (M - M.min(0)) / (M.max(0) - M.min(0) + 1e-12)
```

```
    R = np.corrcoef(X.T)
    C = X.std(0) * (1 - R).sum(0)
    return C / C.sum()

def gra(M, rho=0.5):
    X = (M - M.min(0)) / (M.max(0) - M.min(0) + 1e-12)
    D = np.abs(1 - X)
    xi = (D.min() + rho * D.max()) / (D + rho * D.max())
    return xi.mean(1)

def q1_compare():
    A = pd.read_csv(RES / "q1_association.csv")
    M = A[IND].values
    final = A.sort_values("closeness", ascending=False)
    top_final = set(final[final.top5].feature)
    methods = {
        "熵权 TOPSIS": lambda: entropy_topsis(M)[1],
        "等权 TOPSIS": lambda: topsis(M, np.full(4, 0.25)),
        "CRITIC-TOPSIS": lambda: topsis(M, critic_w(M)),
        "灰色关联分析": lambda: gra(M),
        "单指标稀有度": lambda: M[:, 0],
    }
    rows = []
    for nm, f in methods.items():
        sc, ms = timeit(f, 200)
        B = A.assign(score=sc).sort_values("score", ascending=False)
        top = list(B.drop_duplicates("group").feature[:5])
        tau = kendalltau(A.closeness, sc)[0]
        rows.append({"method": nm, "top5": "、".join(B[B.feature.isin(top)].name), "overlap":
    len(set(top) & top_final),
                     "kendall": tau, "ms": ms})
    C = pd.DataFrame(rows)
    C.to_csv(RES / "cmp_q1.csv", index=False)

    rk = {}
    for k in EVENTS:
        Mk = np.c_[np.abs(2 * A[f"p_{k}"] - 1), np.abs(A[f"z_{k}"])]
        rk[k] = entropy_topsis(Mk)[1]
    rho = spearmanr(rk["2025-07-08"], rk["2026-08-26"])[0]
    t5 = {k: set(A.assign(s=v).sort_values("s", ascending=False).drop_duplicates("group").
    feature[:5]) for k, v in rk.items()}
    return C, {"spearman_events": rho, "top5_overlap_events": len(t5["2025-07-08"] & t5["
    2026-08-26"]),
               "top5_2025": sorted(t5["2025-07-08"]), "top5_2026": sorted(t5["2026-08-26"])}

def q2_compare():
    d = pd.read_csv(RES / "q2_critical_daily.csv", index_col=0, parse_dates=True)
    S = json.load(open(RES / "q2_summary.json", encoding="utf-8"))
    c1, c2 = S["critical"]["c1"], S["critical"]["c2"]
    days = [v.floor("D") for v in EVENTS.values()]
    sea = d[d.index.month.isin(range(5, 11))].dropna()
    near = np.zeros(len(sea), bool)
    for dd in days:
        near |= (sea.index >= dd - pd.Timedelta(days=3)) & (sea.index <= dd)
    y = sea.index.isin(days).astype(int)
    X = sea[["X1", "X2"]].values

    def evaluate(flag):
        hits = int(sum(flag[sea.index.get_loc(dd)] for dd in days))
        fa = int((flag & ~near).sum())
        return hits, fa, fa / (~near).sum()

    def m_two():
        return (sea.X1 >= c1).values & (sea.X2 >= c2).values

    def m_x1():
        return (sea.X1 >= c1).values

    def m_x2():
        return (sea.X2 >= c2).values

    def m_prod():
        r = (sea.X1 / c1) * (sea.X2 / c2)
        thr = min(r.loc[dd] for dd in days)
        return (r >= thr).values
```

```
    def m_lr():
        lr = make_pipeline(StandardScaler(), LogisticRegression(class_weight="balanced")).fit(
    X, y)
        p = lr.predict_proba(X)[:, 1]
        thr = min(p[sea.index.get_loc(dd)] for dd in days)
        return p >= thr

    rows = []
    for nm, f in {"双因子阈值": m_two, "单因子 P15": m_x1, "单因子 PDD3": m_x2, "乘积指数": m_prod,
     "逻辑回归": m_lr}.items():
        flag, ms = timeit(f, 50)
        h, fa, far = evaluate(flag)
        rows.append({"method": nm, "hits": h, "false_days": fa, "FAR": far, "ms": ms})
    T2 = pd.DataFrame(rows)
    T2.to_csv(RES / "cmp_q2_temporal.csv", index=False)

    sens = []
    for k in np.round(np.arange(0.8, 1.201, 0.05), 2):
        for which in ("c1", "c2"):
            a, b = (c1 * k, c2) if which == "c1" else (c1, c2 * k)
            flag = (sea.X1 >= a).values & (sea.X2 >= b).values
            h, fa, far = evaluate(flag)
            sens.append({"which": which, "k": k, "hits": h, "false_days": fa})
    pd.DataFrame(sens).to_csv(RES / "sens_q2.csv", index=False)
    return T2

def q2_spatial_compare():
    from q2 import terrain_stack, read_dbf, BOX
    import glob
    import rasterio
    from common import DATA
    F, tr, lc = terrain_stack()
    hz = read_dbf(glob.glob(str(DATA / "d6" / "*" / "*.dbf"))[0])
    lon, lat = hz.x_coord.astype(float).values, hz.y_coord.astype(float).values
    rr, cc = map(np.array, rasterio.transform.rowcol(tr, lon, lat))
    rng = np.random.default_rng(SEED)
    ny, nx = F["elev"].shape
    pos = set(zip(rr, cc))
    neg = []
    while len(neg) < 10 * len(rr):
        i, j = rng.integers(0, ny), rng.integers(0, nx)
        if F["elev"][i, j] < 5500 and min(np.hypot(i - rr, j - cc)) > 11 and (i, j) not in pos
    :
            neg.append((i, j))
    neg = np.array(neg)
    keys = list(F)
    X = np.vstack([np.column_stack([F[k][rr, cc] for k in keys]), np.column_stack([F[k][neg[:,
     0], neg[:, 1]] for k in keys])])
    y = np.r_[np.ones(len(rr)), np.zeros(len(neg))]
    lat_all = tr.f + tr.e * (np.r_[rr, neg[:, 0]] + 0.5)
    blocks = np.floor((lat_all - BOX[1]) / 0.14).astype(int)
    models = {"逻辑回归": make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000)),
              "支持向量机": make_pipeline(StandardScaler(), SVC(probability=True, random_state=
    SEED)),
              "随机森林": RandomForestClassifier(n_estimators=500, min_samples_leaf=3,
    random_state=SEED, n_jobs=2),
              "梯度提升树": GradientBoostingClassifier(random_state=SEED)}
    rows = []
    for nm, m in models.items():
        pr = np.zeros(len(y))
        t = time.perf_counter()
        for trn, tst in StratifiedKFold(5, shuffle=True, random_state=SEED).split(X, y):
            pr[tst] = m.fit(X[trn], y[trn]).predict_proba(X[tst])[:, 1]
        ms = (time.perf_counter() - t) / 5 * 1000
        pb = np.full(len(y), np.nan)
        for b in np.unique(blocks):
            tst = blocks == b
            if y[tst].sum() and y[~tst].sum():
                pb[tst] = m.fit(X[~tst], y[~tst]).predict_proba(X[tst])[:, 1]
        ok = ~np.isnan(pb)

        thr = np.quantile(pr, 0.9)
        rows.append({"method": nm, "AUC_cv": roc_auc_score(y, pr), "AUC_block": roc_auc_score(
    y[ok], pb[ok]),
                     "hit_top10": float((pr[y == 1] >= thr).mean()), "fit_ms": ms})
    T = pd.DataFrame(rows)
```

```
    T.to_csv(RES / "cmp_q2_spatial.csv", index=False)
    return T

def q3_compare():
    from q3 import Forecaster
    F = Forecaster()
    tau = tuple(json.load(open(RES / "q3_summary.json", encoding="utf-8"))["tau_range_h"])
    rng = np.random.default_rng(SEED)
    rows = []
    cases = [(k, ev, L) for k, ev in EVENTS.items() for L in [12, 24, 36, 48, 72, 96]]

    def evaluate(timing, tr, n=1000):
        cov, wid, err = [], [], []
        t = time.perf_counter()
        for k, ev, L in cases:
            S = F.simulate((ev - pd.Timedelta(hours=L)).floor("h"), tr, rng, n=n, timing=
    timing).dropna()
            if len(S) < 20:
                cov.append(False); continue
            lo, md, hi = S.quantile([0.025, 0.5, 0.975])
            cov.append(lo <= ev <= hi)
            wid.append((hi - lo) / pd.Timedelta(hours=1))
            err.append(abs((md - ev) / pd.Timedelta(hours=1)))
        ms = (time.perf_counter() - t) / len(cases) * 1000
        return np.mean(cov), np.mean(wid), np.mean(err), np.mean(err[:2] + err[6:8]) if len(
    err) >= 8 else np.nan, ms

    for nm, timing, tr in [("滞后融化距平（本文）", "lag", tau), ("无滞后融化距平", "lag", (0, 0)),
                           ("日内均匀", "uniform", tau), ("首个警戒日必触发", "noq", tau)]:
        c, w, e, e12, ms = evaluate(timing, tr)
        rows.append({"method": nm, "coverage": c, "width_h": w, "abs_err_h": e, "abs_err_le24h
    ": e12, "ms": ms})
    T = pd.DataFrame(rows)
    T.to_csv(RES / "cmp_q3.csv", index=False)
    sens = []
    for t in [3, 6, 9, 12, 15, 18, 21]:
        c, w, e, e12, _ = evaluate("lag", (t, t), n=800)
        sens.append({"tau": t, "coverage": c, "width_h": w, "abs_err_h": e, "abs_err_le24h":
    e12})
    pd.DataFrame(sens).to_csv(RES / "sens_q3.csv", index=False)
    return T

if __name__ == "__main__" and len(__import__("sys").argv) > 1 and __import__("sys").argv[1] ==
     "q3":
    print(q3_compare().round(3).to_string())
elif __name__ == "__main__":
    C1, ev = q1_compare()
    print(C1.round(3).to_string()); print(ev)
    print(q2_compare().round(4).to_string())
    print(q3_compare().round(3).to_string())
    print(q2_spatial_compare().round(3).to_string())
    json.dump(ev, open(RES / "cmp_q1_events.json", "w", encoding="utf-8"), ensure_ascii=False,
     indent=1, default=float)
```

B.8 compare_q4.py 问题四：数值收敛、能量线复核与参数网格

```
import json
import time
import numpy as np
import pandas as pd
from joblib import Parallel, delayed
from scipy.spatial import cKDTree
from common import RES
from sim import Terrain, run, release_cells, port_xy
from q4 import chain_tools

def one(T, src, tree, s, p, seed=1):
    t0 = time.time()
    R = run(T, p, seed=seed, src=src)
    dt = time.time() - t0
    dd, kk = tree.query(np.c_[R["x"], R["y"]])
    ch = s[kk] / 1000
    return {**{k: v for k, v in p.items()}, "t_port_min": R["t_port"] / 60, "runout_km": float
    (np.percentile(ch, 95)),
            "dep_median_km": float(np.median(ch)), "exited": R["exited"], "runtime_s": dt,
```

```
            "vmax": float(R["vmax"].max()), "x": R["x"], "y": R["y"]}

def main():
    T = Terrain(2)
    src = release_cells(T)
    P, s, tree = chain_tools(T)
    V0 = 0.2 * src[2].sum() * 900.0

    if (RES / "q4_convergence.csv").exists() and (RES / "q4_sens_grid.csv").exists():
        conv = pd.read_csv(RES / "q4_convergence.csv")
        G = pd.read_csv(RES / "q4_sens_grid.csv")
    else:
        cfg = [{"V0": V0, "N": n, "dt": dt, "tmax": 1800} for n in (200, 400, 800, 1600) for
    dt in (0.5, 1.0, 2.0)]
        rows = Parallel(n_jobs=2)(delayed(one)(T, src, tree, s, c) for c in cfg)
        conv = pd.DataFrame([{k: v for k, v in r.items() if k not in ("x", "y")} for r in rows
    ])
        conv.to_csv(RES / "q4_convergence.csv", index=False)

        grid = [{"V0": v * 1e6, "mu_df": mu, "N": 300, "tmax": 2400} for v in (2, 4, 7.6, 12,
    18) for mu in (0.05, 0.065, 0.08)]
        rows = Parallel(n_jobs=2)(delayed(one)(T, src, tree, s, c) for c in grid)
        G = pd.DataFrame([{k: v for k, v in r.items() if k not in ("x", "y")} for r in rows])
        G.to_csv(RES / "q4_sens_grid.csv", index=False)

    base = [r for r in conv.itertuples() if r.N == 800 and r.dt == 1.0][0]
    zr = T.zraw
    src_lon, src_lat = src[0], src[1]
    x0, y0 = T.ll2xy(src_lon, src_lat)
    i0, j0 = T.xy2ij(x0, y0)
    z_top = float(np.median(zr[np.clip(i0, 0, T.ny - 1), np.clip(j0, 0, T.nx - 1)]))
    k = np.argmin(np.abs(s / 1000 - base.dep_median_km))
    z_dep = float(zr[P[k, 0], P[k, 1]])
    xs, ys = np.median(x0), np.median(y0)
    xt, yt = (P[0, 1] + 0.5) * T.dx, -(P[0, 0] + 0.5) * T.dy
    L = float(np.hypot(xt - xs, yt - ys) + s[k])
    HL_sim = (z_top - z_dep) / L
    HL_emp = 10 ** (-0.15666 * np.log10(V0) + 0.62419)
    out = {k: float(v) for k, v in {"z_top": z_top, "z_dep": z_dep, "L_m": L, "HL_sim": HL_sim
    , "HL_scheidegger": HL_emp,
                                    "L_scheidegger_m": (z_top - z_dep) / HL_emp}.items()}
    json.dump(out, open(RES / "q4_energyline.json", "w"), indent=1)
    print(conv.round(3).to_string())
    print(G.round(3).to_string())
    print(out)

if __name__ == "__main__":
    main()
```

B.9 tables.py 论文表格生成

```
import json
from pathlib import Path
import pandas as pd

ROOT = Path(__file__).resolve().parent.parent
RES, OUT = ROOT / "results", ROOT / "paper"
OUT.mkdir(exist_ok=True)

def w(name, s):
    (OUT / f"tab_{name}.tex").write_text(s, encoding="utf-8")

def j(n):
    return json.load(open(RES / n, encoding="utf-8"))

def t_q1():
    A = pd.read_csv(RES / "q1_association.csv")
    rows = []
    for r in A[(A["rank"] <= 12) | A.top5].to_dict("records"):
        star = r"$^\star$" if r["top5"] else ""
        rows.append(rf"{r['rank']} & {r['name']}{star} & {r['group']} & {r['direction']} & {r
    ['rarity']:.3f} & "
                    rf"{r['consistency']:.2f} & {r['trend']:.2f} & {r['absz']:.2f} & {r['
    closeness']:.3f} \\")
    w("q1_rank", "\n".join(rows))
```

```
    unit = {"VPD": "kPa", "DTR": "°C", "inv_frac": "无量纲", "Rld": "W/m$^2$", "P30": "mm"}
    rows = []
    for r in A[A.top5].to_dict("records"):
        rows.append(rf"{r['name']} & {unit.get(r['feature'], '')} & {r['direction']} & {r['
    x_2025-07-08']:.2f} & {r['clim_mean_2025-07-08']:.2f} & "
                    rf"{r['p_2025-07-08'] * 100:.1f} & {r['x_2026-08-26']:.2f} & {r['
    clim_mean_2026-08-26']:.2f} & "
                    rf"{r['p_2026-08-26'] * 100:.1f} \\")
    w("q1_top5", "\n".join(rows))
    X = json.load(open(RES / "extra_stats.json", encoding="utf-8"))["ext"]
    nm = {"heavy": "强降水日", "heat": "高温日", "warmdry": "高温少雨日"}
    w("q1_ext", "\n".join(rf"{nm[k]} & {X['2025-07-08'][k]} & {X['2025-07-08_clim'][k]:.2f} &
    {X['2026-08-26'][k]} & {X['2026-08-26_clim'][k]:.2f} \\" for k in nm))

def t_q2():
    C = pd.read_csv(RES / "q2_threshold_combos.csv").head(8)
    nm = {"P15": "15 日降水", "P30": "30 日降水", "P45": "45 日降水", "API15": "前期有效降水",
          "PDD3": "源区 3 日正积温", "PDD1": "源区 1 日正积温", "PDD5": "源区 5 日正积温", "VPD3": "
    3 日 VPD", "DTR3": "3 日 DTR"}
    w("q2_combos", "\n".join(rf"{nm[r.X1]} & {nm[r.X2]} & {r.c1:.2f} & {r.c2:.2f} & {r.hits}/2
     & {r.loo_hits}/2 & {r.false_days} & {r.FAR_daily * 100:.2f} \\"
                             for r in C.itertuples()))
    S = j("q2_summary.json")
    ad = S["alarm_days_by_year"]
    w("q2_alarm", "、".join(str(ad[str(y)] if str(y) in ad else ad.get(y, 0)) for y in range
    (2021, 2027)))

def t_q3():
    R = pd.read_csv(RES / "q3_forecast_eval.csv", parse_dates=["issue", "lo", "median", "hi"])
    R = R[R["mode"] == "联合标定"]
    f = lambda t: "无" if pd.isna(t) else t.strftime("%m-%d %H:%M")
    rows = []
    for r in R.itertuples():
        rows.append(rf"{r.event} & {r.lead_h} & {r.p_trigger:.3f} & {r.p48:.3f} & {f(r.lo)} &
    {f(r.median)} & {f(r.hi)} & "
                    rf"{'无' if pd.isna(r.width_h) else f'{r.width_h:.1f}'} & {'是' if r.
    covered else '否'} \\")
    w("q3_eval", "\n".join(rows))
    Cd = pd.read_csv(RES / "q3_pstar.csv")
    w("q3_pstar", "\n".join(rf"{r.p_star:.2f} & {r.hit_rate * 100:.0f} & {r.false_rate *
    100:.2f} & {int(r.warn_days)} & {int(r.false_days)} & {r.youden:.3f} \\"
                            for r in Cd.itertuples()))

def t_q4():
    S = j("q4_summary.json")
    p = S["params"]
    w("q4_params", "\n".join([
        rf"释放体积 $V_0$ & $7.56\times10^6$ m$^3$ & 坡度大于 $30^\circ$、海拔高于 5000 m 的冰体共 {
    S['V_steep_m3'] / 1e6:.1f}$\times10^6$ m$^3$，取其 20\% \\",
        rf"冰岩崩 $\mu,\ \xi$ & {p['mu_ice']}，{p['xi_ice']:.0f} m/s$^2$ & 海拔 {p['z_term']:.0f
    } m 以上 \\",
        rf"泥石流 $\mu,\ \xi$ & {p['mu_df']}，{p['xi_df']:.0f} m/s$^2$ & 沿程侵蚀，体积最多增至 {p
    ['bulk']} 倍 \\",
        rf"山洪 $\mu,\ \xi$ & {p['mu_fl']}，{p['xi_fl']:.0f} m/s$^2$ & 汇水面积$>${p['area_fl
    ']:.0f} km$^2$ 且纵坡$<${p['slope_fl']} \\",
        rf"网格、粒子数、步长 & 55 m$\times$61 m，{p['N']}，{p['dt']} s & 坡度由填洼后的 DEM 计算 \\"
    ]))
    if "mc" in S:
        m = S["mc"]
        cor = m["corr_runout"]
        nm = {"V0": "释放体积", "mu_df": "泥石流 $\\mu$", "xi_df": "泥石流 $\\xi$", "mu_fl": "山洪
     $\\mu$", "xi_fl": "山洪 $\\xi$", "bulk": "侵蚀增大倍数"}
        w("q4_corr", "\n".join(rf"{nm[k]} & {v:.3f} \\" for k, v in sorted(cor.items(), key=
    lambda t: -abs(t[1]))))

def nums_q4():
    S = j("q4_summary.json")
    m = S.get("mc", {})
    q = lambda v, k=0: f"{v:.{k}f}"
    lines = [rf"\newcommand\TPORT{{{q(S['t_port_min'], 1)}}}", rf"\newcommand\HPORT{{{q(S['
    hmax_port'], 1)}}}",
             rf"\newcommand\VFIN{{{q(S['vol_final_m3'] / 1e6, 1)}}}", rf"\newcommand\VMAX{{{q(
    S['vmax'], 0)}}}",
             rf"\newcommand\DEPMED{{{q(S['dep_median_km'], 1)}}}", rf"\newcommand\DAMA{{{q(S['
    dam_reaches_km'][0], 1)}}}",
             rf"\newcommand\DAMB{{{q(S['dam_reaches_km'][1], 1)}}}", rf"\newcommand\PORTKM{{{q
```

```
    (S['port_chainage_km'], 1)}}}",
             rf"\newcommand\EXIT{{{q(S['exited'] * 100, 1)}}}"]
    if m:
        lines += [rf"\newcommand\MCP{{{q(m['p_reach_port'] * 100, 0)}}}",
                  rf"\newcommand\MCTA{{{q(m['t_port_q'][0], 1)}}}", rf"\newcommand\MCTM{{{q(m
    ['t_port_q'][1], 1)}}}",
                  rf"\newcommand\MCTB{{{q(m['t_port_q'][2], 1)}}}",
                  rf"\newcommand\MCRA{{{q(m['runout_q'][0], 1)}}}", rf"\newcommand\MCRM{{{q(m
    ['runout_q'][1], 1)}}}",
                  rf"\newcommand\MCRB{{{q(m['runout_q'][2], 1)}}}",
                  rf"\newcommand\LATM{{{q(S['lateral_base_m'][0])}}}", rf"\newcommand\LATB{{{q
    (S['lateral_base_m'][1])}}}",
                  rf"\newcommand\MCVF{{{q((m['V50_reach'] or 0) / 1e6, 1)}}}"]
    (OUT / "nums_q4.tex").write_text("\n".join(lines), encoding="utf-8")

def t_cmp():
    C = pd.read_csv(RES / "cmp_q1.csv").replace({"单指标稀有度": "只用异常程度", "CRITIC-TOPSIS":
    "CRITIC 赋权 TOPSIS"})
    w("cmp_q1", "\n".join(rf"{r.method} & {r.top5} & {r.overlap}/5 & {r.kendall:.3f} & {r.ms
    :.3f} \\" for r in C.itertuples()))
    C = pd.read_csv(RES / "cmp_q2_temporal.csv")
    n_days = 14 / float(pd.read_csv(RES / "q2_threshold_combos.csv").iloc[0].FAR_daily)
    w("cmp_q2t", "\n".join(rf"{r.method} & {r.hits}/2 & {r.false_days} & {r.false_days /
    n_days * 100:.2f} & {r.ms:.2f} \\" for r in C.itertuples()))
    C = pd.read_csv(RES / "cmp_q2_spatial.csv")
    w("cmp_q2s", "\n".join(rf"{r.method} & {r.AUC_cv:.3f} & {r.AUC_block:.3f} & {r.hit_top10 *
     100:.1f} & {r.fit_ms:.0f} \\" for r in C.itertuples()))
    C = pd.read_csv(RES / "cmp_q3.csv").replace({"滞后融化距平（本文）": "本文模型", "无滞后融化距平"
    : "不计滞后",
                                                 "日内均匀": "日内均匀分配", "首个警戒日必触发": "首
    个警戒日必然触发"})
    w("cmp_q3", "\n".join(rf"{r.method} & {r.coverage * 100:.0f} & {r.width_h:.1f} & {r.
    abs_err_h:.1f} & {r.abs_err_le24h:.1f} & {r.ms:.0f} \\" for r in C.itertuples()))
    if (RES / "cmp_q4.csv").exists():
        C = pd.read_csv(RES / "cmp_q4.csv")
        w("cmp_q4", "\n".join(rf"{r.method} & {'未到达' if pd.isna(r.t_port_min) else f'{r.
    t_port_min:.1f}'} & {r.dep_median_km:.1f} & {r.exited * 100:.1f} & {r.vol_final:.1f} & {r.
    runtime_s:.0f} \\" for r in C.itertuples()))
    if (RES / "q4_convergence.csv").exists():
        C = pd.read_csv(RES / "q4_convergence.csv").sort_values(["dt", "N"])
        w("q4_conv", "\n".join(rf"{r.N} & {r.dt} & {r.t_port_min:.1f} & {r.runout_km:.1f} & {r
    .dep_median_km:.1f} & {r.runtime_s:.1f} \\" for r in C.itertuples()))
        E = j("q4_energyline.json")
        ev = j("cmp_q1_events.json")
        (OUT / "nums_cmp.tex").write_text("\n".join([
            rf"\newcommand\HLsim{{{E['HL_sim']:.3f}}}", rf"\newcommand\HLemp{{{E['
    HL_scheidegger']:.3f}}}",
            rf"\newcommand\Lsim{{{E['L_m'] / 1000:.1f}}}", rf"\newcommand\Lemp{{{E['
    L_scheidegger_m'] / 1000:.1f}}}",
            rf"\newcommand\Ztop{{{E['z_top']:.0f}}}", rf"\newcommand\Zdep{{{E['z_dep']:.0f}}}"
    ,
            rf"\newcommand\EVrho{{{ev['spearman_events']:.2f}}}"]), encoding="utf-8")

if __name__ == "__main__":
    t_q1(); t_q2(); t_q3(); t_q4(); nums_q4(); t_cmp()
    print("ok")
```

B.10 figs_res.py 结果图

```
import sys
import json
import glob
import pickle
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import matplotlib.font_manager as fm
from matplotlib.colors import LightSource, LinearSegmentedColormap, ListedColormap
from matplotlib.lines import Line2D
from matplotlib.patches import Patch, Rectangle
import rasterio

for f in glob.glob("/home/hero/.fonts/*"):
```

```
    fm.fontManager.addfont(f)
sys.path.insert(0, str(__import__("pathlib").Path(__file__).parent / "shumo_tools"))
from shumo_sci import use_sci, panel, tag, top_legend, soft_cmap, CM, BLUE, RED, TEAL, CYAN,
    SLATE, INK, FILL
from common import RES, FIG, DATA, EXT, EVENTS, FEATS, PORT, STATION

use_sci(body_pt=12, font="SimSun")
AMBER = "#D9B06A"
EVC = {"2025-07-08": TEAL, "2026-08-26": RED}

def save(fig, name):
    fig.savefig(FIG / f"{name}.png", dpi=300)
    plt.close(fig)

def hillshade(z, dx, dy):
    return LightSource(azdeg=315, altdeg=40).hillshade(z, vert_exag=1.5, dx=dx, dy=dy)

def lonlat_axes(ax, tr, shape):
    ny, nx = shape
    return [tr[2], tr[2] + tr[0] * nx, tr[5] + tr[4] * ny, tr[5]]

def fig_area():
    from rasterio.windows import from_bounds
    with rasterio.open(EXT / "cop30_N28E085.tif") as r:
        win = from_bounds(85.05, 28.2, 85.65, 29.0, r.transform)
        z = r.read(1, window=win, out_shape=(267, 200)).astype(float)
        tr = r.window_transform(win)
        ext = [85.05, 85.65, 28.2, 29.0]
    dx = 0.6 / 200 * 111320 * np.cos(np.deg2rad(28.6))
    dy = 0.8 / 267 * 110574
    hz = pd.read_csv(RES / "q2_hazard_points.csv")
    B = pickle.load(open(RES / "q4_base.pkl", "rb"))
    with rasterio.open(DATA / "d7" / "DEM.tif") as r:
        z2 = r.read(1, out_shape=(574, 908)).astype(float)
        b = r.bounds
    z2[z2 < -1000] = np.nan
    ext2 = [b.left, b.right, b.bottom, b.top]
    fig, (a, c) = plt.subplots(1, 2, figsize=(16 * CM, 8.2 * CM), gridspec_kw={"width_ratios":
     [0.78, 1.3]})
    a.imshow(hillshade(z, dx, dy), extent=ext, cmap="gray", vmin=0, vmax=1.2, alpha=0.85)
    a.imshow(z, extent=ext, cmap="terrain", alpha=0.35)
    cols = {"降雨型": BLUE, "冰水混合型": TEAL, "冰雪融水型": CYAN, "冰湖溃决型": RED}
    for t, cc in cols.items():
        s = hz[hz["类型"] == t]
        a.scatter(s.lon, s.lat, s=10, color=cc, edgecolor="white", linewidth=0.3, zorder=3)
    a.scatter([STATION[1]], [STATION[0]], marker="^", s=40, color=INK, zorder=4)
    a.scatter([PORT[1]], [PORT[0]], marker="*", s=70, color=RED, edgecolor="white", linewidth
    =0.4, zorder=4)
    a.add_patch(Rectangle((ext2[0], ext2[2]), ext2[1] - ext2[0], ext2[3] - ext2[2], fill=False
    , ec=INK, lw=0.9, ls="--"))
    a.set_xlim(85.05, 85.65); a.set_ylim(28.2, 29.0)
    a.set_xticks([85.1, 85.3, 85.5]); a.set_yticks([28.3, 28.5, 28.7, 28.9])
    a.set_xlabel("经度 / °E"); a.set_ylabel("纬度 / °N")
    panel(a); tag(a, "a")
    c.imshow(hillshade(np.nan_to_num(z2, nan=np.nanmean(z2)), 54.4, 61.4), extent=ext2, cmap="
    gray", vmin=0, vmax=1.2)
    c.imshow(z2, extent=ext2, cmap="terrain", alpha=0.3)
    glon, glat = B["glacier_ll"]
    c.scatter(glon[::3], glat[::3], s=0.3, color="#9EC9E2", zorder=2)
    slon, slat = B["src_ll"]
    c.scatter(slon[::3], slat[::3], s=0.3, color=BLUE, zorder=3)
    P, tr2 = B["P"], B["tr"]
    plon = tr2[2] + (P[:, 1] + 0.5) * tr2[0]
    plat = tr2[5] + (P[:, 0] + 0.5) * tr2[4]
    c.plot(plon, plat, color=RED, lw=1.2, zorder=3)
    c.scatter([PORT[1]], [PORT[0]], marker="*", s=90, color=RED, edgecolor="white", linewidth
    =0.5, zorder=4)
    s = hz[(hz.lon > ext2[0]) & (hz.lat < ext2[3])]
    for t, cc in cols.items():
        q = s[s["类型"] == t]
        c.scatter(q.lon, q.lat, s=10, color=cc, edgecolor="white", linewidth=0.3, zorder=3)
    c.set_xlim(ext2[0], ext2[1]); c.set_ylim(ext2[2], ext2[3])
    c.set_xticks([85.2, 85.3, 85.4, 85.5, 85.6]); c.set_yticks([28.3, 28.4, 28.5])
    c.set_xlabel("经度 / °E")
    panel(c); tag(c, "b")
    hs = [Line2D([], [], marker="o", ls="", ms=4.5, color=cc, label=t.replace("型", "")) for t,
```

```
     cc in cols.items()] +\
         [Line2D([], [], marker="^", ls="", ms=5, color=INK, label="观测塔"),
          Line2D([], [], marker="*", ls="", ms=8, color=RED, label="口岸")]
    top_legend(fig, hs, ncol=6, rows=1, handletextpad=0.2, columnspacing=0.8)
    save(fig, "fig_area")

def fig_q1_process():
    d = pd.read_csv(RES / "q1_daily.csv", index_col=0, parse_dates=True)
    fig, axes = plt.subplots(1, 2, figsize=(16 * CM, 7.2 * CM))
    for ax, (k, ev), tg in zip(axes, EVENTS.items(), "ab"):
        s = d.loc[ev.floor("D") - pd.Timedelta(days=30): ev.floor("D") + pd.Timedelta(days=1)]
        clim = d[np.abs(d.index.dayofyear - ev.dayofyear) <= 30]
        x = (s.index - ev.floor("D")).days
        ax.bar(x, s.P, color=CYAN, width=0.8)
        ax.set_ylim(0, 40)
        ax.set_xlabel("距事件日 / d")
        b = ax.twinx()
        b.plot(x, s.VPD, color=RED, lw=1.5)
        b.axhline(clim.VPD.mean(), color=RED, lw=0.8, ls=":")
        b.plot(x, s.DTR / 20, color=BLUE, lw=1.5)
        b.axhline(clim.DTR.mean() / 20, color=BLUE, lw=0.8, ls=":")
        b.set_ylim(0, 1.3)
        ax.axvline(0, color=INK, lw=0.8, ls="--")
        panel(ax, minor=("x",)); tag(ax, tg)
        b.tick_params(direction="in", labelsize=12)
        if tg == "a":
            ax.set_ylabel("日降水 / mm")
        else:
            b.set_ylabel("VPD / kPa，DTR / 20 °C")
    hs = [Patch(color=CYAN, label="日降水"), Line2D([], [], color=RED, lw=1.5, label="VPD"),
          Line2D([], [], color=BLUE, lw=1.5, label="DTR/20"), Line2D([], [], color=INK, lw
    =0.8, ls=":", label="同期均值")]
    top_legend(fig, hs, ncol=4, rows=1)
    save(fig, "fig_q1_process")

def fig_q1_bl():
    c = pd.read_csv(RES / "q1_diurnal.csv", header=[0, 1, 2], index_col=0)
    fig, axes = plt.subplots(2, 3, figsize=(16 * CM, 10.4 * CM))
    spec = [("dTdz", "温度梯度 / K•(100 m)^-1"), ("Rn", "净辐射 / W•m^-2"), ("H", "感热通量 /
    W•m^-2"),
            ("LE", "潜热通量 / W•m^-2"), ("TKE", "湍流动能 / m²•s^-2"), ("T2", "2 m 气温 / °C")]
    for ax, (v, lab), tg in zip(axes.ravel(), spec, "abcdef"):
        for k in EVENTS:
            ax.plot(c.index, c[(k, v, "pre")], color=EVC[k], lw=1.5)
            ax.plot(c.index, c[(k, v, "clim")], color=EVC[k], lw=1.0, ls="--")
        if v == "dTdz":
            ax.axhline(0, color=SLATE, lw=0.6)
        ax.set_xticks([0, 6, 12, 18, 23])
        ax.set_ylabel(lab, fontsize=11)
        if tg in "def":
            ax.set_xlabel("北京时 / h")
        panel(ax); tag(ax, tg)
    hs = [Line2D([], [], color=TEAL, lw=1.5, label="2025 前一周"), Line2D([], [], color=RED, lw
    =1.5, label="2026 前一周"),
          Line2D([], [], color=INK, lw=1.0, ls="--", label="同期气候")]
    top_legend(fig, hs, ncol=3, rows=1, w_pad=0.8)
    save(fig, "fig_q1_bl")

def fig_q1_rank():
    A = pd.read_csv(RES / "q1_association.csv")
    A = A[(A["rank"] <= 12) | A.top5]
    gcol = {"水汽": CYAN, "热力": RED, "边界层": TEAL, "辐射": AMBER, "天气过程": BLUE, "土壤":
    SLATE, "湍流": "#B9A0C9"}
    fig, (a, b) = plt.subplots(1, 2, figsize=(16 * CM, 10.5 * CM), gridspec_kw={"width_ratios"
    : [1.15, 1]})
    y = np.arange(len(A))[::-1]
    a.barh(y, A.closeness, color=[gcol[g] for g in A.group], height=0.7)
    for yy, t5 in zip(y, A.top5):
        if t5:
            a.scatter(A.closeness.max() * 1.06, yy, marker="*", s=40, color=INK)
    a.set_yticks(y, A.name)
    a.set_xlim(0, 1.15)
    a.set_xlabel("TOPSIS 贴近度")
    panel(a, minor=("x",)); tag(a, "a")
    for k, mk in zip(EVENTS, ["o", "s"]):
        b.scatter(A[f"p_{k}"] * 100, y, marker=mk, s=22, color=EVC[k], zorder=3)
```

```
    b.axvspan(10, 90, color=FILL, zorder=0)
    b.set_yticks(y, [""] * len(y))
    b.set_xlim(-3, 103)
    b.set_xlabel("同期百分位 / %")
    panel(b, minor=("x",)); tag(b, "b")
    hs = [Patch(color=c, label=g) for g, c in gcol.items() if g in set(A.group)] +\
         [Line2D([], [], marker="o", ls="", color=TEAL, label="2025"), Line2D([], [], marker="
    s", ls="", color=RED, label="2026")]
    top_legend(fig, hs, ncol=5, rows=2)
    save(fig, "fig_q1_rank")

def fig_q2_crit():
    ev = pd.read_csv(RES / "q2_rain_events.csv", parse_dates=["start", "end"])
    S = json.load(open(RES / "q2_summary.json", encoding="utf-8"))
    C = pd.read_csv(RES / "q2_critical_daily.csv", index_col=0, parse_dates=True)
    fig, (a, b) = plt.subplots(1, 2, figsize=(16 * CM, 7.6 * CM))
    a.scatter(ev.D, ev.I, s=5, color=SLATE, alpha=0.6)
    D = np.logspace(0, 3, 50)
    for (al, be), ls in zip([(14.82, 0.39), (2.20, 0.44), (73.90, 0.79)], ["-", "--", ":"]):
        a.plot(D, al * D ** -be, color=INK, lw=1.0, ls=ls)
    for k, r in S["rain_before_events"].items():
        a.scatter(r["D"], r["I"], s=50, marker="*", color=EVC[k], edgecolor="white", linewidth
    =0.5, zorder=4)
    a.set_xscale("log"); a.set_yscale("log")
    a.set_xlabel("降雨历时 D / h"); a.set_ylabel("平均雨强 I / mm·h^-1")
    a.set_ylim(0.01, 100)
    panel(a); tag(a, "a")
    sea = C[C.index.month.isin(range(5, 11))].dropna()
    b.scatter(sea.X1, sea.X2, s=4, color=SLATE, alpha=0.5)
    b.scatter(sea[sea.alarm].X1, sea[sea.alarm].X2, s=8, color=AMBER, zorder=3)
    cr = S["critical"]
    b.axvline(cr["c1"], color=INK, lw=0.9, ls="--")
    b.axhline(cr["c2"], color=INK, lw=0.9, ls="--")
    for k, ev_t in EVENTS.items():
        dd = ev_t.floor("D")
        b.scatter(C.loc[dd, "X1"], C.loc[dd, "X2"], s=70, marker="*", color=EVC[k], edgecolor=
    "white", linewidth=0.5, zorder=4)
    b.set_xlabel("前 15 日降水 / mm")
    b.set_ylabel("源区 3 日正积温 / °C·d")
    panel(b); tag(b, "b")
    hs = [Line2D([], [], color=INK, lw=1, ls="-", label="Caine"), Line2D([], [], color=INK, lw
    =1, ls="--", label="Guzzetti"),
          Line2D([], [], color=INK, lw=1, ls=":", label="Dahal"), Line2D([], [], marker="*",
    ls="", ms=8, color=TEAL, label="2025"),
          Line2D([], [], marker="*", ls="", ms=8, color=RED, label="2026"), Line2D([], [],
    marker="o", ls="", ms=4, color=AMBER, label="警戒日")]
    top_legend(fig, hs, ncol=6, rows=1, columnspacing=0.8, handlelength=1.4)
    save(fig, "fig_q2_crit")

def fig_q2_spatial():
    prob = np.load(RES / "q2_suscept.npy")
    meta = json.load(open(RES / "q2_suscept_meta.json"))
    tr = meta["transform"]
    ny, nx = prob.shape
    ext = [tr[2], tr[2] + tr[0] * nx, tr[5] + tr[4] * ny, tr[5]]
    hz = pd.read_csv(RES / "q2_hazard_points.csv")
    S = json.load(open(RES / "q2_summary.json", encoding="utf-8"))["spatial"]
    fig, (a, b) = plt.subplots(1, 2, figsize=(16 * CM, 8.0 * CM), gridspec_kw={"width_ratios":
     [0.85, 1]})
    im = a.imshow(prob, extent=ext, cmap=soft_cmap(RED, "#8C3B2E"), vmin=0, vmax=1)
    a.scatter(hz.lon, hz.lat, s=6, color=BLUE, edgecolor="white", linewidth=0.3)
    a.scatter([PORT[1]], [PORT[0]], marker="*", s=60, color=INK, edgecolor="white", linewidth
    =0.4)
    a.set_xlabel("经度 / °E"); a.set_ylabel("纬度 / °N")
    a.set_xticks([85.2, 85.4]); a.set_yticks([28.3, 28.5, 28.7, 28.9])
    cb = fig.colorbar(im, ax=a, fraction=0.05, pad=0.02)
    cb.ax.tick_params(direction="in")
    panel(a); tag(a, "a")
    if (RES / "cmp_q2_spatial.csv").exists():
        Cs = pd.read_csv(RES / "cmp_q2_spatial.csv")
        labs, a1, a2 = list(Cs.method), list(Cs.AUC_cv), list(Cs.AUC_block)
    else:
        labs = ["逻辑回归", "随机森林"]
        a1 = [S[l]["AUC_random_cv"] for l in labs]
        a2 = [S[l]["AUC_spatial_block"] for l in labs]
    x = np.arange(len(labs))
```

```
    b.bar(x - 0.18, a1, 0.34, color=BLUE)
    b.bar(x + 0.18, a2, 0.34, color=TEAL)
    b.set_xticks(x, [l.replace("梯度提升树", "梯度提升").replace("支持向量机", "SVM") for l in labs
    ])
    b.set_ylim(0.8, 1.0)
    b.set_ylabel("AUC")
    imp = S["rf_importance"]
    ins = b.inset_axes([0.47, 0.12, 0.5, 0.0])
    ins.set_visible(False)
    panel(b, minor=("y",)); tag(b, "b")
    hs = [Patch(color=BLUE, label="随机交叉验证"), Patch(color=TEAL, label="空间分块验证")]
    top_legend(fig, hs, ncol=2, rows=1)
    save(fig, "fig_q2_spatial")

def fig_q3():
    L = pd.read_csv(RES / "q3_tau_lik.csv")
    R = pd.read_csv(RES / "q3_forecast_eval.csv", parse_dates=["issue", "lo", "median", "hi"])
    fig, (a, b) = plt.subplots(1, 2, figsize=(16 * CM, 7.4 * CM), gridspec_kw={"width_ratios":
     [0.8, 1.2]})
    for k in EVENTS:
        a.plot(L.tau, L[f"L_{k}"] / L[f"L_{k}"].max(), color=EVC[k], lw=1.5)
    a.plot(L.tau, L.L_joint / L.L_joint.max(), color=INK, lw=1.5, ls="--")
    a.axhline(0.15, color=SLATE, lw=0.7, ls=":")
    a.set_xlabel("滞后 τ / h"); a.set_ylabel("相对似然")
    a.set_xlim(0, 23)
    panel(a); tag(a, "a")
    r = R[R["mode"] == "联合标定"].dropna(subset=["lo"])
    for k, off in zip(EVENTS, [-0.18, 0.18]):
        q = r[r.event == k]
        ev = EVENTS[k]
        y = np.arange(len(q)) + off
        lo = (q.lo - ev) / pd.Timedelta(hours=1)
        hi = (q.hi - ev) / pd.Timedelta(hours=1)
        md = (q["median"] - ev) / pd.Timedelta(hours=1)
        b.hlines(y, lo, hi, color=EVC[k], lw=2.2)
        b.scatter(md, y, s=18, color="white", edgecolor=EVC[k], zorder=3)
    b.axvline(0, color=INK, lw=0.9, ls="--")
    leads = sorted(r.lead_h.unique())
    b.set_yticks(range(len(leads)), [f"{int(l)} h" for l in leads])
    b.set_xlabel("相对实际触发时刻 / h")
    b.set_ylabel("提前起报")
    panel(b, minor=("x",)); tag(b, "b")
    hs = [Line2D([], [], color=TEAL, lw=2, label="2025"), Line2D([], [], color=RED, lw=2,
    label="2026"),
          Line2D([], [], color=INK, lw=1.5, ls="--", label="联合")]
    top_legend(fig, hs, ncol=3, rows=1)
    save(fig, "fig_q3")

def fig_q3_warn():
    Cd = pd.read_csv(RES / "q3_pstar.csv")
    Rl = pd.read_csv(RES / "q3_rolling.csv", parse_dates=["issue"])
    fig, (a, b) = plt.subplots(1, 2, figsize=(16 * CM, 7.0 * CM), gridspec_kw={"width_ratios":
     [0.8, 1.2]})
    a.plot(Cd.false_rate * 100, Cd.hit_rate * 100, "-o", color=BLUE, ms=4, lw=1.5)
    S = json.load(open(RES / "q3_summary.json", encoding="utf-8"))
    sel = Cd[Cd.p_star == S["p_star"]]
    a.scatter(sel.false_rate * 100, sel.hit_rate * 100, s=70, marker="*", color=RED, zorder=4)
    a.set_xlabel("误报率 / %"); a.set_ylabel("命中率 / %")
    a.set_ylim(-5, 105)
    panel(a); tag(a, "a")
    for yr, c in zip([2024, 2025, 2026], [SLATE, TEAL, RED]):
        q = Rl[Rl.issue.dt.year == yr]
        b.plot(q.issue.dt.dayofyear, q.p48, color=c, lw=1.0)
    b.axhline(S["p_star"], color=INK, lw=0.8, ls="--")
    for k, ev in EVENTS.items():
        b.axvline(ev.dayofyear, color=EVC[k], lw=0.8, ls=":")
    b.set_xlabel("年积日"); b.set_ylabel("48 h 触发概率")
    panel(b); tag(b, "b")
    hs = [Line2D([], [], color=c, lw=1.2, label=str(y)) for y, c in zip([2024, 2025, 2026], [
    SLATE, TEAL, RED])]
    top_legend(fig, hs, ncol=3, rows=1)
    save(fig, "fig_q3_warn")

def _base():
    B = pickle.load(open(RES / "q4_base.pkl", "rb"))
    tr = B["tr"]
```

```
    ny, nx = B["z"].shape
    ext = [tr[2], tr[2] + tr[0] * nx, tr[5] + tr[4] * ny, tr[5]]
    hs = hillshade(B["z"], B["dx"], B["dy"])
    return B, ext, hs

def fig_q4_maps():
    B, ext, hs = _base()
    fig, (a, b) = plt.subplots(1, 2, figsize=(16 * CM, 4.9 * CM))
    for ax in (a, b):
        ax.imshow(hs, extent=ext, cmap="gray", vmin=0, vmax=1.2)
        ax.scatter([PORT[1]], [PORT[0]], marker="*", s=60, color=INK, edgecolor="white",
    linewidth=0.4, zorder=4)
        ax.set_xlim(85.33, 85.58); ax.set_ylim(28.25, 28.36)
        ax.set_xticks([85.35, 85.45, 85.55]); ax.set_yticks([28.26, 28.30, 28.34])
        ax.set_xlabel("经度 / °E")
    h = np.where(B["hmax"] > 0.3, B["hmax"], np.nan)
    im = a.imshow(h, extent=ext, cmap=soft_cmap(BLUE, "#1F3B5A"), vmin=0, vmax=20, alpha=0.9)
    t = np.where(np.isfinite(B["tarr"]), B["tarr"] / 60, np.nan)
    im2 = b.imshow(t, extent=ext, cmap=LinearSegmentedColormap.from_list("t", [RED, AMBER,
    TEAL, BLUE]), vmin=0, vmax=30, alpha=0.9)
    a.set_ylabel("纬度 / °N")
    for ax, m, lab, tg in [(a, im, "最大流深 / m", "a"), (b, im2, "到达时间 / min", "b")]:
        cb = fig.colorbar(m, ax=ax, fraction=0.035, pad=0.02)
        cb.ax.tick_params(direction="in")
        cb.set_label(lab)
        panel(ax); tag(ax, tg)
    fig.tight_layout(w_pad=1.0)
    save(fig, "fig_q4_maps")

def fig_q4_snap():
    B, ext, hs = _base()
    want = [60, 180, 360, 600, 840, 1200]
    snaps = {int(s[0]): s for s in B["snaps"]}
    keys = [min(snaps, key=lambda k: abs(k - w)) for w in want]
    tr = B["tr"]
    fig, axes = plt.subplots(2, 3, figsize=(16 * CM, 7.6 * CM), sharex=True, sharey=True)
    pc = [BLUE, AMBER, RED]
    for ax, k, tg in zip(axes.ravel(), keys, "abcdef"):
        t, x, y, ph, sp = snaps[k]
        lon = tr[2] + x / B["dx"] * tr[0]
        lat = tr[5] - (-y) / B["dy"] * abs(tr[4])
        ax.imshow(hs, extent=ext, cmap="gray", vmin=0, vmax=1.2)
        ax.scatter(lon, lat, s=0.6, c=[pc[p] for p in ph], zorder=3)
        ax.scatter([PORT[1]], [PORT[0]], marker="*", s=40, color=INK, edgecolor="white",
    linewidth=0.3, zorder=4)
        ax.set_xlim(85.33, 85.58); ax.set_ylim(28.25, 28.36)
        ax.set_xticks([85.35, 85.45, 85.55]); ax.set_yticks([28.26, 28.31, 28.36])
        ax.set_title(f"({tg}) {t / 60:.0f} min", fontsize=12, loc="left", pad=4)
        ax.tick_params(direction="in", top=True, right=True)
    hs_ = [Line2D([], [], marker="o", ls="", ms=4, color=c, label=l) for c, l in zip(pc, ["冰岩
    崩", "泥石流", "山洪"])]
    top_legend(fig, hs_, ncol=3, rows=1)
    save(fig, "fig_q4_snap")

def fig_q4_profile():
    from scipy import ndimage
    P = pd.read_csv(RES / "q4_profile.csv")
    B = pickle.load(open(RES / "q4_base.pkl", "rb"))
    pp = B["P"]

    tmin = ndimage.minimum_filter(np.where(np.isfinite(B["tarr"]), B["tarr"], np.inf), 5)
    vmx = ndimage.maximum_filter(B["vmax"], 5)
    P["tarr_min"] = np.where(np.isfinite(tmin[pp[:, 0], pp[:, 1]]), tmin[pp[:, 0], pp[:, 1]] /
     60, np.nan)
    P["vmax"] = vmx[pp[:, 0], pp[:, 1]]
    S = json.load(open(RES / "q4_summary.json", encoding="utf-8"))
    fig, (a, b) = plt.subplots(2, 1, figsize=(16 * CM, 9.0 * CM), sharex=True)
    a.fill_between(P.chain_km, P.z.min() - 100, P.z, color=FILL)
    a.plot(P.chain_km, P.z, color=INK, lw=1.0)
    a.plot(P.chain_km, P.z + P.dep * 5, color=AMBER, lw=1.5)
    a.axvline(S["port_chainage_km"], color=RED, lw=0.9, ls="--")
    a.set_ylabel("高程 / m")
    panel(a); tag(a, "a")
    b.plot(P.chain_km, P.vmax, color=BLUE, lw=1.5)
    c = b.twinx()
    c.plot(P.chain_km, P.tarr_min, color=RED, lw=1.5)
```

```
    c.set_ylabel("到达时间 / min")
    c.tick_params(direction="in", labelsize=12)
    b.axvline(S["port_chainage_km"], color=RED, lw=0.9, ls="--")
    b.set_xlabel("距冰川末端沿程距离 / km")
    b.set_ylabel("最大速度 / m·s^-1")
    panel(b); tag(b, "b")
    hs = [Line2D([], [], color=INK, lw=1, label="河床"), Line2D([], [], color=AMBER, lw=1.5,
    label="堆积×5"),
          Line2D([], [], color=BLUE, lw=1.5, label="速度"), Line2D([], [], color=RED, lw=1.5,
    label="到达时间")]
    top_legend(fig, hs, ncol=4, rows=1)
    save(fig, "fig_q4_profile")

def fig_q4_mc():
    M = pd.read_csv(RES / "q4_mc.csv")
    S = json.load(open(RES / "q4_summary.json", encoding="utf-8"))
    fig, (a, b) = plt.subplots(1, 2, figsize=(16 * CM, 7.0 * CM))
    a.scatter(M.V0 / 1e6, M.reach_port.astype(float) + np.random.default_rng(0).uniform(-0.03,
     0.03, len(M)), s=14, color=BLUE)
    from sklearn.linear_model import LogisticRegression
    if M.reach_port.nunique() > 1:
        lr = LogisticRegression().fit(np.log10(M[["V0"]].values), M.reach_port.values)
        v = np.logspace(6, np.log10(2e7), 100)
        a.plot(v / 1e6, lr.predict_proba(np.log10(v)[:, None])[:, 1], color=RED, lw=1.5)
    a.set_xscale("log")
    a.set_xlabel("释放体积 / 10^6 m³"); a.set_ylabel("到达口岸概率")
    panel(a); tag(a, "a")
    b.hist(M.runout_km, bins=12, color=TEAL)
    b.axvline(S["port_chainage_km"], color=RED, lw=0.9, ls="--")
    q = S["mc"]["runout_q"]
    b.axvspan(q[0], q[2], color=FILL, zorder=0)
    b.set_xlabel("停积距离（95 分位）/ km"); b.set_ylabel("次数")
    panel(b); tag(b, "b")
    fig.tight_layout(w_pad=1.4)
    save(fig, "fig_q4_mc")

def fig_sens():
    s2 = pd.read_csv(RES / "sens_q2.csv")
    s3 = pd.read_csv(RES / "sens_q3.csv")
    g = pd.read_csv(RES / "q4_sens_grid.csv")
    fig, (a, b, c) = plt.subplots(1, 3, figsize=(16 * CM, 6.6 * CM))
    for w, col, mk in [("c1", BLUE, "o"), ("c2", RED, "s")]:
        q = s2[s2.which == w]
        a.plot(q.k, q.false_days, color=col, lw=1.5)
        full = q.hits == 2
        a.scatter(q.k[full], q.false_days[full], marker=mk, s=18, color=col, zorder=3)
        a.scatter(q.k[~full], q.false_days[~full], marker=mk, s=18, facecolor="white",
    edgecolor=col, zorder=3)
    a.set_xlabel("临界值倍数"); a.set_ylabel("误报日数")
    panel(a); tag(a, "a")
    b.plot(s3.tau, s3.abs_err_le24h, "-o", color=BLUE, ms=4, lw=1.5)
    b.plot(s3.tau, s3.abs_err_h, "-s", color=RED, ms=4, lw=1.5)
    b.set_xlabel("滞后 τ / h"); b.set_ylabel("中位数误差 / h")
    panel(b); tag(b, "b")
    for mu, col, mk in zip(sorted(g.mu_df.unique()), [BLUE, TEAL, RED], ["o", "s", "^"]):
        q = g[g.mu_df == mu].sort_values("V0")
        c.plot(q.V0 / 1e6, q.runout_km, marker=mk, ms=4, lw=1.5, color=col)
    c.axhline(21.4, color=INK, lw=0.8, ls="--")
    c.set_xlabel("释放体积 / 10^6 m³"); c.set_ylabel("停积距离 / km")
    panel(c); tag(c, "c")
    fig.tight_layout(w_pad=1.2)
    save(fig, "fig_sens")

def fig_q4_check():
    C = pd.read_csv(RES / "q4_convergence.csv")
    fig, (a, b) = plt.subplots(1, 2, figsize=(16 * CM, 6.4 * CM))
    for dt, col, mk in zip([0.5, 1.0, 2.0], [BLUE, TEAL, RED], ["o", "s", "^"]):
        q = C[C.dt == dt].sort_values("N")
        a.plot(q.N, q.t_port_min, marker=mk, ms=4, lw=1.5, color=col)
        b.plot(q.N, q.runtime_s, marker=mk, ms=4, lw=1.5, color=col)
    for ax in (a, b):
        ax.set_xscale("log"); ax.set_xticks([200, 400, 800, 1600], ["200", "400", "800", "1600
    "])
        ax.set_xlabel("粒子数 N")
    a.set_ylabel("到达口岸时间 / min"); b.set_ylabel("单次运行时间 / s")
    panel(a, minor=("y",)); tag(a, "a"); panel(b, minor=("y",)); tag(b, "b")
```

```
    hs = [Line2D([], [], color=c, marker=m, lw=1.5, ms=4, label=f"Δt={d} s") for d, c, m in
    zip([0.5, 1.0, 2.0], [BLUE, TEAL, RED], ["o", "s", "^"])]
    top_legend(fig, hs, ncol=3, rows=1)
    save(fig, "fig_q4_check")

if __name__ == "__main__":
    which = sys.argv[1:] or ["area", "q1", "q2", "q3", "q4", "extra"]
    if "extra" in which:
        if (RES / "q4_sens_grid.csv").exists():
            fig_sens()
        if (RES / "q4_convergence.csv").exists():
            fig_q4_check()
    if "area" in which:
        fig_area()
    if "q1" in which:
        fig_q1_process(); fig_q1_bl(); fig_q1_rank()
    if "q2" in which:
        fig_q2_crit(); fig_q2_spatial()
    if "q3" in which:
        fig_q3(); fig_q3_warn()
    if "q4" in which:
        fig_q4_maps(); fig_q4_snap(); fig_q4_profile()
        if (RES / "q4_mc.csv").exists():
            fig_q4_mc()
    print("ok")
```

B.11 figs_schema.py 思路图、框架图与示意图

```
import sys
import glob
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import matplotlib.font_manager as fm
from matplotlib.patches import Rectangle, FancyBboxPatch, Ellipse, Polygon, Circle,
    FancyArrowPatch, Wedge
from matplotlib.colors import to_rgb

for _f in glob.glob("/home/hero/.fonts/*"):
    fm.fontManager.addfont(_f)
sys.path.insert(0, str(__import__("pathlib").Path(__file__).parent / "shumo_tools"))
from shumo_fig import (use_style, soften, new_fig, box, arrow, save, BLUE, ORANGE, GREEN,
    AMBER, DARK,
                       L_BLUE, L_ORANGE, L_GREEN, L_AMBER, S_EDGE, S_TEXT, S_BLUE, S_ORANGE,
    S_GREEN, S_AMBER)
from common import FIG
SLATE_C, RED_C = "#A9B4C2", "#D98C7A"
W_EDGE, W_TEXT = "#7C7068", "#3F3833"
W1, W2, W3, W4 = "#FCE2CC", "#FBEFC4", "#DAE8F5", "#DDEFD9"
DEEP = {W1: "#F5BF93", W2: "#F1D980", W3: "#A9CAE8", W4: "#AFD6A7"}

FS = 9.0
CW = 0.03525 * FS
LH = 0.0476 * FS
LAB = 8.5

def arrow(ax, p0, p1, st, color=None):
    ax.add_patch(FancyArrowPatch(p0, p1, arrowstyle="-|>", mutation_scale=9, lw=st["arrow_lw"
    ],
                                 color=color or st["edge"], shrinkA=0, shrinkB=0, zorder=4))

def shade(c, k=0.94):
    return tuple(v * k for v in to_rgb(c))

def chars(t):
    return max(sum(1.0 if ord(c) > 0x2E80 else 0.52 for c in ln) for ln in t.split("\n"))

class B:
    def __init__(self, ax, st, text, cy, x=None, cx=None, fill=S_BLUE, kind="rect", w=None):
        self.w = w or chars(text) * CW + 0.5
        self.h = text.count("\n") * LH + LH + 0.26
        self.l = x if x is not None else cx - self.w / 2
        self.r, self.cx, self.cy = self.l + self.w, self.l + self.w / 2, cy
        self.t, self.b = cy + self.h / 2, cy - self.h / 2
```

```
        box(ax, self.l, self.b, self.w, self.h, text, st, kind=kind, fill=fill, per_line=99)

def line(ax, pts, st):
    xs, ys = zip(*pts)
    ax.plot(xs, ys, color=st["edge"], lw=st["arrow_lw"], zorder=1, solid_capstyle="butt")

def to(ax, st, pts):
    if len(pts) > 2:
        line(ax, pts[:-1], st)
    arrow(ax, pts[-2], pts[-1], st)

def fork(ax, st, a, ds, xb=None):
    xb = xb if xb is not None else (a.r + min(d.l for d in ds)) / 2
    line(ax, [(a.r, a.cy), (xb, a.cy)], st)
    ys = [d.cy for d in ds] + [a.cy]
    line(ax, [(xb, min(ys)), (xb, max(ys))], st)
    for d in ds:
        arrow(ax, (xb, d.cy), (d.l - 0.04, d.cy), st)

def merge(ax, st, ss, d, xb=None):
    xb = xb if xb is not None else (max(s.r for s in ss) + d.l) / 2
    for s in ss:
        line(ax, [(s.r, s.cy), (xb, s.cy)], st)
    ys = [s.cy for s in ss] + [d.cy]
    line(ax, [(xb, min(ys)), (xb, max(ys))], st)
    arrow(ax, (xb, d.cy), (d.l - 0.04, d.cy), st)

def bus(ax, st, ss, ds):
    xb = (max(s.r for s in ss) + min(d.l for d in ds)) / 2
    for s in ss:
        line(ax, [(s.r, s.cy), (xb, s.cy)], st)
    ys = [s.cy for s in ss] + [d.cy for d in ds]
    line(ax, [(xb, min(ys)), (xb, max(ys))], st)
    for d in ds:
        arrow(ax, (xb, d.cy), (d.l - 0.04, d.cy), st)

def h(ax, st, a, b):
    arrow(ax, (a.r + 0.04, a.cy), (b.l - 0.04, b.cy), st)

def band_l(ax, st, x0, x1, y0, y1, tag):
    ax.add_patch(Rectangle((x0, y0), x1 - x0, y1 - y0, fc="none", ec=st["edge"], lw=0.8,
    zorder=0))
    w = chars(tag) * CW + 0.4
    th = LH + 0.16
    ax.add_patch(Rectangle((x0 + 0.3, y1 - th / 2), w, th, fc="#FFF8F0", ec=st["edge"], lw
    =0.8, zorder=2))
    ax.text(x0 + 0.3 + w / 2, y1, tag, ha="center", va="center", fontsize=FS, color=st["text"
    ], zorder=3)

def rbox(ax, st, text, cx, cy, fill):
    w = chars(text) * CW + 0.36
    h = text.count("\n") * LH + LH + 0.16
    box(ax, cx - w / 2, cy - h / 2, w, h, text, st, kind="rect", fill=fill, per_line=99)
    return {"l": cx - w / 2, "r": cx + w / 2, "t": cy + h / 2, "b": cy - h / 2, "cx": cx, "cy"
    : cy, "w": w}

def spread(bs, x0, x1, gap=0.12):
    bs = sorted(bs, key=lambda b: b["cx"])
    for _ in range(50):
        moved = False
        for a, b in zip(bs[:-1], bs[1:]):
            d = a["r"] + gap - b["l"]
            if d > 1e-6:
                a["cx"] -= d / 2; b["cx"] += d / 2
                moved = True
            a["l"], a["r"] = a["cx"] - a["w"] / 2, a["cx"] + a["w"] / 2
            b["l"], b["r"] = b["cx"] - b["w"] / 2, b["cx"] + b["w"] / 2
        for b, sh in ((bs[0], max(0, x0 - bs[0]["l"])), (bs[-1], min(0, x1 - bs[-1]["r"]))):
            if sh:
                b["cx"] += sh; b["l"] += sh; b["r"] += sh
                moved = True
        if not moved:
            break

def spine(ax, st, spec, cy, x0, x1, dy):
    F = spec["fill"]
```

```
    main = spec["main"]
    ws = [chars(t) * CW + 0.36 for t in main]
    g = (x1 - x0 - sum(ws)) / (len(main) - 1)
    M, x = [], x0
    for k, (t, w) in enumerate(zip(main, ws)):
        M.append(rbox(ax, st, t, x + w / 2, cy, DEEP[F] if k == len(main) - 1 else F))
        x += w + g
    for a, b in zip(M[:-1], M[1:]):
        arrow(ax, (a["r"] + 0.03, cy), (b["l"] - 0.03, cy), st)
    out = {"main": M, "up": {}, "down": {}}
    for side, sg in (("up", 1), ("down", -1)):
        meta = []
        for k, t, d in spec[side]:
            w = chars(t) * CW + 0.36
            meta.append({"k": k, "t": t, "d": d, "w": w, "cx": M[k]["cx"], "l": M[k]["cx"] - w
     / 2, "r": M[k]["cx"] + w / 2})
        spread(meta, x0, x1)
        for m in meta:
            mb = M[m["k"]]
            bb = rbox(ax, st, m["t"], m["cx"], cy + sg * dy, "white" if m["d"] == "in" else
    DEEP[F])
            lo, hi = max(bb["l"], mb["l"]) + 0.15, min(bb["r"], mb["r"]) - 0.15
            xa = (lo + hi) / 2
            pa, pb = (xa, bb["b"] if sg > 0 else bb["t"]), (xa, mb["t"] if sg > 0 else mb["b"
    ])
            if m["d"] == "in":
                arrow(ax, (pa[0], pa[1] - sg * 0.03), (pb[0], pb[1] + sg * 0.03), st)
            else:
                arrow(ax, (pb[0], pb[1] + sg * 0.03), (pa[0], pa[1] - sg * 0.03), st)
            out[side][m["k"]] = bb
    return out

def specs(nums):
    return [
        {"fill": W1, "tag": "问题一",
         "main": ["附件 1 观测", "34 项逐日特征", "四项关联指标", "熵权 TOPSIS", "按类别筛选", "前 5
    因素"],
         "up": [(1, "边界层日变化", "out"), (2, "同期背景分布", "in"), (3, "等权与 CRITIC 对比", "
    out"), (5, nums["top5"], "out")],
         "down": [(1, "天气过程与极端", "out"), (4, "显著性检验", "out")]},
        {"fill": W2, "tag": "问题二",
         "main": ["订正 ERA5 降水", "I－D 阈值检验", "双因子临界", "留一检验", "时空分布", nums["crit"
    ]],
         "up": [(2, "问题一关联因素", "in"), (3, "误报日比例", "in"), (4, "逐年警戒日数", "out")],
         "down": [(1, "降雨非直接触发", "out"), (2, "20 种因子组合", "in"), (4, nums["auc"], "in")
    ]},
        {"fill": W3, "tag": "问题三",
         "main": ["订正预报", "日尺度警戒", "警戒日触发 q", "日内滞后 τ", "蒙特卡洛抽样", nums["ci"]],
         "up": [(1, "问题二临界条件", "in"), (3, nums["tau"], "in"), (4, "2000 次", "in"), (5,
    nums["lead"], "out")],
         "down": [(0, "归档预报 1－5 d", "in"), (2, nums["beta"], "in"), (4, nums["pstar"], "out
    "), (5, "问题四停积范围", "in")]},
        {"fill": W4, "tag": "问题四",
         "main": ["附件 4 冰厚", "释放体积", "填洼 DEM 坡度", "粒子 Voellmy", "分阶段取参数", nums["
    tport"]],
         "up": [(3, "网格累加求流深", "in"), (4, "冰岩崩 泥石流 山洪", "in"), (5, "堆积超 5 m 河段",
    "out")],
         "down": [(1, "释放比例 20%", "in"), (3, nums["mc"], "out"), (4, "沿程侵蚀", "in"), (5, "
    建模方式对比", "out")]},
    ]

def flow_style():
    st = soften(use_style("A", body_pt=12, font="SimSun"))
    st["text_pt"] = st["label_pt"] = FS
    st["arrow_lw"] = 0.9
    st["round"] = 0
    st["edge"], st["text"] = W_EDGE, W_TEXT
    return st

def fig_flow(nums):
    st = flow_style()
    W, X0, X1, LANE = 16.0, 0.12, 15.42, 15.72
    dy, bh, gap, top, bot = 0.92, 3.05, 0.62, 0.36, 0.1
    H = top + 4 * bh + 3 * gap + bot
    fig, ax = new_fig(W, H)
    S, R = specs(nums), []
```

```
    for k, sp in enumerate(S):
        t = H - top - k * (bh + gap)
        band_l(ax, st, X0, X1, t - bh, t, sp["tag"])
        R.append(spine(ax, st, sp, t - bh / 2 - 0.06, X0 + 0.25, X1 - 0.25, dy))
    for k, (src, dst) in enumerate([(R[0]["main"][-1], R[1]["up"][2]), (R[1]["main"][-1], R
    [2]["up"][1])]):
        yg = H - top - k * (bh + gap) - bh - gap / 2
        to(ax, st, [(src["cx"], src["b"]), (src["cx"], yg), (dst["cx"], yg), (dst["cx"], dst["
    t"] + 0.03)])
    a, b = R[3]["main"][-1], R[2]["down"][5]
    to(ax, st, [(a["r"], a["cy"]), (LANE, a["cy"]), (LANE, b["cy"]), (b["r"] + 0.03, b["cy"])
    ])
    save(fig, FIG / "fig_flow.png")

def mathfont():
    matplotlib.rcParams.update({"mathtext.fontset": "custom", "mathtext.rm": "Times New Roman"
    ,
                                "mathtext.it": "Times New Roman:italic", "mathtext.bf": "Times
     New Roman:bold"})

def lab(ax, x, y, s, ha="center", va="center", c=DARK, fs=8.5):
    ax.text(x, y, s, ha=ha, va=va, fontsize=fs, color=c, zorder=6)

def arr(ax, p0, p1, c=DARK, lw=1.0, ms=9, ls="-", rad=0.0):
    ax.add_patch(FancyArrowPatch(p0, p1, arrowstyle="-|>", mutation_scale=ms, lw=lw, color=c,
    shrinkA=0, shrinkB=0,
                                 zorder=4, linestyle=ls, connectionstyle=f"arc3,rad={rad}"))

def fig_mech_chain():
    use_style("A", body_pt=12, font="SimSun")
    mathfont()
    fig, ax = new_fig(12.0, 6.0)
    x = np.linspace(0, 12, 400)
    ground = 0.6 + 4.6 / (1 + np.exp((x - 2.6) / 0.75)) + 0.25 * np.exp(-((x - 8.2) / 0.9) **
    2) - 0.03 * x
    ax.fill_between(x, 0, ground, color="#E8E1D3", zorder=1)
    ax.plot(x, ground, color=DARK, lw=1.0, zorder=2)

    gx = x[(x > 0.2) & (x < 2.6)]
    gy = 0.6 + 4.6 / (1 + np.exp((gx - 2.6) / 0.75)) + 0.25 * np.exp(-((gx - 8.2) / 0.9) ** 2)
     - 0.03 * gx
    ax.fill_between(gx, gy, gy + 0.35 * np.clip((2.6 - gx) / 2.4, 0, 1) + 0.05, color=L_BLUE,
    ec=BLUE, lw=0.8, zorder=3)
    blk = Polygon([(2.05, 4.0), (2.55, 3.85), (2.45, 3.45), (1.95, 3.55)], closed=True, fc=
    L_BLUE, ec=BLUE, lw=1.0, zorder=4)
    ax.add_patch(blk)
    for k, (px, py) in enumerate([(2.9, 3.0), (3.15, 2.7), (3.35, 2.45)]):
        ax.add_patch(Polygon([(px, py), (px + 0.18, py - 0.05), (px + 0.1, py - 0.18)], closed
    =True, fc=L_BLUE, ec=BLUE, lw=0.7, zorder=4))

    dx_ = x[(x > 3.6) & (x < 7.2)]
    dy_ = 0.6 + 4.6 / (1 + np.exp((dx_ - 2.6) / 0.75)) + 0.25 * np.exp(-((dx_ - 8.2) / 0.9) **
     2) - 0.03 * dx_
    ax.fill_between(dx_, dy_, dy_ + 0.28 * np.sin(np.pi * (dx_ - 3.6) / 3.6), color="#B88A5A",
     alpha=0.85, zorder=3)
    rng = np.random.default_rng(2)
    for _ in range(14):
        px = rng.uniform(3.9, 6.9)
        py = 0.6 + 4.6 / (1 + np.exp((px - 2.6) / 0.75)) - 0.03 * px + rng.uniform(0.04, 0.18)
        ax.add_patch(Circle((px, py), rng.uniform(0.03, 0.07), fc="#7A5C3A", ec="none", zorder
    =4))
    fx = x[(x > 7.4) & (x < 11.8)]
    fy = 0.6 + 4.6 / (1 + np.exp((fx - 2.6) / 0.75)) + 0.25 * np.exp(-((fx - 8.2) / 0.9) ** 2)
     - 0.03 * fx
    ax.fill_between(fx, fy, fy + 0.16, color=L_BLUE, alpha=0.9, zorder=3)

    ax.add_patch(Polygon([(8.0, 0.83), (8.45, 1.18), (8.9, 0.83)], closed=True, fc="#9C7B55",
    ec=DARK, lw=0.6, zorder=4))

    for k, (bx, bh) in enumerate([(10.3, 0.45), (10.75, 0.6), (11.2, 0.38)]):
        by = 0.6 + 4.6 / (1 + np.exp((bx - 2.6) / 0.75)) - 0.03 * bx + 0.16
        ax.add_patch(Rectangle((bx, by), 0.32, bh, fc="white", ec=DARK, lw=0.8, zorder=4))
        ax.add_patch(Polygon([(bx - 0.04, by + bh), (bx + 0.16, by + bh + 0.18), (bx + 0.36,
    by + bh)], closed=True, fc=L_ORANGE, ec=DARK, lw=0.6, zorder=4))

    for k, (cx, cy, r) in enumerate([(4.9, 5.25, 0.32), (5.3, 5.38, 0.4), (5.75, 5.25, 0.3)]):
```

```
        ax.add_patch(Circle((cx, cy), r, fc="#E6E9EE", ec=S_EDGE, lw=0.6, zorder=2))
    for k in range(5):
        ax.plot([4.85 + 0.25 * k, 4.75 + 0.25 * k], [4.85, 4.55], color=BLUE, lw=0.8, zorder
    =2)
    ax.add_patch(Circle((0.9, 5.45), 0.3, fc=L_AMBER, ec=AMBER, lw=1.0, zorder=3))
    for t in np.linspace(0, 2 * np.pi, 9)[:-1]:
        ax.plot([0.9 + 0.4 * np.cos(t), 0.9 + 0.55 * np.cos(t)], [5.45 + 0.4 * np.sin(t), 5.45
     + 0.55 * np.sin(t)], color=AMBER, lw=1.0)
    arr(ax, (1.35, 5.15), (1.9, 4.35), c=AMBER, lw=0.9, ms=7)

    lab(ax, 2.75, 4.05, "冰崩", ha="left")
    lab(ax, 3.6, 2.75, "碎屑流", ha="left")
    lab(ax, 5.3, 1.85, "泥石流")
    lab(ax, 9.75, 1.15, "山洪")
    lab(ax, 8.45, 1.45, "堰塞体")
    lab(ax, 10.9, 1.78, "口岸")
    lab(ax, 6.1, 5.3, "$P_{15}$", ha="left")
    lab(ax, 1.5, 5.85, "$PDD_3$", ha="left")
    ax.set_xlim(0, 12); ax.set_ylim(0, 6.2)
    save(fig, FIG / "fig_mech_chain.png")

def fig_mech_q3():
    use_style("A", body_pt=12, font="SimSun")
    mathfont()
    fig, ax = new_fig(12.0, 5.4)
    x0, x1 = 0.8, 10.8
    days = 6
    dw = (x1 - x0) / days

    ya, yb = 3.1, 4.9
    ax.plot([x0, x1], [ya, ya], color=DARK, lw=0.8)
    p15 = [0.45, 0.55, 0.7, 0.82, 0.85, 0.8]
    pdd = [0.3, 0.42, 0.55, 0.78, 0.9, 0.86]
    for k in range(days):
        xs = [x0 + k * dw, x0 + (k + 1) * dw]
        ax.plot(xs, [ya + p15[k] * 1.6] * 2, color=BLUE, lw=1.6)
        ax.plot(xs, [ya + pdd[k] * 1.6] * 2, color=ORANGE, lw=1.6)
        if p15[k] >= 0.68 and pdd[k] >= 0.7:
            ax.add_patch(Rectangle((xs[0], ya), dw, 1.75, fc=L_AMBER, ec="none", alpha=0.7,
    zorder=0))
    ax.plot([x0, x1], [ya + 0.68 * 1.6] * 2, color=BLUE, lw=0.8, ls="--")
    ax.plot([x0, x1], [ya + 0.7 * 1.6 + 0.05] * 2, color=ORANGE, lw=0.8, ls=":")
    lab(ax, x0 - 0.1, ya + 0.68 * 1.6, "$c_1$", ha="right", c=BLUE)
    lab(ax, x0 - 0.1, ya + 0.7 * 1.6 + 0.2, "$c_2$", ha="right", c=ORANGE)

    yc = 1.35
    ax.plot([x0, x1], [yc, yc], color=DARK, lw=0.8)
    t = np.linspace(0, days, 600)
    T = 0.55 * np.sin(2 * np.pi * (t - 0.375)) + 0.1
    ax.plot(x0 + t * dw, yc + T, color=ORANGE, lw=1.2)
    for k in range(days):
        ax.plot([x0 + k * dw] * 2, [0.6, ya + 1.75], color="#C8C8C8", lw=0.5, zorder=0)
    pk = x0 + (4 + 0.625) * dw
    trig = pk + 15 / 24 * dw
    ax.scatter([pk], [yc + 0.65], s=14, color=ORANGE, zorder=5)
    arr(ax, (pk, yc + 0.95), (trig, yc + 0.95), c=DARK, lw=0.8, ms=7)
    lab(ax, (pk + trig) / 2, yc + 1.18, "$\\tau$")
    ax.add_patch(Polygon([(trig, yc + 0.05), (trig - 0.12, yc - 0.25), (trig + 0.12, yc -
    0.25)], closed=True, fc=RED if False else ORANGE, ec=DARK, lw=0.6, zorder=5))

    t0 = x0 + 2.5 * dw
    ax.plot([t0, t0], [0.45, ya + 1.85], color=GREEN, lw=1.0, ls="-.")
    arr(ax, (t0, 0.45), (trig, 0.45), c=GREEN, lw=0.9, ms=7)
    lab(ax, t0, ya + 2.05, "$t_0$", c=GREEN)
    lab(ax, (t0 + trig) / 2, 0.2, "提前量", c=GREEN)
    lab(ax, x1 + 0.1, ya + 0.86 * 1.6, "$P_{15}$", ha="left", c=BLUE)
    lab(ax, x1 + 0.1, ya + 0.86 * 1.6 - 0.35, "$PDD_3$", ha="left", c=ORANGE)
    lab(ax, x1 + 0.1, yc, "$T'$", ha="left", c=ORANGE)
    ax.set_xlim(0, 12.4); ax.set_ylim(0, 5.4)
    save(fig, FIG / "fig_mech_q3.png")

def fig_mech_q4():
    use_style("A", body_pt=12, font="SimSun")
    mathfont()
    fig, ax = new_fig(11.0, 5.0)
    th = np.deg2rad(24)
```

```
    L = 7.2
    p0 = np.array([0.8, 0.8 + L * np.tan(th)])
    p1 = np.array([0.8 + L, 0.8])
    ax.fill([p0[0], p1[0], p1[0], p0[0]], [p0[1], p1[1], 0.3, 0.3], color="#E8E1D3", zorder=1)
    ax.plot([p0[0], p1[0]], [p0[1], p1[1]], color=DARK, lw=1.0, zorder=2)
    u = (p1 - p0) / np.linalg.norm(p1 - p0)
    nrm = np.array([u[1] * -1, u[0]]) * -1
    nrm = np.array([-u[1], u[0]]) if (-u[1]) * 0 + u[0] > 0 else nrm
    c = p0 + u * 3.6 + np.array([-u[1], u[0]]) * -1 * -0.0

    hn = np.array([u[1], -u[0]]) * -1
    hn = np.array([-u[1], u[0]])
    if hn[1] < 0:
        hn = -hn
    w = 1.6
    q = [c - u * w / 2, c + u * w / 2, c + u * w / 2 + hn * 0.75, c - u * w / 2 + hn * 0.75]
    ax.add_patch(Polygon(q, closed=True, fc="#C9A57A", ec=DARK, lw=0.9, zorder=3))
    cm = c + hn * 0.38
    arr(ax, cm, cm + u * 1.6, c=BLUE, lw=1.3)
    arr(ax, cm - u * 0.05, cm - u * 1.4, c=ORANGE, lw=1.3)
    arr(ax, cm + hn * 0.9, cm + hn * 0.9 - u * 1.0, c=GREEN, lw=1.3)
    arr(ax, cm, cm + np.array([0, -1.3]), c=DARK, lw=1.0, ms=8)
    lab(ax, *(cm + u * 2.0 - hn * 0.35), "$g\\sin\\theta$", c=BLUE)
    lab(ax, *(cm - u * 1.75 + hn * 0.05), "$\\mu g\\cos\\theta$", c=ORANGE)
    lab(ax, *(cm + hn * 1.25 - u * 0.6), "$g v^2/(\\xi h)$", c=GREEN)
    lab(ax, cm[0] + 0.25, cm[1] - 1.45, "$g$", ha="left")
    lab(ax, *(c + hn * 0.55 + u * 0.55), "$h$")

    ang = Wedge(p1, 1.0, 180 - 24, 180, fc="none", ec=DARK, lw=0.7)
    ax.add_patch(ang)
    lab(ax, p1[0] - 1.3, p1[1] + 0.22, "$\\theta$")

    bx = 8.7
    for k, (nm, fc, mu, xi) in enumerate([("冰岩崩", L_BLUE, "0.10", "600"), ("泥石流", "#E6D3B8
    ", "0.06", "400"),
                                          ("山洪", "#DDEBF4", "0.005", "800")]):
        y = 3.9 - k * 1.25
        ax.add_patch(Rectangle((bx, y - 0.42), 2.0, 0.84, fc=fc, ec=S_EDGE, lw=0.8))
        lab(ax, bx + 1.0, y + 0.18, nm)
        lab(ax, bx + 1.0, y - 0.18, f"$\\mu$={mu}  $\\xi$={xi}", fs=8)
        if k < 2:
            arr(ax, (bx + 1.0, y - 0.45), (bx + 1.0, y - 0.8), c=S_EDGE, lw=0.8, ms=7)
    ax.set_xlim(0, 11.0); ax.set_ylim(0, 5.0)
    save(fig, FIG / "fig_mech_q4.png")

def fig_frames(nums):
    st = flow_style()
    for k, sp in enumerate(specs(nums)):
        fig, ax = new_fig(15.6, 3.0)
        spine(ax, st, sp, 1.5, 0.1, 15.5, 0.92)
        save(fig, FIG / f"fig_frame_q{k + 1}.png")
        plt.close(fig)

def fig_mech_q1():
    use_style("A", body_pt=12, font="SimSun")
    mathfont()
    fig, ax = new_fig(13.0, 5.2)
    for k, x0 in enumerate([0.3, 6.8]):
        x = np.linspace(x0, x0 + 5.9, 200)
        g = 0.5 + 2.4 * (np.abs(x - (x0 + 2.95)) / 2.95) ** 1.6
        ax.fill_between(x, 0.3, g, color="#E8E1D3", zorder=1)
        ax.plot(x, g, color=DARK, lw=0.9, zorder=2)
        ax.add_patch(Rectangle((x0 + 2.55, 0.42), 0.8, 0.12, fc=L_BLUE, ec=BLUE, lw=0.6,
    zorder=3))
        if k == 0:
            ax.add_patch(Circle((x0 + 1.15, 4.6), 0.28, fc=L_AMBER, ec=AMBER, lw=1.0))
            for t in np.linspace(0, 2 * np.pi, 9)[:-1]:
                ax.plot([x0 + 1.15 + 0.36 * np.cos(t), x0 + 1.15 + 0.5 * np.cos(t)], [4.6 +
    0.36 * np.sin(t), 4.6 + 0.5 * np.sin(t)], color=AMBER, lw=0.9)
            arr(ax, (x0 + 1.5, 4.25), (x0 + 2.1, 2.3), c=AMBER, lw=1.0, ms=8)
            for cx in [x0 + 2.2, x0 + 3.7]:
                arr(ax, (cx, 1.0), (cx, 3.2), c=ORANGE, lw=1.0, ms=8, rad=0.3)
            arr(ax, (x0 + 1.6, 1.25), (x0 + 0.8, 2.25), c=GREEN, lw=1.0, ms=8)
            arr(ax, (x0 + 4.3, 1.25), (x0 + 5.1, 2.25), c=GREEN, lw=1.0, ms=8)
            for cx in [x0 + 2.75, x0 + 3.15]:
                ax.plot([cx, cx], [0.58, 1.1], color=BLUE, lw=0.8, ls=":")
```

```
            lab(ax, x0 + 1.8, 4.7, "$R_n$↑", ha="left")
            lab(ax, x0 + 3.25, 1.05, "$LE$", ha="left", c=BLUE)
            lab(ax, x0 + 2.95, 3.45, "对流", c=ORANGE)
            lab(ax, x0 + 0.35, 2.55, "谷风", ha="left", c=GREEN)
            lab(ax, x0 + 0.05, 5.0, "(a)", ha="left", fs=9)
        else:
            for sx, sy in [(x0 + 0.5, 4.8), (x0 + 1.3, 4.5), (x0 + 4.9, 4.7), (x0 + 5.5, 4.4)
    ]:
                ax.plot([sx], [sy], marker="*", ms=4, color=SLATE_C)
            for cx in [x0 + 1.1, x0 + 4.8]:
                arr(ax, (cx, 2.1), (cx, 3.4), c=ORANGE, lw=0.9, ms=7, ls="--")
            arr(ax, (x0 + 0.8, 2.55), (x0 + 2.2, 0.75), c=BLUE, lw=1.1, ms=8)
            arr(ax, (x0 + 5.1, 2.55), (x0 + 3.7, 0.75), c=BLUE, lw=1.1, ms=8)
            ax.add_patch(Rectangle((x0 + 1.9, 0.62), 2.1, 0.55, fc=L_BLUE, ec="none", alpha
    =0.6, zorder=1))
            ax.plot([x0 + 1.5, x0 + 4.4], [1.25, 1.25], color=DARK, lw=0.8, ls="--")

            px, py = x0 + 2.95, 2.0
            ax.plot([px - 0.5, px - 0.5], [py, py + 1.5], color=DARK, lw=0.7)
            ax.plot([px - 0.5, px + 0.4], [py, py], color=DARK, lw=0.7)
            ax.plot([px - 0.35, px + 0.05, px - 0.2], [py + 0.05, py + 0.55, py + 1.4], color=
    ORANGE, lw=1.3)
            lab(ax, px + 0.5, py - 0.05, "$T$", ha="left")
            lab(ax, px - 0.6, py + 1.5, "$z$", ha="right")
            lab(ax, x0 + 1.3, 3.65, "$R_{ld}$↓", ha="left", c=ORANGE)
            lab(ax, x0 + 4.45, 1.3, "逆温层", ha="left")
            lab(ax, x0 + 0.3, 1.9, "冷泄流", ha="left", c=BLUE)
            lab(ax, x0 + 0.05, 5.0, "(b)", ha="left", fs=9)
    ax.set_xlim(0, 13.0); ax.set_ylim(0, 5.2)
    save(fig, FIG / "fig_mech_q1.png")

def fig_mech_q2():
    use_style("A", body_pt=12, font="SimSun")
    mathfont()
    fig, ax = new_fig(11.0, 5.6)
    ox, oy, W, H = 1.0, 0.7, 6.4, 4.4
    arr(ax, (ox, oy), (ox + W, oy), lw=0.9, ms=8)
    arr(ax, (ox, oy), (ox, oy + H), lw=0.9, ms=8)
    c1x, c2y = ox + 3.4, oy + 2.6
    ax.add_patch(Rectangle((c1x, c2y), W - 3.4 - 0.2, H - 2.6 - 0.2, fc=L_ORANGE, ec="none",
    alpha=0.8))
    ax.plot([c1x, c1x], [oy, oy + H - 0.2], color=BLUE, lw=1.0, ls="--")
    ax.plot([ox, ox + W - 0.2], [c2y, c2y], color=ORANGE, lw=1.0, ls="--")
    tr = np.array([[1.4, 1.1], [2.1, 1.3], [2.9, 1.6], [3.6, 1.5], [4.2, 1.9], [4.7, 2.4],
    [4.9, 2.9], [5.15, 3.5]])
    tr[:, 0] += ox - 1.0
    tr[:, 1] += oy - 0.7
    for a_, b_ in zip(tr[:-1], tr[1:]):
        arr(ax, a_ + (b_ - a_) * 0.12, b_ - (b_ - a_) * 0.12, c=SLATE_C, lw=0.9, ms=7)
    ax.scatter(tr[:-1, 0], tr[:-1, 1], s=14, color=SLATE_C, zorder=4)
    ax.scatter([tr[-1, 0]], [tr[-1, 1]], s=60, marker="*", color=RED_C, zorder=5)
    rng = np.random.default_rng(3)
    pts = np.c_[rng.uniform(ox + 0.3, ox + W - 0.4, 40), rng.uniform(oy + 0.2, oy + H - 0.4,
    40)]
    pts = pts[~((pts[:, 0] > c1x) & (pts[:, 1] > c2y))]
    ax.scatter(pts[:, 0], pts[:, 1], s=5, color="#C8C8C8", zorder=2)
    lab(ax, ox + W + 0.1, oy, "$P_{15}$", ha="left", c=BLUE)
    lab(ax, ox, oy + H + 0.2, "$PDD_3$", c=ORANGE)
    lab(ax, c1x, oy - 0.3, "$c_1$", c=BLUE)
    lab(ax, ox - 0.15, c2y, "$c_2$", ha="right", c=ORANGE)
    lab(ax, c1x + 1.45, c2y + 1.4, "触发区")
    lab(ax, tr[-1, 0] + 0.2, tr[-1, 1] + 0.05, "事件", ha="left", c=RED_C)

    bx = 8.3
    ax.add_patch(Rectangle((bx, 0.9), 0.9, 2.4, fc="white", ec=DARK, lw=0.9))
    ax.add_patch(Rectangle((bx, 0.9), 0.9, 1.75, fc=L_BLUE, ec="none"))
    ax.plot([bx - 0.1, bx + 1.0], [0.9 + 1.55, 0.9 + 1.55], color=BLUE, lw=0.9, ls="--")
    lab(ax, bx + 0.45, 0.55, "蓄水", c=BLUE)
    tx = 9.9
    ax.add_patch(FancyBboxPatch((tx, 1.25), 0.36, 2.1, boxstyle="round,pad=0,rounding_size
    =0.18", fc="white", ec=DARK, lw=0.9))
    ax.add_patch(Circle((tx + 0.18, 1.15), 0.3, fc=L_ORANGE, ec=DARK, lw=0.9))
    ax.add_patch(Rectangle((tx + 0.08, 1.15), 0.2, 1.75, fc=L_ORANGE, ec="none"))
    ax.plot([tx - 0.15, tx + 0.55], [1.15 + 1.5, 1.15 + 1.5], color=ORANGE, lw=0.9, ls="--")
    lab(ax, tx + 0.18, 0.55, "热力", c=ORANGE)
    ax.set_xlim(0, 11.0); ax.set_ylim(0, 5.6)
```

```
    save(fig, FIG / "fig_mech_q2.png")

def fig_algo_q4():
    st = flow_style()
    fig, ax = new_fig(9.0, 11.2)
    cx = 3.6
    items = [("读入 DEM 与冰川", "rect"), ("填洼求坡度与汇水", "rect"), ("释放区布设 N 个粒子", "rect
    "),
             ("网格累加体积求流深 h", "rect"), ("判定阶段并取 μ、ξ", "rect"), ("更新速度与位置", "rect
    "),
             ("泥石流段沿程侵蚀", "rect"), ("全部停止或超时？", "judge"), ("输出流深、时间、堆积", "
    rect")]
    y = 10.6
    bx = []
    for t, kind in items:
        h = 0.9 if kind == "judge" else LH + 0.3
        w = chars(t) * CW + (1.6 if kind == "judge" else 0.6)
        box(ax, cx - w / 2, y - h / 2, w, h, t, st, kind=kind, fill=W2 if kind == "judge" else
     W1, per_line=99)
        bx.append((cx, y, w, h))
        y -= h / 2 + 0.55 + (0.9 if kind == "judge" else LH + 0.3) / 2 if False else 1.18 if
    kind != "judge" else 1.45
    for (x1, y1, w1, h1), (x2, y2, w2, h2) in zip(bx[:-1], bx[1:]):
        arrow(ax, (cx, y1 - h1 / 2 - 0.03), (cx, y2 + h2 / 2 + 0.03), st)

    jx, jy, jw, jh = bx[7]
    tx_, ty_, tw, th = bx[3]
    rx = cx + max(w for _, _, w, _ in bx) / 2 + 0.4
    line(ax, [(cx + jw / 2, jy), (rx, jy), (rx, ty_)], st)
    arrow(ax, (rx, ty_), (cx + tw / 2 + 0.03, ty_), st)
    ax.text(cx + jw / 2 + 0.12, jy + 0.18, "否", fontsize=FS, color=st["text"])
    ax.text(cx + 0.12, jy - jh / 2 - 0.3, "是", fontsize=FS, color=st["text"])
    ax.set_xlim(0, 9.0); ax.set_ylim(0, 11.2)
    save(fig, FIG / "fig_algo_q4.png")

if __name__ == "__main__":
    import json
    from common import RES
    q2 = json.load(open(RES / "q2_summary.json", encoding="utf-8"))
    q3 = json.load(open(RES / "q3_summary.json", encoding="utf-8"))
    q4 = json.load(open(RES / "q4_summary.json", encoding="utf-8")) if (RES / "q4_summary.json
    ").exists() else {}
    cr = q2["critical"]
    mc = q4.get("mc", {})
    nums = {"auc": f"分块 AUC {q2['spatial']['随机森林']['AUC_spatial_block']:.2f}",
            "crit": f"P15≥{cr['c1']:.1f} mm\nPDD3≥{cr['c2']:.1f} °C·d",
            "lead": f"提前 {int(min(q3['lead_time_h'].values()))}–{int(max(q3['lead_time_h'].
    values()))} h",
            "ci": f"95% 区间约 {q3['width_h_by_lead']['12']:.0f} h",
            "space": (f"停积 {mc['runout_q'][0]:.1f}–{mc['runout_q'][2]:.1f} km" if mc else "停
    积范围"),
            "tport": f"口岸 {q4.get('t_port_min', 0):.1f} min" if q4 else "到达口岸",
            "mc": (f"到达口岸 {mc['p_reach_port'] * 100:.0f}%" if mc else "到达概率"),
            "top5": "VPD DTR 逆温 长波 P30",
            "far": f"误报日比例 {q2['critical']['FAR_daily'] * 100:.2f}%" if 'FAR_daily' in q2['
    critical'] else "误报日比例 1.36%",
            "tau": f"τ 似然标定 {q3['tau_hat_h']:.0f} h",
            "beta": f"Beta({q3['q_beta'][0]},{q3['q_beta'][1]}) 后验",
            "pstar": f"预警阈值 {q3['p_star']:.2f}"}
    fig_flow(nums)
    fig_frames(nums)
    fig_mech_q1()
    fig_mech_q2()
    fig_algo_q4()
    fig_mech_chain()
    fig_mech_q3()
    fig_mech_q4()
    print("ok", nums)
```

B.12 figs_gis.py GIS 专题图

```
import sys
import glob
import json
import pickle
```

```
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import matplotlib.font_manager as fm
from matplotlib.colors import ListedColormap, LightSource, BoundaryNorm,
    LinearSegmentedColormap
from matplotlib.lines import Line2D
from matplotlib.patches import Patch, Polygon
import rasterio
from rasterio.windows import from_bounds
from rasterio.warp import transform
import netCDF4
from scipy import ndimage

for f in glob.glob("/home/hero/.fonts/*"):
    fm.fontManager.addfont(f)
sys.path.insert(0, str(__import__("pathlib").Path(__file__).parent / "shumo_tools"))
from shumo_sci import use_sci, panel, tag, top_legend, soft_cmap, CM, BLUE, RED, TEAL, CYAN,
    SLATE, INK, FILL
from common import RES, FIG, DATA, EXT, PORT, STATION

use_sci(body_pt=12, font="SimSun")
AMBER = "#D9B06A"
LC_NAME = ["耕地", "林地", "灌丛", "高寒草原", "高寒草甸", "水体", "裸地", "建设用地", "湿地", "冰雪"]
LC_COL = ["#E8D9A0", "#7FB79A", "#A9C79A", "#D9CFA0", "#B9D3A8", "#8FBFD9", "#D8CFC4", "#
    D98C7A", "#9DC3C1", "#F4F7FA"]

def save(fig, name):
    fig.savefig(FIG / f"{name}.png", dpi=300)
    plt.close(fig)

def north_scale(ax, lat0, km=10, loc=(0.06, 0.08), c=INK):
    x0, x1 = ax.get_xlim()
    y0, y1 = ax.get_ylim()
    sx = x0 + loc[0] * (x1 - x0)
    sy = y0 + loc[1] * (y1 - y0)
    dlon = km / (111.32 * np.cos(np.deg2rad(lat0)))
    ax.plot([sx, sx + dlon], [sy, sy], color=c, lw=2.2, solid_capstyle="butt", zorder=6)
    ax.plot([sx, sx + dlon / 2], [sy, sy], color="white", lw=1.0, solid_capstyle="butt",
    zorder=7)
    ax.text(sx + dlon / 2, sy + 0.025 * (y1 - y0), f"{km} km", ha="center", va="bottom",
    fontsize=9, color=c, zorder=7)
    nx_, ny_ = x1 - 0.08 * (x1 - x0), y1 - 0.2 * (y1 - y0)
    h = 0.1 * (y1 - y0)
    w = 0.025 * (x1 - x0)
    ax.add_patch(Polygon([(nx_, ny_ + h), (nx_ - w, ny_), (nx_, ny_ + 0.3 * h)], closed=True,
    fc=c, ec=c, zorder=7))
    ax.add_patch(Polygon([(nx_, ny_ + h), (nx_ + w, ny_), (nx_, ny_ + 0.3 * h)], closed=True,
    fc="white", ec=c, lw=0.6, zorder=7))
    ax.text(nx_, ny_ + h + 0.02 * (y1 - y0), "N", ha="center", va="bottom", fontsize=9, color=
    c, zorder=7)

def fig_gis_lc():
    box = (85.05, 28.2, 85.65, 29.0)
    with rasterio.open(DATA / "d5" / "jilong_LUCD_2023.tif") as r:
        win = from_bounds(*box, r.transform)
        lc = r.read(1, window=win, out_shape=(600, 450))
    with rasterio.open(EXT / "cop30_N28E085.tif") as r:
        win = from_bounds(*box, r.transform)
        z = r.read(1, window=win, out_shape=(600, 450)).astype(float)
    hs = LightSource(315, 40).hillshade(z, vert_exag=1.5, dx=0.6 / 450 * 98000, dy=0.8 / 600 *
     110574)
    hz = pd.read_csv(RES / "q2_hazard_points.csv")
    B = pickle.load(open(RES / "q4_base.pkl", "rb"))
    with rasterio.open(DATA / "d7" / "DEM.tif") as r:
        z2 = r.read(1, out_shape=(574, 908)).astype(float)
        bb = r.bounds
    z2[z2 < -1000] = np.nan
    gy, gx = np.gradient(np.nan_to_num(z2, nan=np.nanmean(z2)), 61.4, 54.4)
    slope = np.degrees(np.arctan(np.hypot(gx, gy)))
    fig, (a, b) = plt.subplots(1, 2, figsize=(16 * CM, 8.6 * CM), gridspec_kw={"width_ratios":
     [0.78, 1.32]})
    ext = [box[0], box[2], box[1], box[3]]
    cm = ListedColormap(LC_COL)
```

```
    lcm = np.ma.masked_equal(lc, 0)
    a.imshow(lcm, extent=ext, cmap=cm, norm=BoundaryNorm(np.arange(0.5, 11.5), 10),
    interpolation="nearest")
    a.imshow(hs, extent=ext, cmap="gray", alpha=0.35)
    a.scatter(hz.lon, hz.lat, s=7, color=INK, edgecolor="white", linewidth=0.3, zorder=4)
    a.scatter([STATION[1]], [STATION[0]], marker="^", s=40, color=RED, edgecolor="white",
    linewidth=0.4, zorder=5)
    a.scatter([PORT[1]], [PORT[0]], marker="*", s=70, color=RED, edgecolor="white", linewidth
    =0.4, zorder=5)
    a.set_xlim(box[0], box[2]); a.set_ylim(box[1], box[3])
    a.set_xticks([85.1, 85.3, 85.5]); a.set_yticks([28.3, 28.5, 28.7, 28.9])
    a.set_xlabel("经度 / °E"); a.set_ylabel("纬度 / °N")
    north_scale(a, 28.6, km=10)
    panel(a); tag(a, "a")
    e2 = [bb.left, bb.right, bb.bottom, bb.top]
    im = b.imshow(slope, extent=e2, cmap=soft_cmap(AMBER, "#8C6A2E"), vmin=0, vmax=60)
    b.imshow(LightSource(315, 40).hillshade(np.nan_to_num(z2, nan=2000), vert_exag=1.2, dx
    =54.4, dy=61.4), extent=e2, cmap="gray", alpha=0.3)
    sys.path.insert(0, str(__import__("pathlib").Path(__file__).parent))
    from sim import Terrain
    T = Terrain(2)
    riv = np.where(T.area > 20, 1.0, np.nan)
    b.imshow(riv, extent=e2, cmap=ListedColormap([BLUE]), interpolation="nearest", zorder=3)
    glon, glat = B["glacier_ll"]
    b.scatter(glon[::4], glat[::4], s=0.2, color="#DDEFFC", zorder=3)
    P, tr2 = B["P"], B["tr"]
    b.plot(tr2[2] + (P[:, 1] + 0.5) * tr2[0], tr2[5] + (P[:, 0] + 0.5) * tr2[4], color=RED, lw
    =1.3, zorder=4)
    b.scatter([PORT[1]], [PORT[0]], marker="*", s=80, color=RED, edgecolor="white", linewidth
    =0.5, zorder=5)
    b.set_xlim(e2[0], e2[1]); b.set_ylim(e2[2], e2[3])
    b.set_xticks([85.2, 85.3, 85.4, 85.5, 85.6]); b.set_yticks([28.3, 28.4, 28.5])
    b.set_xlabel("经度 / °E")
    cb = fig.colorbar(im, ax=b, fraction=0.035, pad=0.02)
    cb.ax.tick_params(direction="in"); cb.set_label("坡度 / °")
    north_scale(b, 28.4, km=5)
    panel(b); tag(b, "b")
    hs_ = [Patch(color=c, label=n) for c, n in zip(LC_COL, LC_NAME) if n not in ("建设用地", "湿
    地", "耕地")]
    hs_[-1] = Patch(facecolor=LC_COL[-1], edgecolor=SLATE, label="冰雪")
    hs_ += [Line2D([], [], color=BLUE, lw=1.5, label="河网"), Line2D([], [], color=RED, lw=1.5,
     label="流路")]
    top_legend(fig, hs_, ncol=5, rows=2, handlelength=1.1, columnspacing=0.8, handletextpad
    =0.3)
    save(fig, "fig_gis_lc")

def fig_gis_glacier():
    ds = netCDF4.Dataset(glob.glob(str(DATA / "d4" / "*.nc"))[0])
    x, y = ds["x"][:], ds["y"][:]
    X, Y = np.meshgrid(x, y)
    lon, lat = transform("EPSG:32645", "EPSG:4326", X.ravel().tolist(), Y.ravel().tolist())
    lon, lat = np.array(lon).reshape(X.shape), np.array(lat).reshape(X.shape)
    m = ds["glacier_mask"][:] > 0
    th = np.ma.masked_where(~m, np.ma.filled(ds["thickness"][:], np.nan))
    v = np.ma.masked_where(~m, np.ma.filled(ds["v"][:], np.nan))
    rel = m & (np.ma.filled(ds["slope"][:], 0) > 30) & (np.ma.filled(ds["dem"][:], 0) > 5000)
    dem = np.ma.filled(ds["dem"][:], np.nan)
    f = sorted(glob.glob(str(DATA / "d3g" / "dhdt_GCS__*2015*.tif")))[0]
    with rasterio.open(f) as r:
        win = from_bounds(lon.min(), lat.min(), lon.max(), lat.max(), r.transform)
        dh = r.read(1, window=win).astype(float)
        tr = r.window_transform(win)
    dh[(np.abs(dh) > 50) | (dh == r.nodata)] = np.nan
    e3 = [tr.c, tr.c + tr.a * dh.shape[1], tr.f + tr.e * dh.shape[0], tr.f]
    fig, axes = plt.subplots(1, 3, figsize=(16 * CM, 7.0 * CM), sharey=True)
    hs = LightSource(315, 40).hillshade(np.nan_to_num(dem, nan=np.nanmean(dem)), vert_exag=1,
    dx=30, dy=30)
    for ax in axes:
        ax.pcolormesh(lon, lat, hs, cmap="gray", shading="auto", alpha=0.6)
    im1 = axes[0].pcolormesh(lon, lat, th, cmap=soft_cmap(BLUE, "#1F3B5A"), shading="auto",
    vmin=0, vmax=150)
    axes[0].contour(lon, lat, rel.astype(float), levels=[0.5], colors=RED, linewidths=0.8)
    im2 = axes[1].pcolormesh(lon, lat, v, cmap=soft_cmap(TEAL, "#2F5E46"), shading="auto",
    vmin=0, vmax=40)
    div = LinearSegmentedColormap.from_list("d", [RED, "#FFFFFF", BLUE])
    im3 = axes[2].imshow(dh, extent=e3, cmap=div, vmin=-1.5, vmax=1.5, alpha=0.95)
```

```
    axes[2].contour(lon, lat, m.astype(float), levels=[0.5], colors=INK, linewidths=0.6)
    for ax, im, lab, tg in zip(axes, [im1, im2, im3], ["冰厚 / m", "流速 / m·a^-1", "高程变化 /
    m·a^-1"], "abc"):
        ax.set_xlim(lon.min(), lon.max()); ax.set_ylim(lat.min(), lat.max())
        ax.set_xticks([85.52, 85.54])
        cb = fig.colorbar(im, ax=ax, orientation="horizontal", fraction=0.06, pad=0.2)
        cb.ax.tick_params(direction="in", labelsize=10); cb.set_label(lab, fontsize=11)
        ax.tick_params(labelsize=10)
        panel(ax); tag(ax, tg)
    axes[0].set_ylabel("纬度 / °N")
    north_scale(axes[0], 28.29, km=1)
    fig.subplots_adjust(left=0.1, right=0.98, top=0.92, bottom=0.16, wspace=0.12)
    save(fig, "fig_gis_glacier")

def fig_gis_zone():
    B = pickle.load(open(RES / "q4_base.pkl", "rb"))
    tr = B["tr"]
    ny, nx = B["z"].shape
    ext = [tr[2], tr[2] + tr[0] * nx, tr[5] + tr[4] * ny, tr[5]]
    hs = LightSource(315, 40).hillshade(B["z"], vert_exag=1.5, dx=B["dx"], dy=B["dy"])
    t = ndimage.minimum_filter(np.where(np.isfinite(B["tarr"]), B["tarr"], np.inf), 3) / 60
    h = B["hmax"]
    cls = np.full(t.shape, np.nan)
    ok = (h > 0.5) & np.isfinite(t)
    for k, (lo, hi) in enumerate([(0, 5), (5, 10), (10, 15), (15, 1e9)]):
        cls[ok & (t >= lo) & (t < hi)] = k
    dam = np.where(B["dep"] > 5, 1.0, np.nan)
    fig, ax = plt.subplots(figsize=(16 * CM, 7.6 * CM))
    ax.imshow(hs, extent=ext, cmap="gray", vmin=0, vmax=1.2)
    cz = ListedColormap([RED, AMBER, TEAL, BLUE])
    ax.imshow(cls, extent=ext, cmap=cz, vmin=-0.5, vmax=3.5, interpolation="nearest", alpha
    =0.95)
    ax.imshow(dam, extent=ext, cmap=ListedColormap([INK]), interpolation="nearest")
    P, s = B["P"], B["s"] / 1000
    plon = tr[2] + (P[:, 1] + 0.5) * tr[0]
    plat = tr[5] + (P[:, 0] + 0.5) * tr[4]
    for km in [0, 5, 10, 15, 20]:
        k = np.argmin(np.abs(s - km))
        ax.plot(plon[k], plat[k], marker="o", ms=4, mfc="white", mec=INK, mew=0.8, zorder=5)
        ax.text(plon[k] + 0.004, plat[k] + 0.004, f"{km}", fontsize=9, color=INK, zorder=6)
    ax.scatter([PORT[1]], [PORT[0]], marker="*", s=110, color=RED, edgecolor="white",
    linewidth=0.6, zorder=6)
    ax.set_xlim(85.34, 85.57); ax.set_ylim(28.255, 28.355)
    ax.set_xticks([85.35, 85.40, 85.45, 85.50, 85.55]); ax.set_yticks([28.26, 28.28, 28.30,
    28.32, 28.34])
    ax.set_xlabel("经度 / °E"); ax.set_ylabel("纬度 / °N")
    north_scale(ax, 28.3, km=2, loc=(0.78, 0.08))
    panel(ax)
    hs_ = [Patch(color=c, label=l) for c, l in zip([RED, AMBER, TEAL, BLUE], ["5 min 内", "5–
    10 min", "10–15 min", "15 min 后"])] +\
          [Patch(color=INK, label="堆积超 5 m"), Line2D([], [], marker="*", ls="", ms=9, color=
    RED, label="口岸")]
    top_legend(fig, hs_, ncol=6, rows=1, handlelength=1.1, columnspacing=0.9, handletextpad
    =0.3)
    save(fig, "fig_gis_zone")

def fig_gis_3d():
    from mpl_toolkits.mplot3d import Axes3D
    B = pickle.load(open(RES / "q4_base.pkl", "rb"))
    tr = B["tr"]
    z = B["z"]
    st = 3
    zz = z[::st, ::st]
    ny, nx = zz.shape
    lon = tr[2] + (np.arange(nx) * st + 0.5) * tr[0]
    lat = tr[5] + (np.arange(ny) * st + 0.5) * tr[4]
    LON, LAT = np.meshgrid(lon, lat)
    tcm = LinearSegmentedColormap.from_list("t", ["#7FB79A", "#C9D6A3", "#D9C9A0", "#B8A48A",
    "#F4F7FA"])
    rgb = LightSource(315, 40).shade(zz, cmap=tcm, vert_exag=1.2, blend_mode="soft",
                                     vmin=1500, vmax=6500, dx=B["dx"] * st, dy=B["dy"] * st)
    fig = plt.figure(figsize=(16 * CM, 9.0 * CM))
    ax = fig.add_axes([0, 0, 1, 1], projection="3d", computed_zorder=False)
    ax.plot_surface(LON, LAT, zz, facecolors=rgb, rstride=1, cstride=1, linewidth=0,
    antialiased=False, shade=False)
    P = B["P"]
```

```
    plon = tr[2] + (P[:, 1] + 0.5) * tr[0]
    plat = tr[5] + (P[:, 0] + 0.5) * tr[4]
    pz = B["z"][P[:, 0], P[:, 1]] + 120
    ax.plot(plon, plat, pz, color=RED, lw=2.0, zorder=10)
    glon, glat = B["glacier_ll"]
    gi = ((tr[5] - np.array(glat)) / abs(tr[4])).astype(int)
    gj = ((np.array(glon) - tr[2]) / tr[0]).astype(int)
    ok = (gi >= 0) & (gi < z.shape[0]) & (gj >= 0) & (gj < z.shape[1])
    ax.scatter(np.array(glon)[ok][::6], np.array(glat)[ok][::6], z[gi[ok], gj[ok]][::6] + 80,
    s=0.6, color=BLUE, depthshade=False, zorder=9)
    pi = int((tr[5] - PORT[0]) / abs(tr[4])); pj = int((PORT[1] - tr[2]) / tr[0])
    ax.scatter([PORT[1]], [PORT[0]], [z[pi, pj] + 300], marker="*", s=140, color=RED,
    edgecolor="white", depthshade=False, zorder=11)
    ax.set_box_aspect((1.6, 1.0, 0.5))
    ax.view_init(elev=42, azim=-70)
    ax.set_xlabel("经度 / °E", labelpad=6); ax.set_ylabel("纬度 / °N", labelpad=6); ax.
    set_zlabel("高程 / m", labelpad=6)
    ax.tick_params(labelsize=9)
    ax.xaxis.pane.fill = ax.yaxis.pane.fill = ax.zaxis.pane.fill = False
    save(fig, "fig_gis_3d")

if __name__ == "__main__":
    fig_gis_lc()
    fig_gis_glacier()
    fig_gis_zone()
    fig_gis_3d()
    print("ok")
```

B.13　cmp_q4b.py 问题四：建模方式对比

```
import json, time
import numpy as np
import pandas as pd
from common import RES
from sim import Terrain, run, release_cells
from q4 import chain_tools

T = Terrain(2)
src = release_cells(T)
P, s, tree = chain_tools(T)
V0 = 0.2 * src[2].sum() * 900.0
cases = {"分阶段参数与侵蚀": {}, "单一泥石流参数": {"mu_ice": 0.06, "xi_ice": 400.0, "mu_fl": 0.06,
     "xi_fl": 400.0},
         "不计侵蚀": {"bulk": 1.0}, "单一参数且不计侵蚀": {"mu_ice": 0.06, "xi_ice": 400.0, "mu_fl"
    : 0.06, "xi_fl": 400.0, "bulk": 1.0}}
rows = []
for nm, p in cases.items():
    t = time.time()
    R = run(T, {"V0": V0, "N": 800, "tmax": 2400, **p}, seed=1, src=src)
    dd, kk = tree.query(np.c_[R["x"], R["y"]])
    ch = s[kk] / 1000
    rows.append({"method": nm, "t_port_min": R["t_port"] / 60, "dep_median_km": float(np.
    median(ch)),
                 "exited": R["exited"], "vol_final": R["vol_final"] / 1e6, "vmax": float(R["
    vmax"].max()), "runtime_s": time.time() - t})
D = pd.DataFrame(rows)
D.to_csv(RES / "cmp_q4.csv", index=False)
print(D.round(3).to_string())
```